多源公开数据约束下秦皇岛旅游客流预测与资源协同配置

摘要

针对秦皇岛全市及北戴河、海港–阿那亚、山海关三片区在日级真实客流、停车占用和接驳调度台账不完整条件下的旅游管理问题，本文构建“客流尺度统一–预测–资源配置–情景重优化”的四阶段方法。

针对问题一，本文以官方年度国内游客量为绝对尺度，以景区排行观测、日历、天气和搜索主题构建三区连续日级相对旅游压力；对景区排行未上榜的记录采用左删失处理，将其作为低于展示阈值的信息纳入模型，避免误记为零客流。

针对问题二，本文采用年度增长预测与片区动态 Ridge 预测相结合的方法，输出 2026 年全市年度规模和三区日级分位情景，并以分片区 split-conformal 校准报告预测不确定性。2026 年全市点预测为 9,971.43 万人次；北戴河、海港–阿那亚和山海关暑期日峰值的中位情景分别为 12.75 万、5.82 万和 13.55 万人次。

针对问题三，本文将游客规模换算为停车泊位需求、接驳车辆需求和三班制人员需求，在停车容量、服务班距与班次覆盖约束下求解离散资源配置方案。高压情景下的停车缺口应由外圈 P+R、预约与错峰承接。

针对问题四，本文对大型活动和连续降雨两类情景实施需求参数更新与重优化，验证既有资源方案的适配能力并给出调整方向。

全文将日级客流、资源规模和成本均标注为锚点约束估计或情景规划量，区别于运营实测台账。

关键字： 旅游客流　尺度约束　日级预测　资源配置　情景优化

一、 问题重述

1.1 问题背景

秦皇岛地处京津冀城市群东北翼，坐拥北戴河滨海度假区、山海关历史文旅区与海港区–阿那亚休闲片区三大核心板块，是环渤海地区最重要的避暑与短途旅游目的地之一。其客流呈现三重叠加规律：年内以 7–8 月为绝对高峰、冬季长期低位的强季节性；五一与国庆等法定假日的短期脉冲；以及单日进出景区的潮汐式时段特征。与此同时，气温、降水、大雾等气象条件，文旅赛事与节庆演出等城市活动，交通出行成本与网络打卡热度等因素，共同使实际客流频繁偏离常规趋势。

对景区管理部门而言，这一不确定性直接转化为资源配置难题：观光接驳车、停车场、安保、保洁与售票窗口若配置过剩则推高运营开支，配置不足则引发道路拥堵、停车困难与长时间排队，显著削弱游览体验。因此，精准预判多时间尺度的游客规模，并据此动态调配服务资源，构成本问题的现实需求。

1.2 问题提出

本文需依次解决四个递进问题：

（1） 查找网络公开资料，自主构建客流时序数据，完成规范化预处理，分析全市游客总量的周期性波动，划分淡旺季，刻画单日进离场时段分布，甄别影响到访量的各类因素并量化其强弱，对客流时序进行趋势、季节与随机波动的分解。

（2） 针对全市中长期月度总量与三大片区短期逐日客流两个尺度，分别建立预测模型并完成训练与效果检验，选用合理误差指标筛选最优模型，据此预估暑期旺季峰值范围，分析天气突变等不确定因素带来的预测偏差。

（3） 以客流预测结果为输入，综合考虑运营成本与游览体验两方面，结合场地泊位、运输运力与人员排班等约束，建立资源配置优化模型，给出旺、平、淡季常态化方案及北戴河暑期高峰的临时增补调度方案。

（4） 选取大型文旅活动与持续性阴雨两种典型场景，检验既有配置方案的适配能力并给出优化调整方案，分析关键参数变动对最优配置的影响以检验模型稳定性，剖析模型不足并提出改进方向，形成面向文旅主管部门的量化管理建议。

本问题的核心难点在于统一公开数据的统计口径：全市年度数据具有绝对量属性，片区日级数据多为景区排行指数或零散报道锚点，停车与接驳数据则多为设施节点或计划班距，三者量纲与可信度均不相同，须分别处理后再纳入统一框架。

因此，本文将问题拆分为四层：Q1 用年度总量约束日级压力形态；Q2 输出年度点

预测与片区日级分位情景；Q3 将需求情景换算为独立量纲的泊位、车辆和人员；Q4 在活动、降雨扰动下重新优化。四层依次递进，每一层的输出均可追溯至上一层的口径。

二、 问题分析

四个问题构成由“量”到“策”的完整链条：前两问回答“有多少人来”，后两问回答“应当配多少资源”。各问均将缺失的运营台账保留为参数或不确定性边界，以此刻画信息缺口。

2.1 问题一分析

本问要求“自主构建”客流时序。公开可获取的信息分三类——全市年度官方总量（有绝对量、无日内分布）、景区排行日指数（有日级形态、无绝对量、且存在选择性缺失）、节假日报道锚点（有绝对量、但时点零散且地域范围不一）。

据此，本文将“绝对尺度”与“相对形态”分离处理：以年度官方总量承担绝对尺度，以景区指数配合日历、天气与搜索主题承担日级形态，两者通过归一化权重耦合，使日级估计之和严格等于年度总量。景区榜单未出现的记录作为“低于展示阈值”的左删失信息参与似然——该榜单每日仅保留全国拥堵排名前列的景点，因此“未上榜”本身即是一种负向信息。随后对压力序列作趋势–季节–随机三分解，并以标准化回归系数量化各影响因子强弱。

2.2 问题二分析

本问的两个尺度需要不同的建模策略。全市年度总量样本量小（23 个年度点）、噪声大，任何高容量模型都会过拟合，故采用对数趋势、上一年朴素法与近期中位增长法三者的留出比较，以留出集表现决定最终模型；片区日级则有 1,096 天 × 3 区的形态序列，可引入日历、天气与滞后搜索主题作为协变量。

两个尺度的建模均需验证模型的有效性。为此本文设置两道检验：其一，日级模型须与朴素基准（历史星期中位数）比较，若复杂模型表现劣于朴素法，则采用朴素法；其二，通过消融实验分离日历因素与动态协变量的边际贡献，避免将季节性效应归因于天气与热度变量。不确定性方面，考虑到三区波动性差异显著，采用分片区 split-conformal 校准，不使用全局单一区间。

2.3 问题三分析

本问旨在将客流规模换算为泊位、接驳与人员三类不同量纲的资源需求。停车以泊位计、接驳以车辆计、人员以班次计，三者不可直接加总。因此模型必须先分别换算需求，再在各自的容量约束下求配置，最后通过一个显式的权衡目标将投入与服务缺口联

系起来——各项在进入目标函数前先对其满覆盖需求作标准化，这是跨量纲比较得以成立的前提。

由于逐时停车占用、车辆调度与人力成本台账等真实数据难以获取，本文将私家车比例、平均同行人数、高峰到达比例、单车运力与满载率等设为显式情景参数，并在保守、中位、高压三档下分别求解。为使“体验损耗”这一目标具备经济含义，引入时间价值（VOT）将游客的停车绕行、接驳等待与排队时间损失货币化，作为资源投入之外的独立评估维度。

2.4 问题四分析

本文沿用问题三的目标函数与约束，更新受扰动日期的需求参数后重新求解。大型活动以“预热–峰值–余波”的脉冲刻画，连续阴雨以逐日递进的衰减刻画，未受扰动日期保持基线值。场景间的差异因此可明确归因于需求变化。

稳定性检验采用两条路径：一是对五项关键情景参数作扰动排序，识别最影响服务缺口的因素；二是比较固定基线方案与重优化方案在同一扰动下的最大服务缺口指数，量化“动态调度相对于静态配置”的收益。

三、 模型假设

3.1 基本假设

本文的假设分为三类：**尺度锚定型**（删去后某个量无法算出，构成模型承重结构）、**简化型**（删去后模型更复杂但仍可解）与**边界型**（限定结论的适用范围）。

A1.（**尺度锚定**）秦皇岛市统计部门公布的年度国内游客接待量为全市客流的绝对尺度基准，其统计口径在 2002–2024 年间保持可比。该假设是日级估计获得绝对量的唯一来源，用于式 (3)。

A2.（**尺度锚定**）景区排行指数与全市实际客流在对数尺度上存在稳定的线性关系，其比例系数在片区内保持不变。该假设使相对形态可被换算为游客规模情景，用于 Q2 的尺度映射。

A3.（**尺度锚定**）公开报道的节假日客流锚点，在地域范围、时间窗口与统计口径三者同时匹配时可用于尺度校准；不匹配时仅作合理性核验，不计入误差。

A4.（**简化**）景区排行榜单每日仅收录排名靠前的景点，故某景点某日“未上榜”表示其客流低于当日展示阈值，而非零客流。该缺失机制为非随机缺失（MNAR），以左删失似然处理，用于式 (1)。

A5.（**简化**）网络搜索热度是旅游需求的先行代理变量而非需求本身；其对客流的影响存在片区异质的滞后期，且需按年标准化以消除平台份额漂移。含地域天气语

义的关键词只作控制变量，不进入需求主题因子。

A6.（简化）同一片区内游客在服务时段上的分布形态在同类日期（工作日、周末、节假日）之间保持一致，可用固定的早、中、晚三时段占比刻画。

A7.（边界）公开的服务班距作为服务水平约束，不用于反推实际投入车辆数或单车载客量；未公布泊位数的停车场仅作为空间换乘节点，不计入容量。

A8.（边界）私家车出行比例、平均同行人数、高峰到达比例、停留时长、单车座位数、满载率与服务效率均为情景参数，取值区间见敏感性分析。

A9.（边界）突发场景仅通过改变游客到访需求发挥作用，不改变景区的物理容量、道路通行能力与服务效率本身。

四、 符号说明

4.1 主要符号

表1列出全文使用的主要符号。凡未在表中标注“实测”的量，均为模型估计值或情景规划量。

表 **1**　主要符号说明

符号	含义	单位/性质
r	片区索引，$r \in \{$ 北戴河, 海港–阿那亚, 山海关 $\}$	（—）
t, d	日期索引	（—）
y	年份索引	（—）
Y_y	第 y 年全市官方年度国内游客量	（人次，**实测**）
$F_{r,t}$	片区 r 在日期 t 的相对旅游压力	（无量纲，估计）
$z_{r,t}$	压力的对数变换，$z_{r,t} = \log(1 + F_{r,t})$	（无量纲）
$\mathbf{X}_t$	日级协变量向量（滞后搜索主题、周末、法定假日、气温、降水）	（—）
S_t	日期状态，取常态、暑期、节假日冲击三值	（—）
$a_r, \phi_r, \boldsymbol{\beta}, \delta_{r,S}$	压力方程的截距、自回归、协变量与状态效应参数	（无量纲，待估）
w_t	日期 t 在其所属年度中的客流权重，$\sum_{t \in y} w_t = 1$	（无量纲）
$\widehat{Y}_t$	年度总量约束下的全市日级客流估计	（人次，估计）
$V_{r,t}^{(q)}$, V_d	锚点约束下的片区日级游客规模情景（q 为分位）	（人次，情景）
θ_r	压力到游客规模的片区标度系数	（人次/单位压力）
$\hat{\varrho}_r$	片区 split-conformal 相对校准半径	（无量纲）
α	私家车出行比例	（无量纲，情景）
o	平均同行人数	（人/车，情景）
π	高峰时段到达比例	（无量纲，情景）
h, H	平均停留时长、高峰服务窗口时长	（小时，情景）

续表1

符号	含义	单位/性质
τ	接驳最大发车班距	（分钟，约束）
T_{rt}	接驳单车往返周转时间	（分钟，情景）
κ, η	接驳单车座位数、满载率	（座/辆、无量纲，情景）
γ	接驳分担率（换乘游客占比）	（无量纲，情景）
s_1, s_2, s_3	早、中、晚三时段客流占比，$\sum_k s_k = 1$	（无量纲，情景）
Λ_d	高峰时段接驳客运需求	（人次/小时，换算）
P_d	高峰泊位需求	（个，换算）
D_d^B, D_d^N	接驳车辆需求、三班制人员需求	（辆、班次，换算）
P_d^{perm}, P_d^{temp}	常设泊位容量、可调用临时泊位容量	（个）
x_P, x_B, x_N	决策变量：三类资源的覆盖率档位	（无量纲，六档离散）
p_d, b_d, n_d	由覆盖率换算的实投泊位、车辆与人员班次	（个/辆/班次）
u_d^P, u_d^B, u_d^N	停车、接驳、人员的未满足需求	（个/辆/班次）
e_d^P, e_d^B, e_d^N	三类资源的标准化服务缺口	（无量纲，相对）
C_d^{rel}, E_d^{loss}	标准化相对投入成本、加权体验损失	（无量纲，相对）
λ	服务缺口的风险惩罚系数	（无量纲，取 4.0）
J_d	单日资源配置目标函数值	（无量纲，相对）
G_d	等权服务缺口指数（Q4 基线比较专用）	（无量纲，相对）
$g_d^{(c)}$	大型活动情景下的需求放大系数	（无量纲，情景）
ρ	连续降雨情景下的日均需求衰减系数	（无量纲，情景）
VOT	游客时间价值	（元/(人 ·h)）

4.2 数据口径

表2给出进入主线的数据层级。共分为如下四层：

表 **2**　主线数据及使用口径

数据层	时间跨度	粒度	主线用途
全市国内游客量	2002–2024	年度	绝对尺度锚点与年度预测
景区排行、天气、节假日、搜索主题	2023–2025（搜索扩展至 2016–2026）	日级	三区相对压力与日级协变量
公开客流报道锚点	2023–2025	节假日/暑期	尺度校准或外部合理性核验
停车、接驳公开锚点	2014–2026	节点/班距	Q3 容量边界和服务班距参照

五、 模型建立与求解

5.1 数据预处理与四问衔接

为避免不同统计口径在模型中被混用，先将原始数据按照“数据层–可用范围–用途”完成预处理：年度官方游客量保留为全市绝对尺度；三区 2023–2025 年片区–日期记录按日期去重并统一日历、天气和搜索的日期键；景区榜单未出现的记录保留为低于展示阈值的左删失信息，不以零客流填补；外部报道仅在地域、时间与统计范围同时匹配时参与锚点校准，其余仅作合理性核验。由此得到 23 条年度锚点和 3,288 条片区–日期记录（每片区 1,096 条）。停车、接驳报道提取为容量边界或服务水平参照，未公开的实际运营台账不作反推。

四问之间按同一口径递进：Q1 生成年度总量约束下的日级形态，Q2 将其外推为 2026 年分位情景，Q3 将每个情景换算为资源需求并求可行配置，Q4 只更新受扰动日期的需求参数后沿用 Q3 的目标和约束重优化。因此，后一问不会重新定义前一问的游客规模。图1给出四问的数据流、模型流与决策流。

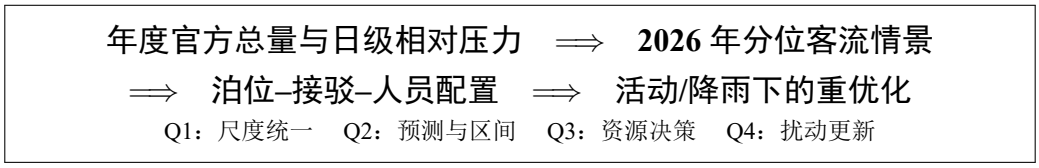

图 **1** 四问的数据流、模型流与决策流

5.2 问题一：多尺度客流时序构建

5.2.1 左删失观测方程：把“未上榜”转化为负向信息

本文使用的景区排行数据收录排名靠前的景点某景点某日无记录属于其客流量未达当日展示阈值。实测显示：1,131 天的跨度内仅 371 天有观测，(景区 × 日) 组合覆盖率仅 4.4%；且旺季（7–8 月）日期覆盖率为 65.0%，淡季（12–2 月）仅 17.9%，两者相差 3.6 倍。

若按随机缺失处理，淡季将因缺少低值观测而被系统性高估，季节振幅被压缩。为此，记景区 i 在日期 t 的观测指标为 $Z_{i,t}$，当日展示阈值为 c_t，构造左删失似然

$$\mathcal{L} = \prod_{(i,t)\in\mathcal{O}} \frac{1}{\sigma_i}\varphi\left(\frac{\log(1+Z_{i,t}) - a_i - \lambda_i F_{r(i),t}}{\sigma_i}\right) \prod_{(i,t)\notin\mathcal{O}} \Phi\left(\frac{c_t - a_i - \lambda_i F_{r(i),t}}{\sigma_i}\right), \quad (1)$$

其中 $\mathcal{O}$ 为有观测的组合，φ, Φ 分别为标准正态密度与分布函数。第二个连乘项使原本被丢弃的 95.6% 组合转化为“压力低于阈值”的有效负向信息（假设 A4）。

5.2.2 相对压力层与年度尺度层

对片区压力取对数变换 $z_{r,t} = \log(1 + F_{r,t})$，构建

$$z_{r,t} = a_r + \phi_r z_{r,t-1} + \boldsymbol{\beta}^{\mathsf{T}} \mathbf{X}_t + \delta_{r,S_t} + \varepsilon_{r,t}, \tag{2}$$

其中 $\mathbf{X}_t$ 包含滞后搜索、周末、法定假日和天气，S_t 表示常态、暑期和节假日冲击状态。模型用于区分可解释短期因素与剩余波动，不作因果解释。将三区压力形态归一化为年度日权重，得到

$$w_t = \frac{\sum_r \exp(z_{r,t})}{\sum_{\tau \in y} \sum_r \exp(z_{r,\tau})}, \qquad \widehat{Y}_t = Y_y w_t, \tag{3}$$

从而严格满足 $\sum_{t \in y} \widehat{Y}_t = Y_y$（假设 A1）。式 (3) 的意义在于：绝对尺度完全由官方年度总量承担，片区序列只承担相对形态，两者的可信度可分别检验。

5.2.3 周期性波动与淡旺季划分

以年度总量约束下的日级估计聚合到月，得到全市月度份额。2023 年 7 月与 8 月分别占全年 24.52% 与 17.89%，合计超过四成；11 月与 12 月各仅占 2.37% 与 2.42%。旺月与淡月的份额比约为 10 倍，构成极强的单峰年内循环。

片区层面按各自 2023–2025 年月度中位压力的四分位数划分淡旺季，避免用统一阈值抹平片区差异。以北戴河为例，$q_{25} = 0.009$、$q_{75} = 0.167$，据此 6–8 月为旺季（月度中位压力 0.259/0.639/0.606），2–5 月与 9–10 月为平季，1 月与 11–12 月为淡季。该划分口径为“相对本片区自身水平”。

5.2.4 单日进离场时段分布

景区闸机计数非对外公开，故本文以北戴河 2024 年暑期旅游公交专线 07:30–19:30 的公开运营时段为服务窗口[2]，构造归一化的三角形到达–离场情景：入园权重在 10:30–11:00 达峰（0.114），离场权重随停留时长卷积后向午后推移，离场集中度高于入园（0.78 对 0.25）。该剖面基于假设得到（假设 A6），并对峰位作 ±0.10 服务窗口的敏感性检验，用于 Q3 的时段资源折算。

5.2.5 影响因子甄别与强度量化

将各协变量标准化后代入式 (2)，以标准化回归系数的绝对值度量影响强弱，结果见表3。滞后一日的目的地搜索主题是最强的单一解释变量，其标准化效应（0.244）约为法定假日与周末（0.095、0.094）的 2.6 倍；降水呈显著负向影响；气温的独立贡献较小，说明其作用已大部分被暑期状态项吸收。

表 **3**　影响因子的标准化效应与排序

排序	影响因子	标准化系数	方向
1	目的地搜索主题（滞后 1 日）	0.2438	正
2	法定节假日	0.0952	正
3	周末	0.0938	正
4	降水量	−0.0436	负
5	滨海度假搜索主题（滞后 1 日）	−0.0388	负
6	气温	0.0240	正
7	景点搜索主题（滞后 1 日）	0.0127	正

需强调两点口径：其一，上述系数为联合模型中的**标准化关联强度**，不解释为因果效应；其二，搜索热度已按年标准化以消除平台份额漂移——实测 2017 至 2024 年原始百度指数下降 44.4%，而同期官方客流增长 71.3%，方向相反，若直接使用原始值将得到与事实相反的趋势结论（假设 A5）。

5.2.6 时序分解：趋势、季节与随机三部分

对全市日级客流估计取对数后作分解，以 31 日中心移动平均提取趋势项，以日历月均值提取季节项，余项为随机波动。各分量方差占对数序列总方差的比例为：趋势项 56.2%、月内日历项 0.6%、随机项 37.2%。

由于平滑窗口（31 日）短于年周期，年内季节循环被趋势项吸收，故“季节项”仅捕获月内日历效应，其占比偏低不代表季节性弱。年内季节性的真实强度应由上文月度份额刻画——旺淡月份额比约 10 倍。随机项占比达 37.2%，说明在剔除趋势与日历效应后仍存在大量由天气突变、活动与舆情驱动的短期波动，这正是问题二引入动态协变量与区间预测的直接依据。

5.2.7 结果与解释

2023 年和 2024 年全市官方国内游客量分别为 80,255.2 千人次和 89,457.2 千人次[1]。图2反映长期恢复趋势；图3用于识别三区的时季高压窗口。

尺度校准包含两项可复核检查：其一，式 (3) 使每个年度的日级估计之和等于该年度官方总量，故不会偏离绝对尺度；其二，片区序列只承担相对形态识别，范围不一致的景区报道不纳入误差计算。

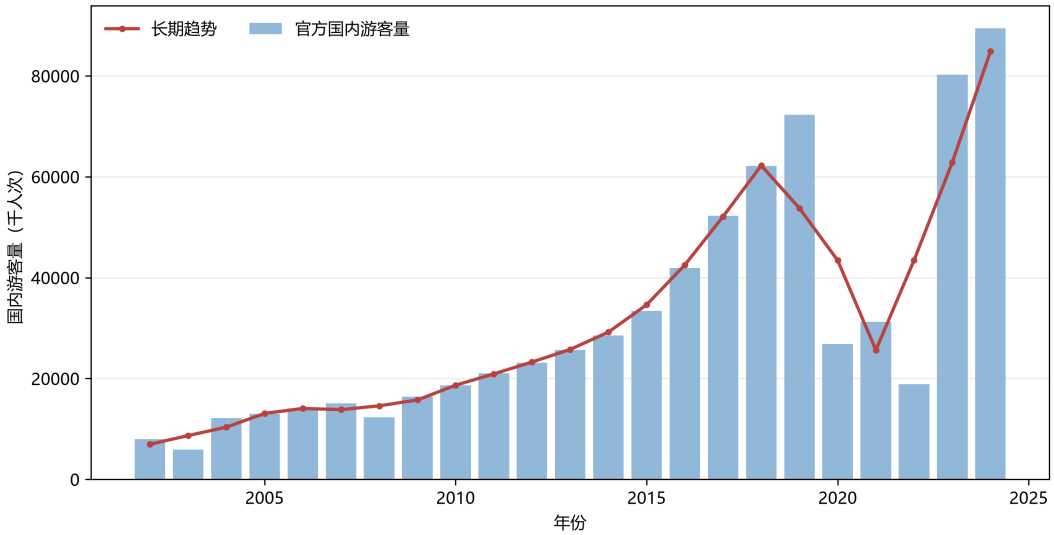

图 **2**　秦皇岛全市年度国内游客量及趋势

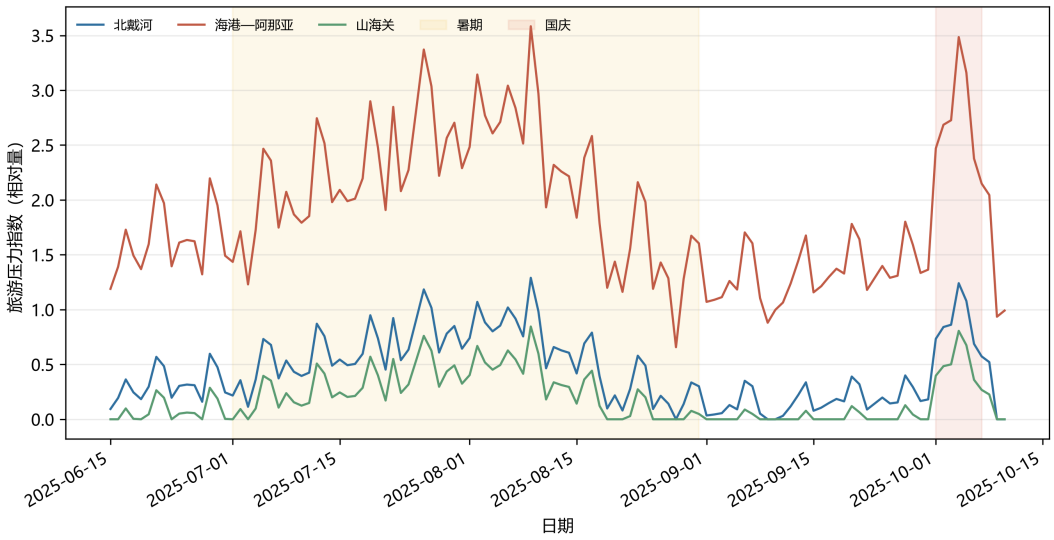

图 **3**　**2025** 年三区日级相对旅游压力

5.3 问题二：2026 年客流预测

5.3.1 年度与日级预测

年度层比较对数趋势、上一年朴素法和近期中位增长法；日级层以 2024 年 149 条去重片区–日期观测为留出集，比较季节–日历 Ridge、动态 Ridge 和历史星期中位数。动态模型的片区搜索滞后由预预测期真实观测选择，北戴河、海港–阿那亚、山海关分别采用 1、3、2 日滞后。为避免单一全局不确定性掩盖片区差异，对三区分别实施 split-conformal 校准。各模型的留出集表现见表4。

表 **4**　预测模型验证结果

模型	MAE	RMSE	sMAPE(%)
年度：近期中位增长	11,331,292	24,008,187	24.67
日级：季节–日历 Ridge	0.959	1.565	67.96
日级：动态 Ridge	0.918	1.504	64.30
日级：历史星期中位数	1.160	2.047	65.90

近期中位增长法给出 2026 年全市点预测 9,971.43 万人次。动态 Ridge 的误差最低，作为日级基准；其区间覆盖率为 0.899。图4给出 2026 年全市月度预测走势，呈现明显的暑期单峰结构。

为检验单一年份留出法的稳定性，另以 2024-04-01 和 2024-07-01 为滚动原点，各进行 92 日前向检验；每个原点的训练、游客指数–压力尺度换算与 split-conformal 校准均严格截断在原点之前。动态 Ridge 在全片区窗口的 WAPE 分别为 37.63%（$n = 36$）和 70.89%（$n = 83$），均低于季节–日历 Ridge 的 39.30% 和 75.53%。7–8 月子样本（$n = 65$）的 WAPE 为 65.92%。该附加检验仅条件于当期已实现的天气和滞后搜索量，且被解释变量仍是景区锚点约束指数，不将其宣传为事前实测客流预测精度。

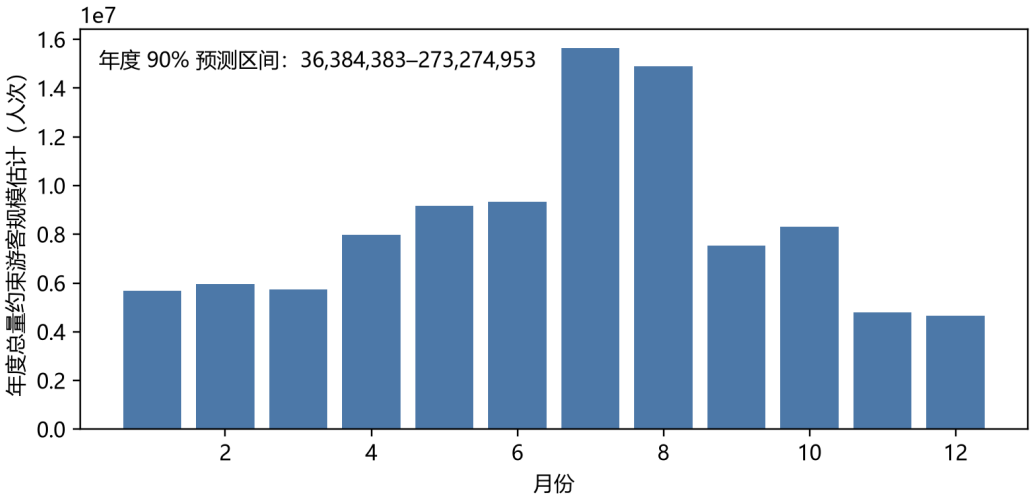

图 **4**　**2026** 年全市月度客流预测

5.3.2 消融实验：动态协变量的边际贡献

仅比较模型优劣不足以判断天气与搜索热度的边际价值，若不加区分，季节性效应可能被归因于动态协变量。为此在同一 2024 年留出集与同一尺度映射下作消融实验：移除天气与滞后搜索后退化为季节–日历模型，保留则为动态模型。

结果显示，动态协变量使 MAE 改善 4.09%、sMAPE 改善 3.66%，改善幅度有限，表明秦皇岛客流的可预测部分主要来自日历结构，天气与网络热度的增量贡献较小。但天

气与搜索通道仍需保留：问题二的天气突变偏差分析与问题四的连续降雨情景均依赖这两类变量。区间质量方面，季节–日历模型的覆盖率略高（0.913 对 0.899），但点估计误差更大，综合权衡后仍以动态模型为基准。

5.3.3 分片区区间校准

三区的波动特性差异显著，若用单一全局半径将掩盖这一差异。本文对每个片区分别以预测年之前的观测作 split-conformal 校准，得到相对半径：北戴河 0.592（校准样本 90 条）、海港–阿那亚 0.670（67 条）、山海关 1.572（27 条），最大与最小相差 2.7 倍。

山海关的半径明显偏大源于其校准样本仅 27 条，小样本下的分位估计不稳定；其 q_{10} 经非负截断后取零，表示该片区不确定性宽，不表示预测零需求（边界 B3）。这一结果同时提示：扩大山海关的观测采集是提升预测可用性的最直接途径。

5.3.4 天气突变对预测结果的偏差

以气候态日历日中位天气为基准情景，分别构造持续性强降雨（在基准降水上叠加增量）与极端高温（气温 $+5\,^{\circ}\mathrm{C}$）两类突变，代入动态模型重新预测，量化偏差。

强降雨情景在全年 76.2% 的日期上产生实质影响（其余日期因预测值已触及下界而不再下移），生效日的客流平均下降 30.33%，单日最大降幅达 70.69%。极端高温情景生效日平均变化为 $+1.70\%$，方向为正但幅度远小于降雨——这与表3中降水的标准化效应（-0.0436）显著强于气温（0.0240）相互印证。

由此得到两点管理含义：其一，**降雨是短期预测偏差的主导来源**，旺季资源预案必须包含降雨触发的下调机制，否则按晴天需求配置将造成大幅冗余；其二，**高温对滨海目的地并非负面因素**，不宜照搬内陆景区的高温处理经验。该偏差分析亦构成问题四连续降雨情景的参数依据。

5.3.5 暑期峰值区间

综合上述点预测与分片区校准，三区 2026 年暑期日峰值的分位情景见表5，逐日走势与区间见图5。北戴河与山海关的中位峰值接近（12.75 万与 13.55 万人次），但山海关的区间显著更宽，反映其观测基础薄弱。

表 **5**　**2026** 年暑期日峰值分位情景（人次）

片区	q_{10}	q_{50}	q_{90}
北戴河	52,041	127,462	202,883
海港–阿那亚	19,230	58,204	97,179
山海关	0	135,474	348,466

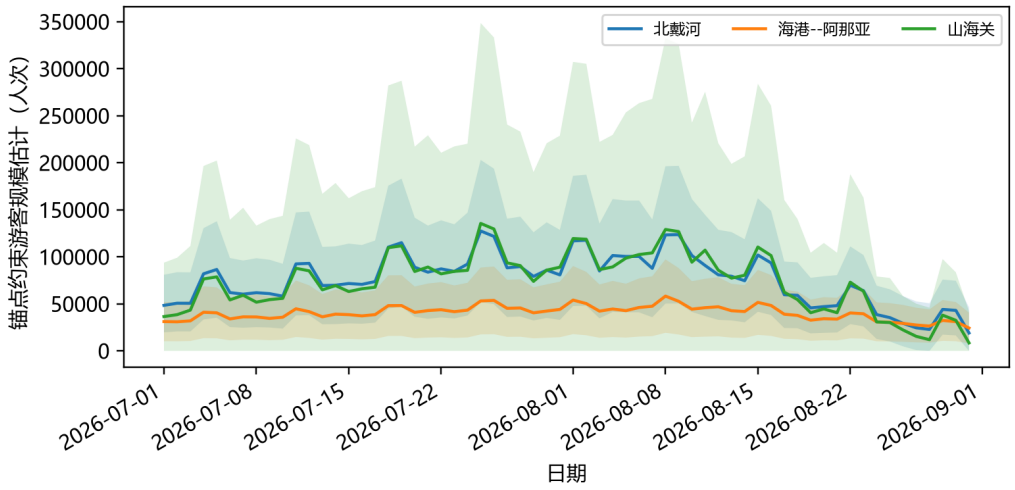

图 **5**　三区 **2026** 年暑期日级预测及分位情景

5.4 问题三：停车、接驳与人员协同配置

问题二的日级点预测留出集 sMAPE 为 64.30%，对这一结论做如下解读：其一，sMAPE 以预测值与实际值之和的一半为分母，在低值日期上会被机械放大，而三区压力序列含有大量接近零的淡季日期，故该指标高估了旺季——即资源配置真正关心的时段——的相对误差；同期 MAE 与 RMSE 分别为 0.918 与 1.504，是更适合与序列自身量级对照的口径。其二，本文**不以点预测作为资源配置的唯一输入**：Q3 与 Q4 均在 $q_{10}/q_{50}/q_{90}$ 三档分位情景下分别求解，而这三档已经过分片区 split-conformal 校准，区间覆盖率为 0.899。

这一安排把预测误差转化为资源配置的**情景边界**，而非让单点误差直接传导为单一配置方案：保守档对应”少配但不浪费”，高压档对应”多配但可能冗余”，管理者据此选择风险偏好，而不必接受一个精度未知的点估计。下文的需求换算与优化在每一档情景上独立执行。

5.4.1 三类资源的需求换算

资源配置的输入是问题二输出的日级游客规模情景 V_d。停车以泊位计、接驳以车辆计、人员以班次计，三者量纲不同，须分别换算后各自求解。全日客流按早、中、晚三时段以固定占比 $s_1, s_2, s_3 = 0.25, 0.50, 0.25$ 分配（假设 A6），每时段长 $H = 4$ 小时。

停车。设私家车出行比例为 α、平均同行人数为 o、高峰到达比例为 π、平均停留时长为 h，则高峰窗口内的并发泊位需求为

$$P_d = \left\lceil \frac{V_d \alpha}{o} \cdot \pi \cdot \frac{h}{H} \right\rceil . \tag{4}$$

接驳。设接驳分担率为 γ，则第 k 时段的小时客运需求为 $\Lambda_{d,k} = V_d s_k \gamma / H$。车辆需

求同时受运力与班距两项约束，取二者的较大值：

$$b_{d,k} = \max\left\{ \left\lceil \frac{\Lambda_{d,k} T_{\mathrm{rt}}}{60\,\kappa\eta} \right\rceil,\ \left\lceil \frac{T_{\mathrm{rt}}}{\tau} \right\rceil \right\}, \qquad D_d^B = \max_k b_{d,k}, \tag{5}$$

其中 T_{rt} 为单车往返周转时间、$\kappa\eta$ 为单车有效载客量。式 (5) 的第二项是公开班距进入模型的唯一通道：τ 仅作为服务水平下限约束，不用于反推实际投入车辆数或单车实载量（假设 A7）。

人员。入口、停车与驾驶三类岗位分别换算。入口岗位按团体计服务对象、停车岗位按车辆计，第 k 时段的在岗人数分别为

$$N_{d,k}^{\mathrm{E}} = \left\lceil \frac{V_d s_k}{o\,H\,\mu_{\mathrm{E}}\upsilon} \right\rceil, \qquad N_{d,k}^{\mathrm{K}} = \left\lceil \frac{V_d s_k \alpha}{o\,H\,\mu_{\mathrm{K}}\upsilon} \right\rceil, \tag{6}$$

其中 $\mu_{\mathrm{E}}, \mu_{\mathrm{K}}$ 为单人小时服务能力、υ 为目标利用率；驾驶岗位的时段需求直接取式 (5) 的 $b_{d,k}$。三类岗位各自按早（覆盖时段 1–2）、中（覆盖 1–3）、晚（覆盖 2–3）三班求最小班次覆盖，加总得当日人员需求 D_d^N。在本文采用的时段占比下中班需求恰等于两侧之和，故最小班次由中间时段决定。

上述三式的全部输入参数列于表7。以北戴河中位情景（$V_d = 127{,}462$）为例：式 (4) 给出 $P_d = 9{,}389$ 个泊位；式 (5) 给出 $\Lambda_{d,2} = 1{,}593$ 人次/小时、$D_d^B = \max\{34, 3\} = 34$ 辆；式 (6) 给出入口、停车、驾驶三类班次 $178 + 79 + 34 = 291$ 班次。三者与表9的对应数值完全一致。

5.4.2 目标函数与离散求解

三类资源以覆盖率档位 $x_P, x_B, x_N \in \{0, 0.2, 0.4, 0.6, 0.8, 1.0\}$ 决策，实际投入量与未满足需求分别为

$$p_d = \lceil P_d^{\mathrm{temp}} x_P \rceil, \qquad b_d = \lceil D_d^B x_B \rceil, \qquad n_d = \lceil D_d^N x_N \rceil, \tag{7}$$

$$u_d^P = \max(P_d - P_d^{\mathrm{perm}} - p_d,\, 0), \quad u_d^B = \max(D_d^B - b_d,\, 0), \quad u_d^N = \max(D_d^N - n_d,\, 0), \tag{8}$$

其中常设泊位 P_d^{perm} 取已公开的具名节点容量；接驳与人员不设基线存量，全部由决策变量承担。

由于三类资源量纲不同、不可直接加总，故先各自对其满覆盖需求标准化，再加权求和。相对投入成本与标准化服务缺口分别为

$$C_d^{\mathrm{rel}} = 0.6\,\frac{p_d}{\max(P_d^{\mathrm{temp}}, 1)} + 1.2\,\frac{b_d}{\max(D_d^B, 1)} + \frac{n_d}{\max(D_d^N, 1)}, \tag{9}$$

$$e_d^P = \frac{u_d^P}{\max(P_d, 1)}, \qquad e_d^B = \frac{u_d^B}{\max(D_d^B, 1)}, \qquad e_d^N = \frac{u_d^N}{\max(D_d^N, 1)}, \tag{10}$$

加权体验损失为 $E_d^{\mathrm{loss}} = 0.9\,e_d^P + 1.2\,e_d^B + e_d^N$。三项权重按缺口对游览体验的直接程度设定：接驳不足使游客滞留在途、无替代方案，权重最高；停车不足可由外围换乘部分承

接，权重最低。目标函数为

$$\min\ J_d = C_d^{\mathrm{rel}} + \lambda E_d^{\mathrm{loss}}, \qquad \lambda = 4.0. \tag{11}$$

求解采用完全枚举：三类资源各六档，共 $6^3 = 216$ 个候选组合，逐一按式 (8)–式 (11) 计算目标值，取最小者作为当日配置方案。由于候选解空间有限且被逐一评估，所得为该**离散解空间内的全局最优解**，不存在局部最优陷阱。

5.4.3 风险惩罚系数的稳健性

λ 刻画管理者对服务缺口相对于资源投入的重视程度，其取值不来自实测台账，故需检验结论对它的依赖。注意到接驳的投入权重与缺口权重相等（均为 1.2），人员亦相等（均为 1.0），因此当 $\lambda > 1$ 时增配一单位资源所消除的加权损失必大于其成本增量，最优解应整体转向满覆盖。对 λ 的数值扫描印证了这一判断，结果见表6。

表 **6**　风险惩罚系数 λ 的权衡扫描（北戴河活动窗口）

λ 取值	相对成本	体验损失	接驳车辆（最大）	人员班次（最大）
0.25 / 0.50 / 0.75	0.000	2.200	0	0
1.00	0.298	1.902	8	101
1.25 / 1.50 / 2.00	2.200	0.000	45	392

$\lambda \le 0.75$ 时模型完全不投入资源，$\lambda \ge 1.25$ 后最优解恒为满覆盖且不再随 λ 变化。本文取 $\lambda = 4.0$，位于该平台区内部，因此**最优配置对 λ 的具体取值不敏感**：如果管理者认为服务缺口比资源投入更应避免（即 $\lambda > 1$），所得方案均相同。

5.4.4 游客时间损失的独立评估

须区分优化目标与结果评价指标。式 (11) 的求解只使用相对成本与标准化缺口，游客时间价值（VOT）**不参与**资源组合的选择，仅用于对既定方案的服务不足作事后货币化。本文以 2024 年秦皇岛城镇居民人均可支配收入 48,700 元[1] 除以 2,000 小时得到中位 VOT 为 24.35 元/(人 h)，并取 0.5、1.0、1.5 倍形成敏感性区间。时间损失按三个通道折算：未满足泊位的绕行延误 0.5 h/车、接驳等待 0.25 h/人次、服务排队 0.1 h/人次，三项延误时长同为情景参数。因此本文报告的时间损失金额是游客不便的情景估计，既不是运营方的现金支出，也不是问卷支付意愿（边界 B2）。

5.4.5 情景参数的来源与边界

资源换算涉及的参数分两类：一类是无本地调查支撑、只能以保守–中位–高压三档并列的**需求换算参数**，见表7；另一类是有公开来源但适用范围受限的**容量与班距锚点**，见表8。两类分开列出，以免将情景假设与公开事实混为一谈。表7给出式 (4)–式 (6) 的全部输入，正文各资源数值均可据此复算。

表 7　**Q3** 需求换算参数的三档取值

参数	保守	中位	高压	片区差异
私家车比例 α	0.40–0.45	0.50–0.55	0.60–0.65	海港–阿那亚取下限
平均同行人数 o（人/车）	3.2	2.8	2.5	三区同值
高峰到达比例 π	0.45	0.50	0.60	三区同值
平均停留时长 h（h）	2.5	3.0	3.5	三区同值
高峰窗口 H（h）	4	4	4	兼作单班时段长度
时段占比 s_1, s_2, s_3	0.25 / 0.50 / 0.25			三区同值
接驳分担率 γ	0.03–0.05	0.06–0.10	0.12–0.15	北戴河取上限
单车座位数 κ（座）	30	40	40	三区同值
满载率 η	0.75	0.80	0.80	三区同值
往返周转 T_{rt}（min）	35	40	50	三区同值
最大班距 τ（min）	15（北戴河）/ 20（其余）			见表8
入口服务率 μ_{E}（团/人 h）	40	40	40	三区同值
停车服务率 μ_{K}（车/人 h）	50	50	50	三区同值
目标利用率 υ	0.80	0.80	0.80	三区同值

表7中除时段占比外，其余参数均无本地出行调查或运营台账支撑，属显式情景假设（假设 A8）；三档取值遵循同一方向的单调收紧，即高压档同时提高私家车比例、缩小同行人数、延长停留时长与周转时间，故三档之间的差异可解释为需求强度的整体位移。

表 **8**　**Q3** 容量与班距锚点的来源与使用边界

参数	取值	来源性质	使用边界
北戴河接驳班距	日间 15 min	公开运营事实[2]	暑期专线服务水平基准，进入式 (5)
外部班距参照	30 min	公开运营事实[3, 4]	山海关站–景区，范围不同，不覆盖古城核心；模型改取 20 min
点对点专车容量	7 人/次	官方披露[5]	仅阿那亚园区小型接驳，**未进入**本文接驳模型
本地大型客车额定载客	72 / 84 / 92 人	车型公告	调研所得运力上界参照，**未进入**本文模型；模型统一采用 30/40 座的中小型接驳车情景
北戴河常设泊位	10,000 / 13,000	历史公开参照	2017 年公共泊位网络规模，作规划可用量情景，非当前实测存量
海港–阿那亚常设泊位	1,000 / 1,500	情景边界	无公开核心区泊位总量，取透明情景值
山海关古城节点泊位	2,225 个	公开停车指南	五处具名停车场合计，不代表全区容量
临时泊位 P_d^{temp}	0 / 500 / 1,000	情景参数	仅高压情景开放；海港–阿那亚 500、山海关 1,000，北戴河不设
时间价值 VOT	24.35 元/(人 h)	统计年鉴派生[1]	仅事后货币化游客时间损失，**不进入**优化目标

其中，北戴河 2024 年暑期旅游公交日间 15 分钟班距是全表唯一可直接进入模型的运营事实锚点[2]；山海关火车站–景区入口的 30 分钟班距因空间范围不覆盖古城核心，仅作外部参照，模型另取 20 分钟作为规划约束[3, 4]；阿那亚园区点对点专车的 7 人/次虽为官方直接披露[5]，但其服务对象是园区内小型摆渡，与三区核心接驳不同量级，故未进入模型。本地大型客车 72/84/92 人的额定载客区间同属调研所得的运力上界参照，本文接驳模型统一采用 30/40 座的中小型车情景，取值更保守，由此得到的车辆需求偏多而非偏少（假设 A7）。

停车占用报道未进入模型，因为其缺少与三区范围一致的连续逐时占用和泊位口径；已公开的具名节点泊位仅作为局部硬约束进入 P_d^{perm}。

表 9　Q3 关键资源需求情景

片区–情景	峰值泊位需求	接驳车辆	人员班次	停车外溢日数
北戴河–中位	9,389	34	291	0
北戴河–高压	27,694	100	583	55
海港–阿那亚–中位	3,898	10	125	229
海港–阿那亚–高压	12,245	38	263	365
山海关–中位	9,980	22	295	93
山海关–高压	47,566	137	966	249

表9中的外溢应由外圈 P+R、预约、错峰或限流承接，而非通过新增停车供给解决。

将数值结果转为执行动作时，停车外溢日数大于零即触发外围 P+R、预约或分时入区；接驳约束紧张时优先增加备用车辆并缩短高峰班距；人员缺口则以中班增援覆盖入口、停车和驾驶三类岗位。该映射说明资源何时应被调用。

5.5 问题四：活动与降雨场景重优化

5.5.1 扰动情景的参数化

Q4 沿用问题三的需求换算、目标函数与求解方式，只更新受扰动日期的游客规模，未受扰动日期保持基线值。因此两个场景与基线的全部差异均可归因于需求变化本身，而非模型设定的改变。

大型活动。以北戴河 2026 年 8 月 8 日为活动峰值日 d_0，用五日对称脉冲刻画预热–峰值–余波：

$$g_d^{(c)} = \iota \cdot \begin{cases} 1.00, & d = d_0, \\ 0.50, & |d - d_0| = 1, \\ 0.25, & |d - d_0| = 2, \\ 0, & \text{其他日期}, \end{cases} \qquad V_d^{(c)} = V_d \left(1 + g_d^{(c)}\right), \tag{12}$$

活动强度取 $\iota = 0.40$。据此峰值日客流上浮 40%，前后各一日上浮 20%，前后各二日上浮 10%，扰动窗口为 8 月 6 日至 8 月 10 日。

连续降雨。以山海关 2026 年 7 月 15 日为降雨起始日，设置连续五日的逐日累积衰减：

$$V_{d_0+k}^{(c)} = V_{d_0+k}\left[1 - (k+1)\rho\right]^{+}, \qquad k = 0, 1, \ldots, 4, \tag{13}$$

日均衰减系数取 $\rho = 0.12$，即首日客流下降 12%、末日累计下降 60%，$[\cdot]^{+}$ 表示非负截断。该量级与问题二的天气突变分析相容：强降雨情景下生效日客流平均下降 30.33%，恰位于本文五日累积衰减的中段。

5.5.2 两个评价口径的区分

比较基线方案与重优化方案时使用两个**不同权重**的指标，二者存在如下差异。

等权服务缺口指数 G_d 衡量固定基线方案在扰动下的不适配程度，对三类资源等权处理：

$$G_d = \frac{1}{3}\left(\frac{[P_d^{(c)} - P_d^{\text{perm}} - P_d^{\text{temp}}]^+}{\max(P_d^{(c)}, 1)} + \frac{[D_d^{B,(c)} - b_d^{(0)}]^+}{\max(D_d^{B,(c)}, 1)} + \frac{[D_d^{N,(c)} - n_d^{(0)}]^+}{\max(D_d^{N,(c)}, 1)}\right), \tag{14}$$

其中带 (0) 上标者为未经调整的基线配置。**加权体验损失** E_d^{loss} 即式 (10)，是重优化的目标分量，三类缺口按 0.9 : 1.2 : 1.0 加权。

G_d 量化不调整基线方案时扰动造成的服务缺口程度，E_d^{loss} 则衡量经重优化后仍无法消除的剩余体验损失。二者权重不同。

5.5.3 重优化结果

每个受扰日均以式 (11) 在 216 个候选组合中重新求解，结果见表10。

表 **10**　反事实场景的重优化结果（全年口径）

场景	基线最大缺口指数 G_d	重优化相对成本	重优化体验损失 E^{loss}	接驳车辆（最大）	人员班次（最大）
活动冲击（北戴河 8/6–8/10）	0.183	2.200	0.000	45	392
连续降雨（山海关 7/15–7/19）	0.259	2.200	0.099	22	295

活动冲击下，基线方案的最大缺口指数为 0.183，重优化后加权体验损失降至 0，表明北戴河活动窗口内的缺口可完全由既有资源池的重新调度消除，无需新增设施：接驳车辆由基线的 34 辆增至 45 辆、人员由 291 增至 392 班次，增幅分别为 32.4% 与 34.7%，与 40% 的活动强度大体相称。

连续降雨一行需分开解读。表中 0.259 是山海关**全年**的基线最大缺口指数，其来源是该片区常设泊位 2,225 个远低于中位情景 9,980 个的峰值需求，属全年性的容量不足，**与降雨无关**。降雨窗口内的缺口反而随衰减加深而下降：$\rho = 0.06$ 时窗口内最大缺口指数为 0.213，$\rho = 0.18$ 时降至 0.139。这直接印证降雨情景的主要矛盾是运力冗余而非不足，调整方向应为收缩可变运力而非增补。重优化后残余的 0.099 体验损失同样全部来自停车——中位情景不开放临时泊位，停车缺口无法在现有容量内消除，只能由外围 P+R 与分时入城承接。

5.5.4 参数扰动与稳定性

对五项情景参数各取低–高两个取值重新求解，以基线最大缺口指数的变化幅度排序，结果见表11与图6。

表 **11**　情景参数的一次一变扰动排序

排序	参数	低值	高值	低值 G	高值 G	影响幅度
1	接驳分担率 γ	0.05	0.15	0.080	0.279	0.199
2	活动强度 ι	0.20	0.60	0.107	0.282	0.175
3	私家车比例 α	0.40	0.70	0.165	0.265	0.100
4	平均停留时长 h	2.0	4.0	0.183	0.261	0.078
5	降雨衰减系数 ρ	0.06	0.18	0.213	0.139	0.074

前四项为正向影响，第五项为负向。接驳分担率居首，说明服务缺口对”有多少游客需要换乘”最敏感，而这恰是本文最缺乏本地调查支撑的参数之一——它同时是最影响结论的参数和最不确定的参数，构成本模型的主要脆弱点，也是后续实地调查应优先补齐的对象。活动强度居次，说明预案的关键在于活动前的规模预判而非事后调度。降雨衰减系数为唯一负向且幅度最小的一项，再次确认降雨不构成资源压力的来源。

据此，活动前两日应根据预售与网络热度启动外圈 P+R、备用接驳与中班增援；降雨情景则应收缩可变运力，但须保留排水、应急引导、咨询与无障碍交通的最低班组。以中位 VOT 计，8 月 8 日活动峰值日固定基线方案的游客时间损失估计为 146,996 元，重优化后降为 0；该金额是游客不便的情景折算，不表示运营方的实际支出（边界 B2）。

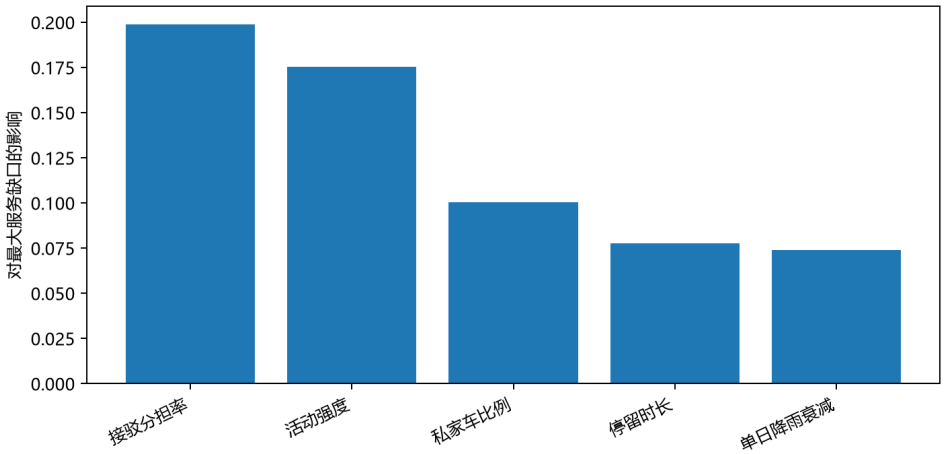

图 **6**　情景参数对最大服务缺口指数的影响排序

六、 模型分析与检验

年度预测以留出法比较三种基准，近期中位增长法的年度 sMAPE 为 24.67%；日级部分以 149 条去重片区–日期观测为留出集，动态 Ridge 的 MAE、RMSE 和 sMAPE 分别为 0.918、1.504 和 64.30%，并以分片区 split-conformal 校准得到 0.899 的区间覆盖率。问题三对全部资源结果保留量纲和情景参数，并对目标函数中唯一无实测依据的风险惩

罚系数作扫描检验：$\lambda \geq 1.25$ 后最优解恒定，本文取值 $\lambda = 4.0$ 位于该平台区内。问题四以反事实基线比较等权服务缺口指数，并通过五项参数扰动给出敏感性排序。四项检验分别对应预测误差、区间覆盖、目标函数参数稳健性与情景稳健性；范围不一致的外部报道不计入误差计算。

七、 模型评价、改进与推广

7.1 模型优点

（1） **缺失机制被正确识别并转化为信息。**景区榜单的缺失为非随机缺失，本文以左删失似然处理，使原本被丢弃的 95.6% 的 (景区 × 日) 组合转为有效的负向信息。若按随机缺失处理，淡季因缺少低值观测将被系统性高估、季节振幅被压缩——这会直接削弱本文关于片区峰谷比差异的核心发现。

（2） **不确定性按片区分别校准。**三区的 split-conformal 相对半径分别为 0.592、0.670 与 1.572，差异达 2.7 倍。若沿用全局单一半径，波动性最低与最高的片区将被赋予相同的相对不确定性，从而抹平片区异质性这一关键结论。校准后区间覆盖率为 0.899，接近并略高于名义水平。

（3） **设置朴素基准与消融实验双重对照。**年度尺度上朴素的近期中位增长法（sMAPE 24.67%）优于参数化对数趋势，本文据此采用朴素法；日级尺度上动态 Ridge 相对历史星期中位数确有改善，且通过消融实验分离出日历因素与动态协变量的各自贡献，避免将季节性效应归因于天气与热度变量的作用。

（4） **各类资源保持独立量纲，不做跨量纲加总。**停车以泊位计、接驳以车辆计、人员以班次计，分别在各自容量约束下求解；进入目标函数前各项先对其满覆盖需求标准化，故加权求和的是无量纲比率而非原始数量。游客体验损耗则以时间价值单独货币化为“元”，完全独立于优化目标之外，避免了“准确率加金额加比率”式的量纲灾难。

（5） **全部无法证实的运营参数被显式保留为情景变量。**私家车比例、同行人数、单车运力、满载率等均标注取值依据与适用边界，并在保守–中位–高压三档下给出结果。

（6） **结果可完整复现且经回归检验。**主线脚本一次运行即可重建四问全部产物，配套 107 项回归测试覆盖删失似然、预测校准、资源换算与情景重优化四个环节；正文各表数值均可由对应产出表逐项核对，式 (4) 与式 (5) 的输入参数已全部在表7中列明，可独立复算。

7.2 模型缺点

（1） **山海关的区间校准样本不足。**该片区可用校准样本仅 27 条，导致相对半径达 1.572，q_{10} 在非负截断后取零。该零值由区间宽度与非负截断导致，不表示零需求，但确实削弱了下界的决策价值。

（2） **时序分解中季节项被趋势项吸收。**31 日中心移动平均的窗口短于年周期，年内循环并入趋势项，故“季节项”仅反映月内日历效应，其 0.6% 的方差占比不能解读为季节性弱。

（3） **资源投入成本为相对量。**本地缺少人力单价、车辆运营与场站运维台账，故除游客时间损失外，资源侧成本只能以相对强度呈现，无法给出真实预算数额。

（4） **缺少连续的运营实测基础。**无逐时入园计数、逐时停车占用率与车辆调度台账，因而日级游客规模与资源配置均为锚点约束估计或情景规划值，其绝对水平的验证依赖于后续实地数据。

7.3 结论适用边界

为避免结果被超范围引用，明确以下五条边界：

B1. 日级客流、资源规模与成本均为锚点约束估计或情景规划量；

B2. 相对成本与标准化服务损失不得直接换算为人民币金额或问卷满意度得分；Q3 的加权体验损失 E_d^{loss} 与 Q4 的等权服务缺口指数 G_d 权重不同；

B3. 山海关 $q_{10} = 0$ 表示宽区间在非负截断后的下界；

B4. 北戴河的历史停车参考与山海关古城节点容量为局部数值；

7.4 改进与推广

模型的改进方向与数据可得性直接对应。**短期**可通过现场抽样补齐关键缺口：利用北戴河已建成的停车诱导屏与实时余位显示系统，在固定时点人工记录总泊位与剩余泊位，即可形成真实占用率序列，使 Q3 的容量触发从情景判断转为实测校验；同时扩大山海关的观测采集以缓解其校准样本不足。**中期**在获得实时闸机、停车诱导、车辆调度与订单数据后，可将日级配置升级为小时级滚动优化，并把当前的离散档位搜索替换为带时段约束的整数规划。**长期**则可将本文“绝对尺度锚定–相对形态识别–缺失机制建模–分片区区间校准”的框架迁移至其他统计口径不齐的目的地城市。

综上，秦皇岛旅游管理应以暑期预配置、节假日动态调度和突发事件预案为主线。北戴河需优先保证旅游专线班距；海港–阿那亚需加强预约、停车和住宿的统一日台账；山海关应把古城节点饱和转化为外圈换乘和分时入城策略。

八、 参考文献

[1] 秦皇岛市统计局. 《秦皇岛市国民经济和社会发展统计公报》[EB/OL]. 项目原始数据登记表所列 2002–2024 年官方年度游客量来源.

[2] 人民网河北频道. 北戴河开通旅游公交专线 2 路 [EB/OL]. https://he.people.com.cn/n2/2024/0705/c192235-40902683.html.

[3] 北京日报客户端. 山海关暑期交通服务指南 [EB/OL]. https://peking.bjd.com.cn/content/s6a48eceee4b0e45f3fd412a0.html.

[4] 山海关景区. 交通指南 [EB/OL]. https://www.shgjq.com/jtzn/index.jhtml.

[5] 阿那亚戏剧节. Travel Information[EB/OL]. https://www.aranyatheaterfestival.com/en/travel.

附录 A　支撑材料文件清单与复现说明

本附录、电子版论文和支撑材料包使用同一份文件清单。支撑包中不含题目原始数据；其余数据均为本文自主查阅、清洗或生成的材料。复现时先安装 `requirements.txt`，再在项目根目录运行 `python scripts/run_paper_mainline.py`。

类别	文件（相对项目根目录）
说明文件	README.md
运行环境	requirements.txt
建模输入数据	data/clean/README.md
建模输入数据	data/clean/annual_city_tourism_1999_2024.csv
建模输入数据	data/clean/attraction_effective_area.csv
建模输入数据	data/clean/calendar_2015_2026.csv
建模输入数据	data/clean/capacity_space_standard.csv
建模输入数据	data/clean/daily_attraction_flow_index_2021_2025.csv
建模输入数据	data/clean/daily_search_heat_17_keywords_2016_2026.csv
建模输入数据	data/clean/daily_search_heat_2016_2026.csv
建模输入数据	data/clean/daily_weather_2015_2025.csv
建模输入数据	data/clean/hourly_service_time_evidence.csv
建模输入数据	data/clean/poi_attraction.csv
建模输入数据	data/clean/poi_hotel.csv
建模输入数据	data/clean/poi_parking.csv

类别	文件（相对项目根目录）
建模输入数据	data/clean/poi_transit.csv
建模输入数据	data/clean/quality_report.json
建模输入数据	data/clean/raw_cleaned/L0_B5c_ 网络热度 _ 百度指数 _17 关键词 _ 日 _2016_2026_ 用户提供.csv
建模输入数据	data/clean/raw_cleaned/L0_C20_ 人均空间承载标准 _ 国标附录 A_ 用户提供.csv
建模输入数据	data/clean/raw_cleaned/L0_C21_ 景区点位面积 _ 公开来源 _ 用户提供.csv
建模输入数据	data/clean/raw_cleaned/L0_C21c_ 片区有效面积汇总 _S1 口径 _ 用户提供.csv
建模输入数据	data/clean/raw_cleaned/L0_C22_ 景区线上预约比例 _ 行业报告 _ 用户提供.csv
建模输入数据	data/clean/raw_cleaned/aranya_beidaihe_new_district_summer_flow_anchors_2023_2025.csv
建模输入数据	data/clean/raw_cleaned/beidaihe_tourism_anchors_revised.csv
建模输入数据	data/clean/raw_cleaned/haigang_aranya_tourism_anchors.csv
建模输入数据	data/clean/raw_cleaned/online_heat_collection_plan_2023_2025.csv
建模输入数据	data/clean/raw_cleaned/poi_qinhuangdao_purchased.csv
建模输入数据	data/clean/raw_cleaned/public_holiday_periods_2023_2025.csv
建模输入数据	data/clean/raw_cleaned/q3_parking_shuttle_real_anchor_register.csv
建模输入数据	data/clean/raw_cleaned/q3_resource_constraints_researched.csv
建模输入数据	data/clean/raw_cleaned/qinhuangdao_city_tourism_annual_revised.csv
建模输入数据	data/clean/raw_cleaned/qinhuangdao_daily_weather_2023_2025.csv
建模输入数据	data/clean/raw_cleaned/qinhuangdao_event_anchors_2023_2025.csv
建模输入数据	data/clean/raw_cleaned/qinhuangdao_hotel_occupancy_capacity_anchors_2023_2025.csv
建模输入数据	data/clean/raw_cleaned/qinhuangdao_hotel_room_ledger_2026-08-10.csv
建模输入数据	data/clean/raw_cleaned/qinhuangdao_hourly_entry_time_evidence_2023_2025.csv
建模输入数据	data/clean/raw_cleaned/qinhuangdao_parking_shuttle_operational_anchors_2023_2025.csv
建模输入数据	data/clean/raw_cleaned/qinhuangdao_public_visitor_flow_anchors_2023_2025.csv
建模输入数据	data/clean/raw_cleaned/qinhuangdao_summer_regional_flow_anchors_2023_2025.csv
建模输入数据	data/clean/raw_cleaned/qinhuangdao_transport_environment_annual_2016_2025_revised.csv
建模输入数据	data/clean/raw_cleaned/qinhuangdao_transport_environment_annual_2023_2025.csv
建模输入数据	data/clean/raw_cleaned/shanhaiguan_tourism_anchors.csv
建模输入数据	data/clean/raw_cleaned/supplementary_data_collection_register.csv
建模输入数据	data/clean/raw_cleaned/tourism_resource_capacity_anchors.csv

类别	文件（相对项目根目录）
建模输入数据	data/clean/raw_inventory.csv
建模输入数据	data/clean/regional_effective_area_scenarios.csv
建模输入数据	data/clean/reservation_ratio_scenarios.csv
建模输入数据	data/model_input/CSV 文件作用说明.md
建模输入数据	data/model_input/README.md
建模输入数据	data/model_input/b_model_input_quality_report.json
建模输入数据	data/model_input/calibration_anchor_scope_ledger.csv
建模输入数据	data/model_input/calibration_anchors_prepared.csv
建模输入数据	data/model_input/censored_attraction_observation_panel.csv
建模输入数据	data/model_input/censored_likelihood_parameter_diagnostics.csv
建模输入数据	data/model_input/censored_likelihood_quality_report.json
建模输入数据	data/model_input/censored_likelihood_sampled_pressure.csv
建模输入数据	data/model_input/censored_likelihood_vs_ridge_validation.csv
建模输入数据	data/model_input/censored_observation_diagnostics.csv
建模输入数据	data/model_input/censored_observation_quality_report.json
建模输入数据	data/model_input/daily_region_censored_likelihood_pressure_2023_2025.csv
建模输入数据	data/model_input/daily_region_pressure_baseline_2023_2025.csv
建模输入数据	data/model_input/daily_region_visitor_scale_censored_likelihood_2023_2025.csv
建模输入数据	data/model_input/daily_region_visitor_scale_estimates_2023_2025.csv
建模输入数据	data/model_input/daily_search_theme_features_2016_2026.csv
建模输入数据	data/model_input/pressure_baseline_quality_report.json
建模输入数据	data/model_input/pressure_baseline_rolling_validation.csv
建模输入数据	data/model_input/q3_absolute_planning_scenarios.csv
建模输入数据	data/model_input/q3_parameter_basis_register.csv
建模输入数据	data/model_input/q3_vot_scenarios.csv
建模输入数据	data/model_input/sampled_region_daily_pressure_baseline.csv
建模输入数据	data/model_input/search_keyword_rules.csv
建模输入数据	data/model_input/visitor_scale_anchor_fit.csv
建模输入数据	data/model_input/visitor_scale_censored_likelihood_anchor_fit.csv
建模输入数据	data/model_input/visitor_scale_quality_report.json
结果与中间图表	outputs/figures/.gitkeep
结果与中间图表	outputs/figures/example_figure.png

类别	文件（相对项目根目录）
结果与中间图表	outputs/figures/q1_city_annual_trend.png
结果与中间图表	outputs/figures/q1_regional_peak_pressure_2025.png
结果与中间图表	outputs/question1_analysis/README.md
结果与中间图表	outputs/question1_analysis/q1_analysis_quality_report.json
结果与中间图表	outputs/question1_analysis/q1_city_annual_trend.csv
结果与中间图表	outputs/question1_analysis/q1_city_daily_annual_constrained.csv
结果与中间图表	outputs/question1_analysis/q1_city_daily_decomposition.csv
结果与中间图表	outputs/question1_analysis/q1_city_entry_exit_time_profiles.csv
结果与中间图表	outputs/question1_analysis/q1_city_monthly_cyclic_profile.csv
结果与中间图表	outputs/question1_analysis/q1_factor_strength.csv
结果与中间图表	outputs/question1_analysis/q1_poi_semantic_coverage.csv
结果与中间图表	outputs/question1_analysis/q1_regional_pressure_decomposition.csv
结果与中间图表	outputs/question1_analysis/q1_regional_season_classification.csv
结果与中间图表	outputs/question1_analysis/q1_service_time_scenarios.csv
结果与中间图表	outputs/question1_analysis/tables/table1_data_scope.csv
结果与中间图表	outputs/question1_analysis/tables/table1_data_scope.md
结果与中间图表	outputs/question1_analysis/tables/table2_variables_parameters.csv

类别	文件（相对项目根目录）
结果与中间图表	outputs/question1_analysis/tables/table2_variables_parameters.md
结果与中间图表	outputs/question1_analysis/tables/table3_key_results_validation.csv
结果与中间图表	outputs/question1_analysis/tables/table3_key_results_validation.md
结果与中间图表	outputs/question2_analysis/q2_ablation_dynamic_covariates_2026.csv
结果与中间图表	outputs/question2_analysis/q2_annual_model_comparison.csv
结果与中间图表	outputs/question2_analysis/q2_city_monthly_forecast.png
结果与中间图表	outputs/question2_analysis/q2_city_monthly_forecast_2026.csv
结果与中间图表	outputs/question2_analysis/q2_daily_conformal_calibration_2026.csv
结果与中间图表	outputs/question2_analysis/q2_daily_model_comparison.csv
结果与中间图表	outputs/question2_analysis/q2_daily_rolling_origin_validation.csv
结果与中间图表	outputs/question2_analysis/q2_external_anchor_validation.csv
结果与中间图表	outputs/question2_analysis/q2_quality_report.json
结果与中间图表	outputs/question2_analysis/q2_region_daily_forecast_2026.csv
结果与中间图表	outputs/question2_analysis/q2_regional_search_lag_selection.csv
结果与中间图表	outputs/question2_analysis/q2_summer_peak_range_2026.csv
结果与中间图表	outputs/question2_analysis/q2_summer_regional_daily_forecast.png
结果与中间图表	outputs/question2_analysis/q2_weather_shock_2026.csv

类别	文件（相对项目根目录）
结果与中间图表	outputs/question3_analysis/q3_absolute_cost_experience_pareto_2026.csv
结果与中间图表	outputs/question3_analysis/q3_absolute_daily_optimized_plan_2026.csv
结果与中间图表	outputs/question3_analysis/q3_absolute_daily_resource_plan_2026.csv
结果与中间图表	outputs/question3_analysis/q3_absolute_resource_sensitivity_2026.csv
结果与中间图表	outputs/question3_analysis/q3_absolute_seasonal_resource_plan_2026.csv
结果与中间图表	outputs/question3_analysis/q3_bdh_temporary_capacity_relaxation.csv
结果与中间图表	outputs/question3_analysis/q3_beidaihe_absolute_summer_temporary_plan_2026.csv
结果与中间图表	outputs/question4_analysis/README_ 封存说明.md
结果与中间图表	outputs/question4_analysis/q4_baseline_reoptimization_summary_2026.csv
结果与中间图表	outputs/question4_analysis/q4_event_counterfactual_2026.csv
结果与中间图表	outputs/question4_analysis/q4_lambda_plateau_diagnostic_2026.csv
结果与中间图表	outputs/question4_analysis/q4_lambda_tradeoff_2026.csv
结果与中间图表	outputs/question4_analysis/q4_rain_counterfactual_2026.csv
结果与中间图表	outputs/question4_analysis/q4_robustness_sensitivity_ranking.csv
结果与中间图表	outputs/question4_analysis/q4_scenario_service_gap.png
结果与中间图表	outputs/question4_analysis/q4_scenario_window_adjustment_summary.csv
结果与中间图表	outputs/question4_analysis/q4_sensitivity_ranking.png

类别	文件（相对项目根目录）
完整源程序	scripts/build_censored_likelihood_visitor_scale.py
完整源程序	scripts/build_censored_observation_diagnostics.py
完整源程序	scripts/build_delivery_diagnostics.py
完整源程序	scripts/build_hierarchical_censored_pressure.py
完整源程序	scripts/build_pressure_baseline.py
完整源程序	scripts/build_q1_paper_figures.py
完整源程序	scripts/build_q1_timeseries_analysis.py
完整源程序	scripts/build_q2_forecasts.py
完整源程序	scripts/build_q3_absolute_resource_plan.py
完整源程序	scripts/build_q4_scenario_analysis.py
完整源程序	scripts/build_search_theme_features.py
完整源程序	scripts/run_paper_mainline.py
完整源程序	src/analysis/q1_paper_figures.py
完整源程序	src/analysis/q1_timeseries.py
完整源程序	src/analysis/q2_forecast_outputs.py
完整源程序	src/analysis/q3_absolute_resource_outputs.py
完整源程序	src/analysis/q4_scenario_outputs.py
完整源程序	src/analysis/vot_outputs.py
完整源程序	src/features/censored_diagnostics.py
完整源程序	src/features/search_features.py
完整源程序	src/features/search_keyword_rules.py
完整源程序	src/models/hierarchical_censored_pressure.py
完整源程序	src/models/pressure_baseline.py
完整源程序	src/models/question2_forecasting.py
完整源程序	src/models/question3_absolute_capacity.py
完整源程序	src/models/question3_absolute_optimizer.py
完整源程序	src/models/question3_staff_scheduling.py
完整源程序	src/models/question4_scenarios.py
完整源程序	src/models/scale_calibration.py
完整源程序	src/models/value_of_time.py
完整源程序	src/utils/delivery_status.py
完整源程序	src/utils/paper_mainline.py

附录 B　完整源程序代码

以下仅列出一键运行 scripts/run_paper_mainline.py 所需的主线源文件；数据清洗、消融检验和备选方案代码不影响四问结果复现，故不随电子版附录重复列出。

2.1　共同预处理、校验与主线复现

scripts/build_delivery_diagnostics.py **功能说明：**汇总四问交付状态并写入可复核的完成记录。

```python
"""Create delivery-layer diagnostics for Q4 plateaus and POI semantic coverage."""

from pathlib import Path
import pandas as pd

ROOT = Path(__file__).resolve().parents[1]


def main() -> None:
    q4_dir = ROOT / "outputs" / "question4_analysis"
    tradeoff = pd.read_csv(q4_dir / "q4_lambda_tradeoff_2026.csv", encoding="utf-8-sig")
    key = ["mean_relative_cost", "mean_experience_loss", "max_temporary_spaces", "max_shuttle_vehicles", "max_staff_shifts"]
    tradeoff["same_as_previous"] = tradeoff[key].eq(tradeoff[key].shift()).all(axis=1)
    tradeoff["diagnostic"] = tradeoff["same_as_previous"].map({True:
  "discrete_solution_plateau__not_continuous_marginal_effect", False: "resource_configuration_switch"})
    tradeoff.to_csv(q4_dir / "q4_lambda_plateau_diagnostic_2026.csv", index=False, encoding="utf-8-sig")

    poi = pd.read_csv(ROOT / "data" / "clean" / "raw_cleaned" / "poi_qinhuangdao_purchased.csv", encoding="utf-8-sig")
    rules = {
        "hotel_accommodation": "酒店|宾馆|民宿|旅馆|客栈|度假村",
        "parking": "停车场|停车",
        "attraction_recreation": "景区|公园|海滩|浴场|博物馆|长城|动物园|乐园|广场",
        "transport": "车站|公交|码头|机场|高铁|铁路",
    }
    name = poi["name"].fillna("").astype(str)
    rows = []
    for label, pattern in rules.items():
        matched = name.str.contains(pattern, regex=True)
        rows.append({"semantic_family": label, "keyword_rule": pattern, "poi_count": int(matched.sum()),
  "share_of_purchased_poi": float(matched.mean()), "interpretation":
  "name_keyword_proxy_for_spatial_coverage_not_an_operating_capacity"})
    rows.append({"semantic_family": "all_purchased_poi", "keyword_rule": "not_applicable", "poi_count": int(len(poi)),
  "share_of_purchased_poi": 1.0, "interpretation": "raw purchased POI count"})
    pd.DataFrame(rows).to_csv(ROOT / "outputs" / "question1_analysis" / "q1_poi_semantic_coverage.csv", index=False,
  encoding="utf-8-sig")


if __name__ == "__main__":
    main()
```

scripts/build_search_theme_features.py **功能说明：**由清洗后的百度搜索观测构造主题化搜索特征。

```python
"""Build model-ready search theme features from cleaned B5c observations."""

from pathlib import Path
import sys

import pandas as pd
```

```

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))

from src.features.search_features import build_search_theme_features
from src.features.search_keyword_rules import qinhuangdao_search_keyword_rules


def main() -> None:
    source = ROOT / "data" / "clean" / "daily_search_heat_17_keywords_2016_2026.csv"
    output_dir = ROOT / "data" / "model_input"
    output_dir.mkdir(parents=True, exist_ok=True)
    heat = pd.read_csv(source, encoding="utf-8-sig")
    rules = qinhuangdao_search_keyword_rules()
    feature_source = heat.merge(rules, on="keyword", how="left", validate="many_to_one")
    features = build_search_theme_features(feature_source)
    rules.to_csv(output_dir / "search_keyword_rules.csv", index=False, encoding="utf-8-sig")
    features.to_csv(output_dir / "daily_search_theme_features_2016_2026.csv", index=False, encoding="utf-8-sig")


if __name__ == "__main__":
    main()
```

scripts/run_paper_mainline.py **功能说明：**按既定顺序调用 Q1–Q4 的全部主线脚本，并检查交付结果是否齐全。

```
"""Run only the four-question paper mainline; archived diagnostics stay optional."""

from pathlib import Path
import subprocess
import sys

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))
from src.utils.paper_mainline import PAPER_MAINLINE_SCRIPTS
from src.utils.delivery_status import assess_delivery_status, fallback_after_failed_script, freeze_verified_artifacts,
  write_delivery_status


def main() -> None:
    current_script = "not_started"
    try:
        for index, name in enumerate(PAPER_MAINLINE_SCRIPTS, start=1):
            current_script = name
            print(f"[{index}/{len(PAPER_MAINLINE_SCRIPTS)}] running {name}")
            subprocess.run([sys.executable, str(ROOT / "scripts" / name)], check=True)
    except subprocess.CalledProcessError:
        status = fallback_after_failed_script(ROOT, current_script)
        write_delivery_status(ROOT, status)
        print(f"[FALLBACK] {status['delivery_mode']}: {status['missing_questions']}")
        raise
    status = assess_delivery_status(ROOT)
    if status["delivery_mode"] != "full_model":
        write_delivery_status(ROOT, status)
        raise RuntimeError(f"mainline completed but delivery checkpoints failed: {status['missing_questions']}")
    freeze_verified_artifacts(ROOT)
    write_delivery_status(ROOT, status)
    print("[OK] paper mainline completed; verified Q1-Q4 artifact snapshot refreshed.")


if __name__ == "__main__":
    main()
```

src/analysis/vot_outputs.py **功能说明：**将游客时间损失按明确参数换算为可比较的货币成本。

```python
"""Attach transparent VOT-valued traveller time-loss diagnostics to plans."""

from __future__ import annotations

import pandas as pd

from src.models.value_of_time import estimate_traveler_time_loss


_REQUIRED_PLAN_COLUMNS = {
    "visitor_estimate_q50", "parking_unserved_spaces", "average_party_size",
    "transfer_share", "shuttle_shortfall", "recommended_shuttle_vehicles",
    "staff_shortfall", "recommended_total_staff_shifts",
}
_REQUIRED_VOT_COLUMNS = {
    "vot_scenario", "vot_cny_per_hour", "parking_delay_hours",
    "shuttle_delay_hours", "staff_delay_hours",
}


def attach_vot_costs(plan: pd.DataFrame, vot_scenarios: pd.DataFrame) -> pd.DataFrame:
    """Return plan with low/central/high visitor-time-loss estimates in CNY.

    The resource-selection objective is deliberately not changed: resource unit
    costs are not available as a verified local cash ledger.  These columns are
    traveller time-loss diagnostics under transparent scenario assumptions.
    """
    if missing := _REQUIRED_PLAN_COLUMNS - set(plan.columns):
        raise ValueError(f"plan missing columns: {sorted(missing)}")
    if missing := _REQUIRED_VOT_COLUMNS - set(vot_scenarios.columns):
        raise ValueError(f"vot_scenarios missing columns: {sorted(missing)}")
    if vot_scenarios["vot_scenario"].duplicated().any():
        raise ValueError("vot_scenarios must contain one row per vot_scenario")

    result = plan.copy()
    normalized = vot_scenarios.set_index("vot_scenario")
    if "central" not in normalized.index:
        raise ValueError("vot_scenarios must include central")
    for scenario, vot in normalized.iterrows():
        records = result.apply(
            lambda row: estimate_traveler_time_loss(
                daily_visitors=float(row["visitor_estimate_q50"]),
                parking_unserved_spaces=float(row["parking_unserved_spaces"]),
                average_party_size=float(row["average_party_size"]),
                parking_delay_hours=float(vot["parking_delay_hours"]),
                transfer_share=float(row["transfer_share"]),
                shuttle_shortfall_fraction=float(row["shuttle_shortfall"]) / max(float(row["recommended_shuttle_vehicles"]),
1.0),
                shuttle_delay_hours=float(vot["shuttle_delay_hours"]),
                staff_shortfall_fraction=float(row["staff_shortfall"]) / max(float(row["recommended_total_staff_shifts"]),
1.0),
                staff_delay_hours=float(vot["staff_delay_hours"]),
                vot_cny_per_hour=float(vot["vot_cny_per_hour"]),
            ),
            axis=1,
        ).apply(pd.Series)
        if scenario == "central":
            result["traveler_time_loss_hours"] = records["traveler_time_loss_hours"]
        result[f"traveler_time_loss_cny_{scenario}"] = records["traveler_time_loss_cny"]
    result["vot_cost_status"] = "traveler_time_loss_scenario_estimate_not_operator_cash_ledger"
    return result
```

src/features/search_features.py **功能说明：**对搜索词时间序列进行质量控制和主题聚合。

```python
"""Model-ready Baidu search features with explicit quality boundaries."""

from __future__ import annotations

import pandas as pd


_REQUIRED = {"date", "keyword", "search_index", "quality_grade", "theme"}


def build_search_theme_features(frame: pd.DataFrame) -> pd.DataFrame:
    """Return annual-standardized, non-weather A/B search-theme factors and lags."""
    missing = _REQUIRED.difference(frame.columns)
    if missing:
        raise ValueError(f"missing required columns: {sorted(missing)}")

    data = frame.copy()
    data["date"] = pd.to_datetime(data["date"], errors="coerce")
    data["search_index"] = pd.to_numeric(data["search_index"], errors="coerce")
    for column in ["keyword", "quality_grade", "theme"]:
        data[column] = data[column].astype("string").str.strip()
    is_weather = data["theme"].str.lower().eq("weather") | data["keyword"].str.contains("天气", na=False)
    data = data.loc[
        data["date"].notna()
        & data["search_index"].notna()
        & data["quality_grade"].isin(["A", "B"])
        & ~is_weather
    ].copy()
    if data.empty:
        return pd.DataFrame(columns=["date", "source_keyword_count"])

    data["year"] = data["date"].dt.year
    group = data.groupby(["keyword", "year"])["search_index"]
    mean = group.transform("mean")
    std = group.transform("std").fillna(0)
    data["annual_z"] = ((data["search_index"] - mean) / std.where(std.ne(0), 1)).fillna(0)

    theme_daily = (
        data.groupby(["date", "theme"], as_index=False)["annual_z"]
        .mean()
        .pivot(index="date", columns="theme", values="annual_z")
        .rename(columns=lambda theme: f"theme_{theme}")
        .reset_index()
        .sort_values("date")
        .reset_index(drop=True)
    )
    counts = data.groupby("date")["keyword"].nunique().rename("source_keyword_count").reset_index()
    result = theme_daily.merge(counts, on="date", how="left")
    theme_columns = [column for column in result.columns if column.startswith("theme_")]
    for column in theme_columns:
        result[f"{column}_lag_1"] = result[column].shift(1)
        result[f"{column}_lag_2"] = result[column].shift(2)
        result[f"{column}_lag_3"] = result[column].shift(3)
        result[f"{column}_lag_7"] = result[column].shift(7)
    return result
```

src/features/search_keyword_rules.py **功能说明：**给出搜索词语义分类和可用性筛选规则。

```python
"""Audited semantic and quality rules for the 17-keyword B5c source."""
```

```

import pandas as pd


def qinhuangdao_search_keyword_rules() -> pd.DataFrame:
    rows = [
        ("秦皇岛", "A", "destination"),
        ("北戴河", "A", "destination"),
        ("山海关", "A", "destination"),
        ("阿那亚", "A", "coastal_resort"),
        ("秦皇岛旅游", "A", "destination"),
        ("秦皇岛景点", "A", "attraction"),
        ("鸽子窝公园", "A", "attraction"),
        ("秦皇岛天气", "A", "weather"),
        ("北戴河旅游", "B", "destination"),
        ("北戴河景点", "B", "attraction"),
        ("山海关景点", "B", "attraction"),
        ("山海关旅游", "C", "intent"),
        ("山海关门票", "C", "intent"),
        ("北戴河攻略", "C", "intent"),
        ("秦皇岛攻略", "C", "intent"),
        ("老龙头门票", "C", "intent"),
        ("北戴河民宿", "D", "accommodation"),
    ]
    return pd.DataFrame(rows, columns=["keyword", "quality_grade", "theme"])
```

src/models/scale_calibration.py **功能说明：**将相对压力与报道锚点结合，形成带不确定性的游客规模。

```
"""Convert relative regional pressure into explicitly anchor-constrained scale scenarios."""

from __future__ import annotations

import numpy as np
import pandas as pd


def calibrate_daily_visitor_scale(
    pressure: pd.DataFrame,
    anchors: pd.DataFrame,
    single_anchor_width: float = 0.20,
    carry_forward_width: float = 0.10,
) -> tuple[pd.DataFrame, pd.DataFrame]:
    """Calibrate per-region-year pressure scales using only direct-scope anchor periods."""
    required_pressure = {"date", "region_code", "pressure_index"}
    required_anchors = {"region_code", "period_start", "period_end", "visitor_total", "scope_decision"}
    if missing := required_pressure.difference(pressure.columns):
        raise ValueError(f"pressure missing columns: {sorted(missing)}")
    if missing := required_anchors.difference(anchors.columns):
        raise ValueError(f"anchors missing columns: {sorted(missing)}")
    if not 0 <= single_anchor_width < 1 or not 0 <= carry_forward_width < 1:
        raise ValueError("scenario widths must be in [0, 1)")

    result = pressure.copy()
    result["date"] = pd.to_datetime(result["date"], errors="coerce")
    result["pressure_index"] = pd.to_numeric(result["pressure_index"], errors="coerce")
    anchor_frame = anchors.loc[anchors["scope_decision"].eq("direct_region_scale")].copy()
    anchor_frame["period_start"] = pd.to_datetime(anchor_frame["period_start"], errors="coerce")
    anchor_frame["period_end"] = pd.to_datetime(anchor_frame["period_end"], errors="coerce")
    anchor_frame["visitor_total"] = pd.to_numeric(anchor_frame["visitor_total"], errors="coerce")
    anchor_frame = anchor_frame.dropna(subset=["period_start", "period_end", "visitor_total"])

    fits: list[dict[str, object]] = []
```

```
    for anchor_id, anchor in anchor_frame.reset_index(drop=True).iterrows():
        mask = result["region_code"].eq(anchor["region_code"]) & result["date"].between(
            anchor["period_start"], anchor["period_end"]
        )
        pressure_total = float(result.loc[mask, "pressure_index"].sum())
        if pressure_total <= 0:
            continue
        fit_row = {
                "anchor_id": anchor_id + 1,
                "region_code": anchor["region_code"],
                "calibration_year": int(anchor["period_start"].year),
                "period_start": anchor["period_start"],
                "period_end": anchor["period_end"],
                "visitor_total": float(anchor["visitor_total"]),
                "pressure_period_total": pressure_total,
                "implied_scale": float(anchor["visitor_total"]) / pressure_total,
            }
        for provenance_column in ["source_dataset", "frequency", "metric_scope", "facility_or_scope"]:
            if provenance_column in anchor.index:
                fit_row[provenance_column] = anchor[provenance_column]
        fits.append(fit_row)
    anchor_fit = pd.DataFrame(fits)
    if anchor_fit.empty:
        raise ValueError("no direct anchor overlaps the supplied pressure series")

    if "frequency" not in anchor_fit.columns:
        anchor_fit["frequency"] = ""
    anchor_fit["calibration_role"] = "diagnostic_only"
    scale_rows: list[dict[str, object]] = []
    for (region_code, year), group in anchor_fit.groupby(["region_code", "calibration_year"]):
        annual_group = group.loc[group["frequency"].eq("annual")]
        primary_group = annual_group if not annual_group.empty else group
        anchor_fit.loc[primary_group.index, "calibration_role"] = "primary_scale"
        baseline_scale = float(np.exp(np.log(primary_group["implied_scale"]).mean()))
        if len(primary_group) == 1:
            low_scale = baseline_scale * (1 - single_anchor_width)
            high_scale = baseline_scale * (1 + single_anchor_width)
            interval_method = "single_anchor_scenario"
        else:
            low_scale = float(primary_group["implied_scale"].min())
            high_scale = float(primary_group["implied_scale"].max())
            interval_method = "multi_anchor_spread"
        scale_rows.append(
            {
                "region_code": region_code,
                "calibration_year": year,
                "baseline_scale": baseline_scale,
                "conservative_scale": low_scale,
                "high_scale": high_scale,
                "scale_anchor_count": int(len(primary_group)),
                "scale_interval_method": interval_method,
            }
        )
    scales = pd.DataFrame(scale_rows)
    anchor_fit = anchor_fit.merge(
        scales.loc[:, ["region_code", "calibration_year", "baseline_scale"]],
        on=["region_code", "calibration_year"],
        how="left",
    )
    anchor_fit["fitted_period_total"] = anchor_fit["pressure_period_total"] * anchor_fit["baseline_scale"]
    anchor_fit["relative_fit_error"] = (
        anchor_fit["fitted_period_total"] / anchor_fit["visitor_total"] - 1
    )

```

```
    result["calibration_year"] = result["date"].dt.year
    result = result.merge(scales, on=["region_code", "calibration_year"], how="left")
    result["scale_source_year"] = result["calibration_year"].where(result["baseline_scale"].notna())
    for region_code, indices in result.groupby("region_code").groups.items():
        row_indices = list(indices)
        subset = result.loc[row_indices].sort_values("date")
        for column in ["baseline_scale", "conservative_scale", "high_scale", "scale_anchor_count", "scale_interval_method",
  "scale_source_year"]:
            result.loc[subset.index, column] = subset[column].ffill()
        years_since_source = result.loc[subset.index, "calibration_year"] - result.loc[subset.index, "scale_source_year"]
        carried = years_since_source.gt(0)
        carried_indices = subset.index[carried]
        if len(carried_indices):
            result.loc[carried_indices, "conservative_scale"] *= (1 - carry_forward_width) **
  years_since_source.loc[carried_indices]
            result.loc[carried_indices, "high_scale"] *= (1 + carry_forward_width) ** years_since_source.loc[carried_indices]
            result.loc[carried_indices, "scale_interval_method"] = "carry_forward_scenario"
    if result["baseline_scale"].isna().any():
        raise ValueError("some pressure rows precede the first direct anchor for their region")
    result["visitor_estimate_baseline"] = result["pressure_index"] * result["baseline_scale"]
    result["visitor_estimate_conservative"] = result["pressure_index"] * result["conservative_scale"]
    result["visitor_estimate_high"] = result["pressure_index"] * result["high_scale"]
    result["estimate_label"] = "anchor_constrained_estimate"
    return result.sort_values(["date", "region_code"]).reset_index(drop=True), anchor_fit
```

src/utils/delivery_status.py **功能说明：**核验各问题的必要输出，并在失败时保留已验证成果。

```
"""Checkpointed paper-delivery state with a recoverable verified-artifact path."""

from __future__ import annotations

from datetime import datetime, timezone
import json
from pathlib import Path
import shutil


REQUIRED_ARTIFACTS: dict[str, tuple[str, ...]] = {
    "Q1": (
        "outputs/question1_analysis/q1_city_daily_annual_constrained.csv",
        "outputs/question1_analysis/q1_analysis_quality_report.json",
    ),
    "Q2": (
        "outputs/question2_analysis/q2_annual_model_comparison.csv",
        "outputs/question2_analysis/q2_region_daily_forecast_2026.csv",
        "outputs/question2_analysis/q2_daily_model_comparison.csv",
        "outputs/question2_analysis/q2_daily_conformal_calibration_2026.csv",
    ),
    "Q3": (
        "outputs/question3_analysis/q3_absolute_daily_resource_plan_2026.csv",
        "outputs/question3_analysis/q3_absolute_daily_optimized_plan_2026.csv",
        "outputs/question3_analysis/q3_absolute_resource_sensitivity_2026.csv",
    ),
    "Q4": (
        "outputs/question4_analysis/q4_event_counterfactual_2026.csv",
        "outputs/question4_analysis/q4_rain_counterfactual_2026.csv",
        "outputs/question4_analysis/q4_baseline_reoptimization_summary_2026.csv",
        "outputs/question4_analysis/q4_robustness_sensitivity_ranking.csv",
    ),
}


```

```python
def _missing_by_question(root: Path) -> dict[str, list[str]]:
    return {
        question: [relative for relative in files if not (root / relative).is_file()]
        for question, files in REQUIRED_ARTIFACTS.items()
        if any(not (root / relative).is_file() for relative in files)
    }


def assess_delivery_status(root: Path) -> dict[str, object]:
    """Assess current outputs without changing them."""
    root = Path(root)
    missing = _missing_by_question(root)
    snapshot = root / "outputs" / "verified_artifacts"
    if not missing:
        mode = "full_model"
        delivery_root = "outputs"
        scope = "Q1-Q4 full model outputs are present."
    elif snapshot.is_dir():
        mode = "verified_artifact_fallback"
        delivery_root = "outputs/verified_artifacts"
        scope = "Use only the last verified snapshot; do not combine it with partially regenerated outputs."
    else:
        mode = "no_deliverable_snapshot"
        delivery_root = None
        scope = "No complete current output and no verified snapshot are available."
    return {
        "delivery_mode": mode,
        "delivery_root": delivery_root,
        "missing_questions": sorted(missing),
        "missing_artifacts": missing,
        "scope_note": scope,
    }


def freeze_verified_artifacts(root: Path) -> Path:
    """Replace the verified snapshot only after all four checkpoints pass."""
    root = Path(root)
    status = assess_delivery_status(root)
    if status["delivery_mode"] != "full_model":
        raise ValueError("cannot freeze artifacts before all Q1-Q4 checkpoints pass")
    snapshot = root / "outputs" / "verified_artifacts"
    if snapshot.exists():
        shutil.rmtree(snapshot)
    for directory in ("question1_analysis", "question2_analysis", "question3_analysis", "question4_analysis"):
        shutil.copytree(root / "outputs" / directory, snapshot / directory)
    return snapshot


def fallback_after_failed_script(root: Path, failed_script: str) -> dict[str, object]:
    """Force the verified-snapshot mode after any mainline script failure."""
    root = Path(root)
    snapshot = root / "outputs" / "verified_artifacts"
    if snapshot.is_dir():
        mode = "verified_artifact_fallback"
        delivery_root: str | None = "outputs/verified_artifacts"
        scope = "A mainline script failed. Use only the prior verified snapshot; current outputs may be partial."
    else:
        mode = "no_deliverable_snapshot"
        delivery_root = None
        scope = "A mainline script failed and no prior verified snapshot is available."
    return {
        "delivery_mode": mode,
        "delivery_root": delivery_root,
        "missing_questions": [],
```

```
        "missing_artifacts": {},
        "failed_script": failed_script,
        "scope_note": scope,
    }


def write_delivery_status(root: Path, status: dict[str, object] | None = None) -> Path:
    """Persist an auditable delivery decision for the paper handoff."""
    root = Path(root)
    payload = dict(status or assess_delivery_status(root))
    payload["generated_at_utc"] = datetime.now(timezone.utc).replace(microsecond=0).isoformat()
    destination = root / "outputs" / "delivery_status.json"
    destination.parent.mkdir(parents=True, exist_ok=True)
    destination.write_text(json.dumps(payload, ensure_ascii=False, indent=2), encoding="utf-8")
    return destination
```

src/utils/paper_mainline.py **功能说明：**定义一键复现时各主线脚本的固定执行顺序。

```
"""Single, intentionally simple paper-production pipeline definition."""

PAPER_MAINLINE_SCRIPTS = (
    "build_search_theme_features.py",
    "build_pressure_baseline.py",
    "build_censored_observation_diagnostics.py",
    "build_hierarchical_censored_pressure.py",
    "build_censored_likelihood_visitor_scale.py",
    "build_q1_timeseries_analysis.py",
    "build_q1_paper_figures.py",
    "build_q2_forecasts.py",
    "build_q3_absolute_resource_plan.py",
    "build_q4_scenario_analysis.py",
    "build_delivery_diagnostics.py",
)
```

2.2 问题一：时序构建

scripts/build_censored_likelihood_visitor_scale.py **功能说明：**用外部锚点把相对压力校准为游客规模估计。

```
"""Calibrate Q1 relative pressure to visitor-scale estimates using reported anchors."""

from pathlib import Path
import sys
import pandas as pd
ROOT=Path(__file__).resolve().parents[1]; sys.path.insert(0,str(ROOT))
from src.models.scale_calibration import calibrate_daily_visitor_scale
p=pd.read_csv(ROOT/'data/model_input/daily_region_censored_likelihood_pressure_2023_2025.csv',encoding='utf-8-sig')
a=pd.read_csv(ROOT/'data/model_input/calibration_anchor_scope_ledger.csv',encoding='utf-8-sig')
e,f=calibrate_daily_visitor_scale(p,a)
e.to_csv(ROOT/'data/model_input/daily_region_visitor_scale_censored_likelihood_2023_2025.csv',index=False,encoding='utf-8-sig')
f.to_csv(ROOT/'data/model_input/visitor_scale_censored_likelihood_anchor_fit.csv',index=False,encoding='utf-8-sig')
print(len(e),len(f))
```

scripts/build_censored_observation_diagnostics.py **功能说明：**诊断景区排名观测的左删失结构与样本覆盖。

```
"""Build Q1 left-censored ranking-observation diagnostics from actual sampled dates."""
```

```

from __future__ import annotations

import json
from pathlib import Path
import sys

import pandas as pd

PROJECT_ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(PROJECT_ROOT))

from src.features.censored_diagnostics import build_censored_observation_diagnostics


def main() -> None:
    model_input_dir = PROJECT_ROOT / "data" / "model_input"
    panel = pd.read_csv(model_input_dir / "censored_attraction_observation_panel.csv", encoding="utf-8-sig")
    diagnostics = build_censored_observation_diagnostics(panel)
    diagnostics.to_csv(
        model_input_dir / "censored_observation_diagnostics.csv",
        index=False,
        encoding="utf-8-sig",
    )
    censor_rate = diagnostics["censor_rate"]
    quality = {
        "groups": int(len(diagnostics)),
        "valid_likelihood_groups": int(diagnostics["total_log_likelihood"].notna().sum()),
        "no_density_observation_groups": int(diagnostics["uncertainty_flag"].eq("no_density_observation").sum()),
        "no_positive_density_index_groups": int(diagnostics["uncertainty_flag"].eq("no_positive_density_index").sum()),
        "small_observed_sample_groups": int(diagnostics["uncertainty_flag"].eq("small_observed_sample").sum()),
        "censor_rate_summary": {
            "min": float(censor_rate.min()),
            "median": float(censor_rate.median()),
            "max": float(censor_rate.max()),
        },
        "note": "Diagnostics apply only to actual ranking sample dates; left-censored rows are not imputed visitor values.",
    }
    (model_input_dir / "censored_observation_quality_report.json").write_text(
        json.dumps(quality, ensure_ascii=False, indent=2), encoding="utf-8"
    )
    print(json.dumps(quality, ensure_ascii=False, indent=2))


if __name__ == "__main__":
    main()
```

scripts/build_hierarchical_censored_pressure.py **功能说明：**估计分区分层左删失客流压力，并输出连续日序列。

```

"""Fit the Q1 censored hierarchical pressure model and export daily regional pressure."""

from pathlib import Path
import sys
import pandas as pd
import json

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))
from src.models.hierarchical_censored_pressure import (
    prepare_censored_likelihood_data,
    fit_hierarchical_censored_pressure,
    generate_continuous_hierarchical_pressure,
```

```
)

panel = pd.read_csv(ROOT / 'data/model_input/censored_attraction_observation_panel.csv', encoding='utf-8-sig')
target_regions = ['BDH', 'HGA', 'SHG']
panel = panel.loc[panel['region_code'].isin(target_regions)].copy()
calendar = pd.read_csv(ROOT / 'data/clean/calendar_2015_2026.csv')
weather = pd.read_csv(ROOT / 'data/clean/daily_weather_2015_2025.csv')
search = pd.read_csv(ROOT / 'data/model_input/daily_search_theme_features_2016_2026.csv')
for frame in [calendar, weather, search]: frame['date'] = pd.to_datetime(frame['date'])
cov =
  calendar[['date','是否法定放假','是否周末']].merge(weather[['date','日均气温_℃','日降水量_mm']],on='date',how='left').merge(search[['date','theme_at
cov.columns =
  ['date','is_holiday','is_weekend','temperature','precipitation','search_attraction_lag1','search_coastal_lag1','search_destination_lag1']
data = prepare_censored_likelihood_data(panel, cov)
fit = fit_hierarchical_censored_pressure(data)
if not fit['converged']: raise RuntimeError(fit['message'])
out = generate_continuous_hierarchical_pressure(fit, cov.loc[cov.date.between('2023-01-01','2025-12-31')], target_regions)
out.to_csv(ROOT / 'data/model_input/daily_region_censored_likelihood_pressure_2023_2025.csv', index=False,
  encoding='utf-8-sig')
fit['sampled_pressure'].to_csv(ROOT / 'data/model_input/censored_likelihood_sampled_pressure.csv', index=False,
  encoding='utf-8-sig')
fit['parameter_diagnostics'].to_csv(ROOT / 'data/model_input/censored_likelihood_parameter_diagnostics.csv', index=False,
  encoding='utf-8-sig')
baseline = pd.read_csv(ROOT / 'data/model_input/daily_region_pressure_baseline_2023_2025.csv', encoding='utf-8-sig')
baseline['date'] = pd.to_datetime(baseline['date'])
comparison = fit['sampled_pressure'].merge(baseline[['date','region_code','pressure_index']], on=['date','region_code'],
  how='left', suffixes=('_censored','_ridge'))
comparison['absolute_difference'] = (comparison['pressure_index_censored'] - comparison['pressure_index_ridge']).abs()
comparison.to_csv(ROOT / 'data/model_input/censored_likelihood_vs_ridge_validation.csv', index=False, encoding='utf-8-sig')
report={
    'continuous_rows':int(len(out)),
    'regions':sorted(out.region_code.unique().tolist()),
    'objective':float(fit['objective']),
    'sampled_groups':int(len(fit['sampled_pressure'])),
    'dynamic_rho_adjacent_sample':float(fit['rho']),
    'mean_absolute_difference_vs_ridge':float(comparison['absolute_difference'].mean()),
    'comparison_note':'same-sampled-day output difference; not a rolling forecast error',
    'unsampled_day_projection':'joint_covariate_region_mean; dynamic innovation set to zero',
}
(ROOT /
  'data/model_input/censored_likelihood_quality_report.json').write_text(json.dumps(report,ensure_ascii=False,indent=2),encoding='utf-8')
print(len(out), fit['objective'])
```

scripts/build_pressure_baseline.py **功能说明：**拟合问题一的透明相对旅游压力基准模型。

```
"""Fit and export the first transparent relative-pressure baseline."""

from __future__ import annotations

import json
from pathlib import Path
import sys

import pandas as pd

PROJECT_ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(PROJECT_ROOT))

from src.models.pressure_baseline import fit_pressure_baseline, rolling_time_validation


def main() -> None:
```

```
    model_input_dir = PROJECT_ROOT / "data" / "model_input"
    clean_dir = PROJECT_ROOT / "data" / "clean"
    panel = pd.read_csv(model_input_dir / "censored_attraction_observation_panel.csv")
    calendar = pd.read_csv(clean_dir / "calendar_2015_2026.csv")
    weather = pd.read_csv(clean_dir / "daily_weather_2015_2025.csv")
    search = pd.read_csv(model_input_dir / "daily_search_theme_features_2016_2026.csv")
    calendar_dates = pd.to_datetime(calendar["date"], errors="coerce")
    weather_dates = pd.to_datetime(weather["date"], errors="coerce")
    start_date = max(calendar_dates.min(), pd.Timestamp("2023-01-01"))
    end_date = min(calendar_dates.max(), weather_dates.max(), pd.Timestamp("2025-12-31"))
    region_codes = ["BDH", "SHG", "HGA"]
    prediction_frame = pd.MultiIndex.from_product(
        [pd.date_range(start_date, end_date, freq="D"), region_codes],
        names=["date", "region_code"],
    ).to_frame(index=False)
    pressure = fit_pressure_baseline(
        panel,
        calendar,
        weather,
        search,
        prediction_frame=prediction_frame,
    )
    pressure.to_csv(
        model_input_dir / "daily_region_pressure_baseline_2023_2025.csv",
        index=False,
        encoding="utf-8-sig",
    )
    rolling_scores = rolling_time_validation(
        panel,
        calendar,
        weather,
        search,
        cutoffs=["2023-06-30", "2024-06-30"],
    )
    rolling_scores.to_csv(
        model_input_dir / "pressure_baseline_rolling_validation.csv",
        index=False,
        encoding="utf-8-sig",
    )
    quality = {
        "rows": int(len(pressure)),
        "date_start": str(pressure["date"].min().date()),
        "date_end": str(pressure["date"].max().date()),
        "region_codes": sorted(pressure["region_code"].unique().tolist()),
        "estimate_label": "relative_pressure_index",
        "rolling_validation_rows": int(len(rolling_scores)),
        "rolling_mae": rolling_scores["mae"].round(6).tolist(),
        "note": "Relative pressure only; no row is an observed or estimated absolute visitor count.",
    }
    (model_input_dir / "pressure_baseline_quality_report.json").write_text(
        json.dumps(quality, ensure_ascii=False, indent=2), encoding="utf-8"
    )
    print(json.dumps(quality, ensure_ascii=False, indent=2))


if __name__ == "__main__":
    main()
```

scripts/build_q1_paper_figures.py **功能说明：**从问题一结果生成论文图和结果表。

```
"""Render Q1 sealed analysis outputs as the paper's figures and tables."""

from pathlib import Path
import sys
```

```

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))

from src.analysis.q1_paper_figures import build_q1_paper_figures, build_q1_paper_tables

paths = build_q1_paper_figures(ROOT / "outputs/question1_analysis", ROOT / "outputs/figures")
table_dir = ROOT / "outputs/question1_analysis/tables"
table_dir.mkdir(parents=True, exist_ok=True)
def markdown_table(table):
    rows = ["| " + " | ".join(map(str, table.columns)) + " |", "|" + "|".join(["---"] * len(table.columns)) + "|"]
    rows.extend("| " + " | ".join(map(str, row)) + " |" for row in table.itertuples(index=False, name=None))
    return "\n".join(rows) + "\n"
for name, table in build_q1_paper_tables(ROOT / "outputs/question1_analysis").items():
    table.to_csv(table_dir / f"{name}.csv", index=False, encoding="utf-8-sig")
    (table_dir / f"{name}.md").write_text(markdown_table(table), encoding="utf-8")
print("\n".join(str(path) for path in paths.values()))
```

scripts/build_q1_timeseries_analysis.py **功能说明：**生成问题一的时序分解、校验和质量报告。

```
"""Build the auditable descriptive-analysis outputs required by Question 1."""

from pathlib import Path
import sys

import pandas as pd


ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))

from src.analysis.q1_timeseries import build_q1_analysis_outputs


annual = pd.read_csv(ROOT / "data/clean/annual_city_tourism_1999_2024.csv", encoding="utf-8-sig")
annual = annual.loc[annual["指标"].eq("国内旅游人次")].copy()
pressure = pd.read_csv(ROOT / "data/model_input/daily_region_censored_likelihood_pressure_2023_2025.csv", encoding="utf-8-sig")
regional_visitor_estimates = pd.read_csv(ROOT /
  "data/model_input/daily_region_visitor_scale_censored_likelihood_2023_2025.csv", encoding="utf-8-sig")
parameters = pd.read_csv(ROOT / "data/model_input/censored_likelihood_parameter_diagnostics.csv", encoding="utf-8-sig")
service = pd.read_csv(ROOT / "data/clean/hourly_service_time_evidence.csv", encoding="utf-8-sig")

report = build_q1_analysis_outputs(
    annual=annual,
    pressure=pressure,
    parameters=parameters,
    service_evidence=service,
    output_directory=ROOT / "outputs/question1_analysis",
    annual_value_column="数值",
    regional_visitor_estimates=regional_visitor_estimates,
)
print(report)
```

src/analysis/q1_paper_figures.py **功能说明：**将问题一分析结果绘制为论文使用的图表。

```
"""Transform sealed Q1 outputs into the compact figures and tables used in the paper."""

from pathlib import Path

import matplotlib
```

```
matplotlib.use("Agg", force=True)
import matplotlib.pyplot as plt
import pandas as pd


def build_q1_paper_tables(analysis_dir: Path) -> dict[str, pd.DataFrame]:
    quality = pd.read_json(analysis_dir / "q1_analysis_quality_report.json", typ="series")
    table1 = pd.DataFrame([
        ["全市年度国内游客量", "2002—2024", "年度", "千人次", "官方绝对量锚点"],
        ["三大片区旅游压力", "2023—2025", "日级", "相对指数", "时序构建与空间联动"],
        ["搜索、天气与节假日", "2016—2026/2023—2025", "日级", "指数/观测值", "协变量动态层"],
    ], columns=["数据项", "时间跨度", "粒度", "单位", "模型用途"])
    table2 = pd.DataFrame([
        ["Y_y", "全市年度国内游客量", "年度尺度绝对约束"],
        ["F_r,t", "片区日级旅游压力", "相对量；不等同真实日游客数"],
        ["X_t", "搜索、天气、节假日等协变量", "解释短期波动"],
        ["S_t", "隐状态（常态/旺季/冲击）", "刻画状态切换"],
    ], columns=["符号", "定义", "作用与口径"])
    table3 = pd.DataFrame([
        ["年度锚点", "2002—2024 年官方全市国内游客量", "用于年度总量约束"],
        ["日级输出", "2023—2025 年三大片区连续压力序列", "相对压力，不作为真实人数"],
        ["质量报告", str(quality.get("coverage", "见质量报告")), "详细指标见 q1_analysis_quality_report.json"],
    ], columns=["项目", "结果", "解释"])
    return {"table1_data_scope": table1, "table2_variables_parameters": table2, "table3_key_results_validation": table3}


def build_q1_paper_figures(analysis_dir: Path, output_dir: Path) -> dict[str, Path]:
    """Create paper-ready Q1 figures from sealed Q1 analysis outputs."""
    output_dir.mkdir(parents=True, exist_ok=True)
    annual = pd.read_csv(analysis_dir / "q1_city_annual_trend.csv", encoding="utf-8-sig")
    regional = pd.read_csv(analysis_dir / "q1_regional_pressure_decomposition.csv", encoding="utf-8-sig")
    plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "DejaVu Sans"]
    plt.rcParams["axes.unicode_minus"] = False

    fig, ax = plt.subplots(figsize=(9.2, 4.8))
    ax.bar(annual["year"], annual["official_tourists"], color="#91b7d9", label="官方国内游客量")
    ax.plot(annual["year"], annual["trend_component"], color="#b9413e", marker="o", ms=3, lw=2, label="长期趋势")
    ax.set(xlabel="年份", ylabel="国内游客量（千人次）")
    ax.legend(frameon=False, ncol=2); ax.grid(axis="y", alpha=.25); fig.tight_layout()
    annual_png = output_dir / "q1_city_annual_trend.png"
    fig.savefig(annual_png, dpi=300, bbox_inches="tight"); plt.close(fig)

    panel = regional[(regional["date"] >= "2025-06-15") & (regional["date"] <= "2025-10-10")].copy()
    labels = {"BDH": "北戴河", "HGA": "海港—阿那亚", "SHG": "山海关"}
    colors = {"BDH": "#2f6f9f", "HGA": "#c05a45", "SHG": "#5a9b72"}
    fig, ax = plt.subplots(figsize=(9.2, 4.8))
    for code, label in labels.items():
        d = panel[panel["region_code"] == code]
        ax.plot(pd.to_datetime(d["date"]), d["pressure_index"], lw=1.35, color=colors[code], label=label)
    ax.axvspan(pd.Timestamp("2025-07-01"), pd.Timestamp("2025-08-31"), color="#f4c95d", alpha=.13, label="暑期")
    ax.axvspan(pd.Timestamp("2025-10-01"), pd.Timestamp("2025-10-07"), color="#df7861", alpha=.13, label="国庆")
    ax.set(xlabel="日期", ylabel="旅游压力指数（相对量）")
    ax.legend(frameon=False, ncol=5, fontsize=8); ax.grid(axis="y", alpha=.25); fig.autofmt_xdate(); fig.tight_layout()
    regional_png = output_dir / "q1_regional_peak_pressure_2025.png"
    fig.savefig(regional_png, dpi=300, bbox_inches="tight"); plt.close(fig)
    return {"annual": annual_png, "regional": regional_png}
```

src/analysis/q1_timeseries.py **功能说明：**整理问题一的趋势、季节性与分区联动分析输出。

```
"""Auditable descriptive analysis for Question 1 tourism time-series outputs."""

from __future__ import annotations
```

```

import json
from pathlib import Path

import numpy as np
import pandas as pd


def _require_columns(frame: pd.DataFrame, columns: set[str]) -> None:
    missing = columns.difference(frame.columns)
    if missing:
        raise ValueError(f"missing required columns: {sorted(missing)}")
    if frame.empty:
        raise ValueError("input data frame is empty")


def analyze_city_annual_trend(source: pd.DataFrame, value_column: str = "数值") -> pd.DataFrame:
    """Return official annual totals with a transparent three-year smooth trend."""
    _require_columns(source, {"year", value_column})
    result = source.loc[:, ["year", value_column]].copy()
    result["year"] = pd.to_numeric(result["year"], errors="raise").astype(int)
    result["official_tourists"] = pd.to_numeric(result[value_column], errors="raise")
    result = result.sort_values("year").drop_duplicates("year", keep="last")
    result["trend_component"] = result["official_tourists"].rolling(3, center=True, min_periods=1).mean()
    result["random_component"] = result["official_tourists"] - result["trend_component"]
    result["unit"] = "thousand_person_visits"
    result["series_scope"] = "official_citywide_annual_domestic_tourists"
    return result.loc[:, ["year", "official_tourists", "trend_component", "random_component", "unit",
  "series_scope"]].reset_index(drop=True)


def decompose_regional_pressure(source: pd.DataFrame, rolling_window: int = 31) -> pd.DataFrame:
    """Additively decompose log regional pressure into trend, month season and residual."""
    _require_columns(source, {"date", "region_code", "pressure_index"})
    if rolling_window <= 0:
        raise ValueError("rolling_window must be positive")
    result = source.loc[:, ["date", "region_code", "pressure_index"]].copy()
    result["date"] = pd.to_datetime(result["date"], errors="raise")
    result["pressure_index"] = pd.to_numeric(result["pressure_index"], errors="raise").clip(lower=0)
    result = result.sort_values(["region_code", "date"]).reset_index(drop=True)
    result["log_pressure"] = np.log1p(result["pressure_index"])
    result["trend_component"] = result.groupby("region_code", group_keys=False)["log_pressure"].transform(
        lambda values: values.rolling(rolling_window, center=True, min_periods=1).mean()
    )
    result["month"] = result["date"].dt.month
    detrended = result["log_pressure"] - result["trend_component"]
    seasonal_lookup = detrended.groupby([result["region_code"], result["month"]]).mean()
    result["seasonal_component"] = [seasonal_lookup.loc[(region, month)] for region, month in zip(result["region_code"],
  result["month"])]
    result["random_component"] = result["log_pressure"] - result["trend_component"] - result["seasonal_component"]
    result["series_scope"] = "estimated_regional_daily_relative_pressure"
    return result.loc[:, ["date", "region_code", "pressure_index", "log_pressure", "trend_component", "seasonal_component",
  "random_component", "series_scope"]]


def classify_regional_seasons(source: pd.DataFrame) -> pd.DataFrame:
    """Classify each region-month from its multi-year median relative pressure."""
    _require_columns(source, {"date", "region_code", "pressure_index"})
    result = source.loc[:, ["date", "region_code", "pressure_index"]].copy()
    result["date"] = pd.to_datetime(result["date"], errors="raise")
    result["pressure_index"] = pd.to_numeric(result["pressure_index"], errors="raise").clip(lower=0)
    result["month"] = result["date"].dt.month
    monthly = result.groupby(["region_code", "month"],
  as_index=False)["pressure_index"].median().rename(columns={"pressure_index": "monthly_median_pressure"})
```

```
    quantiles = monthly.groupby("region_code")["monthly_median_pressure"].quantile([0.25, 0.75]).unstack().rename(columns={0.25: "q25", 0.75: "q75"})
    monthly = monthly.join(quantiles, on="region_code")
    monthly["season_label"] = np.select(
        [
            monthly["monthly_median_pressure"] > monthly["q75"],
            (monthly["monthly_median_pressure"] < monthly["q25"]) |
            (monthly["monthly_median_pressure"].eq(0) & monthly["monthly_median_pressure"].eq(monthly["q25"])),
        ],
        ["peak_season", "off_season"], default="shoulder_season",
    )
    monthly["classification_basis"] = "region_specific_2023_2025_monthly_median_pressure_quantiles"
    return monthly.loc[:, ["region_code", "month", "monthly_median_pressure", "q25", "q75", "season_label", "classification_basis"]].sort_values(["region_code", "month"]).reset_index(drop=True)


def rank_factor_strength(parameters: pd.DataFrame) -> pd.DataFrame:
    """Rank fitted covariates by absolute standardized joint-model coefficient."""
    _require_columns(parameters, {"parameter_group", "parameter", "estimate_standardized"})
    result = parameters.loc[
        parameters["parameter_group"].eq("covariate") & ~parameters["parameter"].eq("intercept"),
        ["parameter", "estimate_standardized"],
    ].copy()
    result["estimate_standardized"] = pd.to_numeric(result["estimate_standardized"], errors="raise")
    result["absolute_standardized_effect"] = result["estimate_standardized"].abs()
    result["effect_direction"] = np.where(result["estimate_standardized"] >= 0, "positive", "negative")
    result["rank"] = result["absolute_standardized_effect"].rank(method="first", ascending=False).astype(int)
    result["interpretation"] = "joint_model_standardized_association_not_causal_effect"
    return result.sort_values("rank").reset_index(drop=True)


def build_service_time_scenarios(evidence: pd.DataFrame) -> pd.DataFrame:
    """Preserve service-time evidence as scenarios without inventing hourly counts."""
    required = {"region_name", "period_start", "period_end", "day_type", "time_start", "time_end", "metric_name", "evidence_type", "data_status", "source_url", "is_hourly_count"}
    _require_columns(evidence, required)
    result = evidence.loc[:, sorted(required)].copy()
    result["period_start"] = pd.to_datetime(result["period_start"], errors="raise")
    result["period_end"] = pd.to_datetime(result["period_end"], errors="raise")
    result["hourly_visitor_count"] = np.nan
    result["not_observed_hourly_flow"] = True
    result["use_in_q1"] = "service_window_or_peak_time_scenario_only"
    return result.sort_values(["region_name", "period_start", "time_start"]).reset_index(drop=True)


def construct_annual_constrained_city_daily_series(
    regional_estimates: pd.DataFrame, annual_totals: pd.DataFrame,
    annual_value_column: str = "数值",
) -> pd.DataFrame:
    """Scale the three-core-area daily shape to each available official annual total."""
    _require_columns(regional_estimates, {"date", "region_code", "visitor_estimate_baseline"})
    _require_columns(annual_totals, {"year", annual_value_column})
    regional = regional_estimates.loc[:, ["date", "region_code", "visitor_estimate_baseline"]].copy()
    regional["date"] = pd.to_datetime(regional["date"], errors="raise")
    regional["visitor_estimate_baseline"] = pd.to_numeric(regional["visitor_estimate_baseline"], errors="raise").clip(lower=0)
    daily = regional.groupby("date", as_index=False)["visitor_estimate_baseline"].sum().rename(columns={"visitor_estimate_baseline": "three_region_daily_shape"})
    daily["year"] = daily["date"].dt.year
    official = annual_totals.loc[:, ["year", annual_value_column]].copy()
    official["year"] = pd.to_numeric(official["year"], errors="raise").astype(int)
    official["official_annual_tourists_thousand"] = pd.to_numeric(official[annual_value_column], errors="raise")
    daily = daily.merge(official.loc[:, ["year", "official_annual_tourists_thousand"]], on="year", how="inner")
    annual_shape = daily.groupby("year")["three_region_daily_shape"].transform("sum")
    if annual_shape.le(0).any():
```

```
        raise ValueError("regional annual shape must be positive for every constrained year")
    daily["annual_scale_factor"] = daily["official_annual_tourists_thousand"] * 1000 / annual_shape
    daily["city_daily_visitor_estimate"] = daily["three_region_daily_shape"] * daily["annual_scale_factor"]
    daily["estimate_label"] = "annual_anchor_constrained_city_daily_estimate"
    daily["series_scope"] = "citywide_daily_estimate__official_annual_total_constrained__three_core_regions_supply_shape"
    return daily.loc[:, ["date", "year", "three_region_daily_shape", "official_annual_tourists_thousand",
  "annual_scale_factor", "city_daily_visitor_estimate", "estimate_label",
  "series_scope"]].sort_values("date").reset_index(drop=True)


def _clock_minutes(value: object) -> int:
    hour, minute = str(value).split(":")[:2]
    return int(hour) * 60 + int(minute)


def _clock_text(minutes: int) -> str:
    return f"{minutes // 60:02d}:{minutes % 60:02d}"


def _triangular_weight(position: np.ndarray, left: float, mode: float, right: float) -> np.ndarray:
    rising = (position - left) / (mode - left)
    falling = (right - position) / (right - mode)
    return np.maximum(0.0, np.minimum(rising, falling))


def build_entry_exit_time_profiles(evidence: pd.DataFrame, bin_minutes: int = 30) -> pd.DataFrame:
    """Create normalized, assumption-driven intra-day entry/exit service profiles."""
    if bin_minutes <= 0:
        raise ValueError("bin_minutes must be positive")
    scenarios = build_service_time_scenarios(evidence)
    profiles: list[dict[str, object]] = []
    for scenario_id, row in scenarios.reset_index(drop=True).iterrows():
        start, end = _clock_minutes(row["time_start"]), _clock_minutes(row["time_end"])
        if end <= start:
            raise ValueError("time_end must be after time_start")
        metric = str(row["metric_name"])
        if "日出" in metric:
            scenario_type, entry_shape, exit_shape = "sunrise_viewing", (0.0, 0.18, 0.55), (0.30, 0.65, 1.0)
        elif "夜" in metric or "演艺" in metric:
            scenario_type, entry_shape, exit_shape = "night_tourism", (0.0, 0.50, 0.85), (0.30, 0.88, 1.0)
        else:
            scenario_type, entry_shape, exit_shape = "daytime_service", (0.0, 0.25, 0.70), (0.30, 0.78, 1.0)
        starts = np.arange(start, end, bin_minutes)
        ends = np.minimum(starts + bin_minutes, end)
        midpoint = (starts + ends) / 2
        position = (midpoint - start) / (end - start)
        entry = _triangular_weight(position, *entry_shape) * (ends - starts)
        exit_ = _triangular_weight(position, *exit_shape) * (ends - starts)
        entry = entry / entry.sum(); exit_ = exit_ / exit_.sum()
        for index in range(len(starts)):
            profiles.append({
                "scenario_id": scenario_id,
                "region_name": row["region_name"],
                "period_start": row["period_start"], "period_end": row["period_end"],
                "day_type": row["day_type"], "scenario_type": scenario_type,
                "time_bin_start": _clock_text(int(starts[index])), "time_bin_end": _clock_text(int(ends[index])),
                "entry_weight": float(entry[index]), "exit_weight": float(exit_[index]),
                "entry_peak_fraction": entry_shape[1], "exit_peak_fraction": exit_shape[1],
                "entry_peak_fraction_sensitivity": "base_plus_or_minus_0.10_of_service_window",
                "profile_status": "assumption_driven_not_observed_hourly_flow",
                "evidence_type": row["evidence_type"], "data_status": row["data_status"], "source_url": row["source_url"],
            })
    return pd.DataFrame(profiles)

```

```

def build_q1_analysis_outputs(
    annual: pd.DataFrame, pressure: pd.DataFrame, parameters: pd.DataFrame,
    service_evidence: pd.DataFrame, output_directory: str | Path,
    annual_value_column: str = "数值", regional_visitor_estimates: pd.DataFrame | None = None,
) -> dict[str, object]:
    """Build the complete auditable table set for Question 1 descriptive analysis."""
    output = Path(output_directory)
    output.mkdir(parents=True, exist_ok=True)
    city = analyze_city_annual_trend(annual, value_column=annual_value_column)
    decomposition = decompose_regional_pressure(pressure)
    seasons = classify_regional_seasons(pressure)
    factors = rank_factor_strength(parameters)
    service = build_service_time_scenarios(service_evidence)
    regional_visitor_estimates = pressure if regional_visitor_estimates is None else regional_visitor_estimates
    city_daily = construct_annual_constrained_city_daily_series(regional_visitor_estimates, annual, annual_value_column)
    city_for_decomposition = city_daily.loc[:, ["date",
  "city_daily_visitor_estimate"]].rename(columns={"city_daily_visitor_estimate": "pressure_index"})
    city_for_decomposition["region_code"] = "CITY"
    city_decomposition = decompose_regional_pressure(city_for_decomposition).drop(columns="region_code")
    city_monthly = city_daily.assign(month=city_daily["date"].dt.month).groupby(["year", "month"],
  as_index=False)["city_daily_visitor_estimate"].sum()
    city_monthly["annual_city_total"] = city_monthly.groupby("year")["city_daily_visitor_estimate"].transform("sum")
    city_monthly["monthly_share"] = city_monthly["city_daily_visitor_estimate"] / city_monthly["annual_city_total"]
    entry_exit = build_entry_exit_time_profiles(service_evidence)
    city.to_csv(output / "q1_city_annual_trend.csv", index=False, encoding="utf-8-sig")
    decomposition.to_csv(output / "q1_regional_pressure_decomposition.csv", index=False, encoding="utf-8-sig")
    seasons.to_csv(output / "q1_regional_season_classification.csv", index=False, encoding="utf-8-sig")
    factors.to_csv(output / "q1_factor_strength.csv", index=False, encoding="utf-8-sig")
    service.to_csv(output / "q1_service_time_scenarios.csv", index=False, encoding="utf-8-sig")
    city_daily.to_csv(output / "q1_city_daily_annual_constrained.csv", index=False, encoding="utf-8-sig")
    city_decomposition.to_csv(output / "q1_city_daily_decomposition.csv", index=False, encoding="utf-8-sig")
    city_monthly.to_csv(output / "q1_city_monthly_cyclic_profile.csv", index=False, encoding="utf-8-sig")
    entry_exit.to_csv(output / "q1_city_entry_exit_time_profiles.csv", index=False, encoding="utf-8-sig")
    report: dict[str, object] = {
        "city_annual_rows": int(len(city)),
        "city_annual_range": [int(city["year"].min()), int(city["year"].max())],
        "regional_pressure_rows": int(len(decomposition)),
        "regional_pressure_range": [str(decomposition["date"].min().date()), str(decomposition["date"].max().date())],
        "regions": sorted(decomposition["region_code"].unique().tolist()),
        "pressure_unit": "estimated_relative_pressure_index_not_official_daily_visitors",
        "hourly_flow_claim": "not_observed",
        "city_daily_rows": int(len(city_daily)),
        "city_daily_years": sorted(city_daily["year"].unique().tolist()),
        "city_daily_method": "official_annual_total_constrained_by_three_core_region_daily_shape",
        "entry_exit_profile_method": "normalized_triangular_service_scenario_with_peak_shift_sensitivity",
        "decomposition_method": "log1p_pressure__31day_centered_moving_average__calendar_month_mean",
    }
    (output / "q1_analysis_quality_report.json").write_text(json.dumps(report, ensure_ascii=False, indent=2), encoding="utf-8")
    return report
```

src/features/censored_diagnostics.py **功能说明：**计算左删失观测的覆盖、阈值和诊断指标。

```

"""Diagnostics for sampled ranking observations with left-censored attraction rows."""

from __future__ import annotations

from math import erf, pi, sqrt

import numpy as np
import pandas as pd

```

```
def _as_bool(values: pd.Series) -> pd.Series:
    if pd.api.types.is_bool_dtype(values):
        return values.fillna(False)
    return values.astype("string").str.lower().isin(["true", "1", "yes"])


def _normal_cdf(value: float) -> float:
    return 0.5 * (1.0 + erf(value / sqrt(2.0)))


def build_censored_observation_diagnostics(
    panel: pd.DataFrame,
    min_log_std: float = 0.25,
) -> pd.DataFrame:
    """Summarize observed ranking density and left-censored likelihood by sampled region-day."""
    required = {"date", "region_code", "is_observed", "visitor_index"}
    if missing := required.difference(panel.columns):
        raise ValueError(f"panel missing columns: {sorted(missing)}")
    if min_log_std <= 0:
        raise ValueError("min_log_std must be positive")

    source = panel.copy()
    source["date"] = pd.to_datetime(source["date"], errors="coerce")
    source["visitor_index"] = pd.to_numeric(source["visitor_index"], errors="coerce")
    source["is_observed"] = _as_bool(source["is_observed"])
    rows: list[dict[str, object]] = []
    for (date, region_code), group in source.groupby(["date", "region_code"], dropna=False):
        observed_values = group.loc[group["is_observed"], "visitor_index"]
        positive_observed = observed_values.loc[observed_values.gt(0)]
        density_count = int(group["is_observed"].sum())
        censored_count = int((~group["is_observed"]).sum())
        total_count = int(len(group))
        row: dict[str, object] = {
            "date": date,
            "region_code": region_code,
            "density_count": density_count,
            "left_censored_count": censored_count,
            "censor_rate": censored_count / total_count if total_count else np.nan,
            "censor_threshold_index": np.nan,
            "log_index_mean": np.nan,
            "log_index_std": np.nan,
            "density_log_likelihood": np.nan,
            "censor_log_likelihood": np.nan,
            "total_log_likelihood": np.nan,
            "uncertainty_flag": "no_density_observation",
        }
        if density_count == 0:
            rows.append(row)
            continue
        if positive_observed.empty:
            row["uncertainty_flag"] = "no_positive_density_index"
            rows.append(row)
            continue
        log_values = np.log1p(observed_values.loc[observed_values.ge(0)].dropna().to_numpy(dtype=float))
        log_mean = float(np.mean(log_values))
        raw_std = float(np.std(log_values, ddof=1)) if density_count >= 2 else 0.0
        log_std = max(raw_std, min_log_std)
        threshold = float(positive_observed.min())
        log_threshold = float(np.log1p(threshold))
        standardized_threshold = (log_threshold - log_mean) / log_std
        cdf_probability = max(_normal_cdf(standardized_threshold), 1e-12)
        density_ll = float(
            np.sum(-0.5 * ((log_values - log_mean) / log_std) ** 2 - np.log(log_std * sqrt(2 * pi)))
```

```
        )
        censor_ll = float(censored_count * np.log(cdf_probability))
        row.update(
            {
                "censor_threshold_index": threshold,
                "log_index_mean": log_mean,
                "log_index_std": log_std,
                "density_log_likelihood": density_ll,
                "censor_log_likelihood": censor_ll,
                "total_log_likelihood": density_ll + censor_ll,
                "uncertainty_flag": "small_observed_sample" if density_count <= 2 else "estimated_log_std",
            }
        )
        rows.append(row)
    return pd.DataFrame(rows).sort_values(["date", "region_code"]).reset_index(drop=True)
```

src/models/hierarchical_censored_pressure.py **功能说明:** 构建并拟合结合协变量、空间分区和左删失机制的压力模型。

```
"""Data preparation for the Question 1 hierarchical left-censored pressure model."""

from __future__ import annotations

import numpy as np
import pandas as pd
from scipy.optimize import minimize
from scipy.special import log_ndtr


def prepare_censored_likelihood_data(panel: pd.DataFrame, covariates: pd.DataFrame) -> dict[str, object]:
    """Separate ranked density observations from same-group left-censored counts."""
    required = {"date", "region_code", "attraction_name", "is_observed", "visitor_index"}
    if missing := required.difference(panel.columns):
        raise ValueError(f"panel missing columns: {sorted(missing)}")
    if "date" not in covariates.columns:
        raise ValueError("covariates missing column: date")
    source = panel.copy()
    source["date"] = pd.to_datetime(source["date"], errors="coerce")
    source["visitor_index"] = pd.to_numeric(source["visitor_index"], errors="coerce")
    source["is_observed"] = source["is_observed"].astype("string").str.lower().isin(["true", "1", "yes"])
    features = covariates.copy()
    features["date"] = pd.to_datetime(features["date"], errors="coerce")
    numeric_features = [
        column for column in features.columns
        if column != "date" and pd.api.types.is_numeric_dtype(features[column])
    ]
    for column in numeric_features:
        features[column] = pd.to_numeric(features[column], errors="coerce")
        features[column] = features[column].fillna(features[column].median())
    source = source.merge(features, on="date", how="left")
    groups = []
    observed = []
    censored = []
    for key, group in source.groupby(["date", "region_code"], sort=True):
        positive = group.loc[group["is_observed"] & group["visitor_index"].gt(0)]
        if positive.empty:
            continue
        group_id = len(groups)
        threshold_log = float(np.log1p(positive["visitor_index"].min()))
        record = {"group_id": group_id, "date": key[0], "region_code": key[1], "censored_count":
  int((~group["is_observed"]).sum()), "threshold_log": threshold_log}
        record.update({column: float(group.iloc[0][column]) for column in numeric_features})
        groups.append(record)
```

```
        for _, row in positive.iterrows():
            observed.append({"group_id": group_id, "attraction_name": row["attraction_name"], "observed_log_index": float(np.log1p(row["visitor_index"]))})
        for _, row in group.loc[~group["is_observed"]].iterrows():
            censored.append({"group_id": group_id, "attraction_name": row["attraction_name"], "threshold_log": threshold_log})
    observed_frame = pd.DataFrame(observed)
    censored_frame = pd.DataFrame(censored)
    group_frame = pd.DataFrame(groups)
    attraction_names = sorted(pd.concat([observed_frame.get("attraction_name", pd.Series(dtype=str)), censored_frame.get("attraction_name", pd.Series(dtype=str))]).dropna().unique().tolist())
    attraction_map = {name: index for index, name in enumerate(attraction_names)}
    if not observed_frame.empty:
        observed_frame["attraction_id"] = observed_frame["attraction_name"].map(attraction_map)
    if not censored_frame.empty:
        censored_frame["attraction_id"] = censored_frame["attraction_name"].map(attraction_map)
    regions = sorted(group_frame["region_code"].unique().tolist()) if not group_frame.empty else []
    region_map = {region: index for index, region in enumerate(regions)}
    if not group_frame.empty:
        group_frame["region_id"] = group_frame["region_code"].map(region_map)
        feature_matrix = group_frame.loc[:, numeric_features].to_numpy(dtype=float) if numeric_features else np.empty((len(group_frame), 0))
        feature_mean = feature_matrix.mean(axis=0) if numeric_features else np.empty(0)
        feature_std = feature_matrix.std(axis=0) if numeric_features else np.empty(0)
        feature_std[feature_std == 0] = 1.0
        standardized_features = (feature_matrix - feature_mean) / feature_std if numeric_features else feature_matrix
    else:
        feature_mean = feature_std = np.empty(0); standardized_features = np.empty((0, len(numeric_features)))
    attraction_region = np.zeros(len(attraction_names), dtype=int)
    if attraction_names:
        attraction_regions = pd.concat([observed_frame[["attraction_name", "group_id"]], censored_frame[["attraction_name", "group_id"]]]).merge(group_frame[["group_id", "region_id"]], on="group_id", how="left").drop_duplicates("attraction_name")
        attraction_region = attraction_regions.set_index("attraction_name").loc[attraction_names, "region_id"].to_numpy(dtype=int)
    return {
        "groups": group_frame,
        "observed": observed_frame,
        "censored": censored_frame,
        "observed_log_index": observed_frame.get("observed_log_index", pd.Series(dtype=float)).to_numpy(),
        "censored_count": group_frame.get("censored_count", pd.Series(dtype=int)).to_numpy(),
        "group_count": len(group_frame),
        "attraction_count": len(attraction_names),
        "region_count": len(regions),
        "region_names": regions,
        "feature_names": numeric_features,
        "feature_mean": feature_mean,
        "feature_std": feature_std,
        "standardized_features": standardized_features,
        "attraction_region": attraction_region,
    }


def negative_log_posterior(params: dict[str, object], data: dict[str, object], lambda_alpha: float = 0.1) -> float:
    """Evaluate observed normal density plus grouped left-censored normal probability."""
    groups = data["groups"]
    observed = data["observed"]
    eta = np.asarray(params["group_eta"], dtype=float)
    alpha = np.asarray(params["attraction_alpha"], dtype=float)
    sigma = float(np.exp(float(params["log_sigma"])))
    if len(eta) != len(groups) or sigma <= 0:
        return float("inf")
    observed_attraction_id = observed["attraction_id"].to_numpy(dtype=int)
    # The legacy objective predates censored-attraction effects and accepts an
    # observed-attraction-only alpha vector. Keep that public behavior intact.
    if len(alpha) <= observed_attraction_id.max(initial=-1):
```

```
        observed_attraction_id = pd.factorize(observed["attraction_name"], sort=True)[0]
    observed_mean = eta[observed["group_id"].to_numpy(dtype=int)] + alpha[observed_attraction_id]
    z = observed["observed_log_index"].to_numpy(dtype=float)
    density_nll = float(np.sum(0.5 * ((z - observed_mean) / sigma) ** 2 + np.log(sigma * np.sqrt(2 * np.pi))))
    threshold = groups["threshold_log"].to_numpy(dtype=float)
    standardized = (threshold - eta) / sigma
    cdf = 0.5 * (1 + np.vectorize(__import__("math").erf)(standardized / np.sqrt(2)))
    censor_nll = float(-np.sum(groups["censored_count"].to_numpy(dtype=float) * np.log(np.maximum(cdf, 1e-12))))
    return density_nll + censor_nll + lambda_alpha * float(np.sum(alpha**2))


def fit_censored_pressure_core(data: dict[str, object]) -> dict[str, object]:
    """Fit sampled-day latent pressures with attraction effects and left-censor likelihood."""
    group_count = int(data["group_count"])
    attraction_count = int(data["attraction_count"])
    initial = np.zeros(group_count + attraction_count + 1)
    initial[-1] = 0.0

    def objective(vector: np.ndarray) -> float:
        params = {"group_eta": vector[:group_count], "attraction_alpha": vector[group_count:-1], "log_sigma": vector[-1]}
        return negative_log_posterior(params, data)

    def gradient(vector: np.ndarray) -> np.ndarray:
        eta, alpha, log_sigma = vector[:group_count], vector[group_count:-1], vector[-1]
        sigma = np.exp(log_sigma); groups, obs = data["groups"], data["observed"]
        gids = obs["group_id"].to_numpy(int); aids = obs["attraction_id"].to_numpy(int); z =
  obs["observed_log_index"].to_numpy(float)
        residual = eta[gids] + alpha[aids] - z
        g_eta = np.bincount(gids, weights=residual / sigma**2, minlength=group_count)
        g_alpha = np.bincount(aids, weights=residual / sigma**2, minlength=attraction_count) + 0.2 * alpha
        q = (groups["threshold_log"].to_numpy(float) - eta) / sigma
        pdf = np.exp(-0.5*q*q) / np.sqrt(2*np.pi)
        cdf = np.maximum(0.5*(1 + np.vectorize(__import__('math').erf)(q/np.sqrt(2))), 1e-12)
        hazard = pdf / cdf; counts = groups["censored_count"].to_numpy(float)
        g_eta += counts * hazard / sigma
        g_log_sigma = np.sum(1 - (residual/sigma)**2) + np.sum(counts * hazard * q)
        return np.r_[g_eta, g_alpha, g_log_sigma]

    fit = minimize(objective, initial, jac=gradient, method="L-BFGS-B", bounds=[(None, None)] * (len(initial) - 1) + [(-4,
  4)], options={"maxiter": 2000, "maxfun": 100000})
    sampled_pressure = data["groups"].loc[:, ["date", "region_code"]].copy()
    sampled_pressure["pressure_index"] = np.maximum(np.expm1(fit.x[:group_count]), 0.0)
    sampled_pressure["estimate_label"] = "censored_likelihood_pressure_index"
    return {"converged": bool(fit.success), "group_eta": fit.x[:group_count], "attraction_alpha": fit.x[group_count:-1],
  "log_sigma": float(fit.x[-1]), "objective": float(fit.fun), "message": fit.message, "sampled_pressure": sampled_pressure}


def fit_hierarchical_censored_pressure(
    data: dict[str, object], lambda_alpha: float = 0.1, lambda_dynamic: float = 0.25,
    lambda_beta: float = 0.05, lambda_region: float = 0.05,
) -> dict[str, object]:
    """Jointly estimate covariates, region effects and AR-constrained sampled residuals."""
    group_count, attraction_count, region_count = int(data["group_count"]), int(data["attraction_count"]),
  int(data["region_count"])
    if group_count == 0:
        raise ValueError("no sampled groups available for hierarchical fit")
    x = np.column_stack([np.ones(group_count), np.asarray(data["standardized_features"], dtype=float)])
    feature_names = ["intercept", *list(data["feature_names"])]
    p = x.shape[1]
    groups, observed, censored = data["groups"], data["observed"], data["censored"]
    group_region = groups["region_id"].to_numpy(dtype=int)
    obs_g = observed["group_id"].to_numpy(dtype=int); obs_a = observed["attraction_id"].to_numpy(dtype=int); obs_z =
  observed["observed_log_index"].to_numpy(dtype=float)
    cen_g = censored.get("group_id", pd.Series(dtype=int)).to_numpy(dtype=int); cen_a = censored.get("attraction_id",
```

```
  pd.Series(dtype=int)).to_numpy(dtype=int); cen_threshold = censored.get("threshold_log",
  pd.Series(dtype=float)).to_numpy(dtype=float)
    attraction_region = np.asarray(data["attraction_region"], dtype=int)
    previous = np.full(group_count, -1, dtype=int)
    for _, frame in groups.sort_values(["region_code", "date"]).groupby("region_code", sort=False):
        identifiers = frame["group_id"].to_numpy(dtype=int)
        if len(identifiers) > 1:
            previous[identifiers[1:]] = identifiers[:-1]

    core = fit_censored_pressure_core(data)
    target = np.asarray(core["group_eta"], dtype=float)
    design = np.column_stack([x, pd.get_dummies(group_region, dtype=float).to_numpy()[:, 1:]])
    coefficients = np.linalg.pinv(design) @ target
    beta0 = coefficients[:p]; region0 = np.r_[0.0, coefficients[p:]]
    residual0 = target - x @ beta0 - region0[group_region]
    initial = np.r_[beta0, region0[1:], residual0, np.zeros(attraction_count), np.repeat(float(core["log_sigma"]),
  region_count), 0.0]
    b_start, u_start = p, p + region_count - 1
    a_start, sigma_start, rho_index = u_start + group_count, u_start + group_count + attraction_count, u_start + group_count +
  attraction_count + region_count

    def unpack(vector: np.ndarray):
        beta = vector[:p]
        regional = np.r_[0.0, vector[b_start:u_start]]
        u = vector[u_start:a_start]
        raw_alpha = vector[a_start:sigma_start]
        alpha = raw_alpha.copy()
        for region_id in range(region_count):
            mask = attraction_region == region_id
            if mask.any(): alpha[mask] -= alpha[mask].mean()
        sigma = np.exp(vector[sigma_start:rho_index])
        rho = np.tanh(vector[rho_index])
        eta = x @ beta + regional[group_region] + u
        return beta, regional, u, raw_alpha, alpha, sigma, rho, eta

    def objective_gradient(vector: np.ndarray):
        beta, regional, u, raw_alpha, alpha, sigma, rho, eta = unpack(vector)
        sigma_group = sigma[group_region]
        residual = eta[obs_g] + alpha[obs_a] - obs_z
        value = float(np.sum(0.5 * (residual / sigma_group[obs_g]) ** 2 + np.log(sigma_group[obs_g] * np.sqrt(2 * np.pi))))
        g_eta = np.bincount(obs_g, weights=residual / sigma_group[obs_g] ** 2, minlength=group_count)
        g_alpha = np.bincount(obs_a, weights=residual / sigma_group[obs_g] ** 2, minlength=attraction_count)
        g_log_sigma = np.bincount(group_region[obs_g], weights=1 - (residual / sigma_group[obs_g]) ** 2,
  minlength=region_count)
        if len(cen_g):
            q = (cen_threshold - eta[cen_g] - alpha[cen_a]) / sigma_group[cen_g]
            log_cdf = log_ndtr(q)
            value -= float(np.sum(log_cdf))
            log_pdf = -0.5 * q * q - 0.5 * np.log(2 * np.pi)
            hazard = np.exp(np.minimum(log_pdf - log_cdf, 50.0))
            contribution = hazard / sigma_group[cen_g]
            g_eta += np.bincount(cen_g, weights=contribution, minlength=group_count)
            g_alpha += np.bincount(cen_a, weights=contribution, minlength=attraction_count)
            g_log_sigma += np.bincount(group_region[cen_g], weights=hazard * q, minlength=region_count)
        value += lambda_alpha * float(np.sum(alpha ** 2)) + lambda_beta * float(np.sum(beta[1:] ** 2)) + lambda_region *
  float(np.sum(regional[1:] ** 2))
        g_alpha += 2 * lambda_alpha * alpha
        g_beta = x.T @ g_eta; g_beta[1:] += 2 * lambda_beta * beta[1:]
        g_region = np.bincount(group_region, weights=g_eta, minlength=region_count); g_region[1:] += 2 * lambda_region *
  regional[1:]
        g_u = g_eta.copy(); g_rho = 0.0
        current = np.flatnonzero(previous >= 0)
        if len(current):
            prior = previous[current]; dynamic_residual = u[current] - rho * u[prior]
```

```
            value += lambda_dynamic * float(np.sum(dynamic_residual ** 2))
            g_u[current] += 2 * lambda_dynamic * dynamic_residual
            g_u[prior] -= 2 * lambda_dynamic * rho * dynamic_residual
            g_rho = float(np.sum(-2 * lambda_dynamic * dynamic_residual * u[prior]) * (1 - rho ** 2))
        centered_alpha_gradient = g_alpha.copy()
        for region_id in range(region_count):
            mask = attraction_region == region_id
            if mask.any(): centered_alpha_gradient[mask] -= centered_alpha_gradient[mask].mean()
        gradient = np.r_[g_beta, g_region[1:], g_u, centered_alpha_gradient, g_log_sigma, g_rho]
        return value, gradient

    def objective(vector: np.ndarray) -> float:
        return objective_gradient(vector)[0]

    def gradient(vector: np.ndarray) -> np.ndarray:
        return objective_gradient(vector)[1]

    bounds = [(None, None)] * len(initial)
    for index in range(sigma_start, rho_index): bounds[index] = (-4.0, 4.0)
    fit = minimize(objective, initial, jac=gradient, method="L-BFGS-B", bounds=bounds, options={"maxiter": 3000, "maxfun": 150000, "ftol": 1e-9})
    beta, regional, u, raw_alpha, alpha, sigma, rho, eta = unpack(fit.x)
    sampled = groups.loc[:, ["date", "region_code"]].copy()
    sampled["pressure_index"] = np.maximum(np.expm1(eta), 0.0)
    sampled["estimate_label"] = "censored_likelihood_pressure_index"
    diagnostics = pd.concat([
        pd.DataFrame({"parameter_group": "covariate", "parameter": feature_names, "estimate_standardized": beta}),
        pd.DataFrame({"parameter_group": "region_effect", "parameter": data["region_names"], "estimate_standardized": regional}),
        pd.DataFrame({"parameter_group": "dispersion", "parameter": data["region_names"], "estimate_standardized": sigma}),
        pd.DataFrame({"parameter_group": ["dynamic"], "parameter": ["rho_adjacent_sample"], "estimate_standardized": [rho]}),
    ], ignore_index=True)
    return {"converged": bool(fit.success) and bool(np.all(sigma > 0)), "objective": float(fit.fun), "message": str(fit.message), "sampled_pressure": sampled, "parameter_diagnostics": diagnostics, "group_eta": eta, "covariate_beta": beta, "region_effect": regional, "attraction_alpha": alpha, "regional_sigma": sigma, "rho": float(rho), "feature_names": list(data["feature_names"]), "feature_mean": np.asarray(data["feature_mean"], dtype=float), "feature_std": np.asarray(data["feature_std"], dtype=float), "region_names": list(data["region_names"])}


def generate_continuous_hierarchical_pressure(fit: dict[str, object], covariates: pd.DataFrame, regions: list[str]) -> pd.DataFrame:
    """Generate calendar-day pressure from the fitted joint covariate and region layer.

    Unsampled days use the conditional regional mean (innovation equal to zero),
    rather than a second, separately fitted ridge model.
    """
    if "date" not in covariates.columns:
        raise ValueError("covariates missing column: date")
    feature_names = list(fit["feature_names"])
    missing = set(feature_names).difference(covariates.columns)
    if missing:
        raise ValueError(f"covariates missing fitted features: {sorted(missing)}")
    cov = covariates.copy()
    cov["date"] = pd.to_datetime(cov["date"], errors="coerce")
    for index, column in enumerate(feature_names):
        cov[column] = pd.to_numeric(cov[column], errors="coerce").fillna(float(np.asarray(fit["feature_mean"])[index]))
    grid = pd.MultiIndex.from_product([cov["date"].dropna().drop_duplicates().sort_values(), regions], names=["date", "region_code"]).to_frame(index=False)
    grid = grid.merge(cov[["date", *feature_names]], on="date", how="left")
    standardized = (grid[feature_names].to_numpy(dtype=float) - np.asarray(fit["feature_mean"])) / np.asarray(fit["feature_std"])
    design = np.column_stack([np.ones(len(grid)), standardized])
    fitted_regions = {name: index for index, name in enumerate(fit["region_names"])}
    unknown = set(regions).difference(fitted_regions)
```

```
    if unknown:
        raise ValueError(f"regions not present in fitted model: {sorted(unknown)}")
    region_effect = np.asarray(fit["region_effect"], dtype=float)
    eta = design @ np.asarray(fit["covariate_beta"], dtype=float) + grid["region_code"].map(lambda value:
  region_effect[fitted_regions[value]]).to_numpy(dtype=float)
    grid["pressure_index"] = np.maximum(np.expm1(eta), 0.0)
    grid["estimate_label"] = "censored_likelihood_pressure_index"
    grid["projection_source"] = "joint_covariate_region_mean"
    return grid.loc[:, ["date", "region_code", "pressure_index", "estimate_label", "projection_source"]].sort_values(["date",
  "region_code"]).reset_index(drop=True)


def project_continuous_pressure(sampled_pressure: pd.DataFrame, covariates: pd.DataFrame, regions: list[str]) -> pd.DataFrame:
    """Project censored-likelihood sampled pressures to requested calendar days with ridge covariates."""
    source = sampled_pressure.copy(); source["date"] = pd.to_datetime(source["date"])
    cov = covariates.copy(); cov["date"] = pd.to_datetime(cov["date"])
    numeric = [c for c in cov.columns if c != "date" and pd.api.types.is_numeric_dtype(cov[c])]
    grid = pd.MultiIndex.from_product([cov["date"].drop_duplicates(), regions],
  names=["date","region_code"]).to_frame(index=False)
    train = source.merge(cov, on="date", how="left"); pred = grid.merge(cov, on="date", how="left")
    for c in numeric:
        fill = train[c].median(); train[c] = train[c].fillna(fill); pred[c] = pred[c].fillna(fill)
    all_regions = pd.get_dummies(pd.concat([train["region_code"], pred["region_code"]]), dtype=float)
    x = np.column_stack([np.ones(len(train)), train[numeric].to_numpy(float), all_regions.iloc[:len(train)].to_numpy(float)])
    xp = np.column_stack([np.ones(len(pred)), pred[numeric].to_numpy(float), all_regions.iloc[len(train):].to_numpy(float)])
    coef = np.linalg.pinv(x.T @ x + np.eye(x.shape[1]) * 1.0) @ x.T @ np.log1p(train["pressure_index"].to_numpy(float))
    pred["pressure_index"] = np.maximum(np.expm1(xp @ coef), 0.0); pred["estimate_label"] =
  "censored_likelihood_pressure_index"
    return pred.loc[:,
  ["date","region_code","pressure_index","estimate_label"]].sort_values(["date","region_code"]).reset_index(drop=True)
```

src/models/pressure_baseline.py **功能说明：**提供问题一的两状态正则化压力基准模型及滚动检验。

```
"""Transparent two-state regularized baseline for relative tourism pressure."""

from __future__ import annotations

import numpy as np
import pandas as pd


def _calendar_flag(frame: pd.DataFrame, column: str) -> pd.Series:
    if column not in frame.columns:
        return pd.Series(0, index=frame.index, dtype=int)
    return pd.to_numeric(frame[column], errors="coerce").fillna(0).clip(0, 1)


def assign_calendar_regime(calendar: pd.DataFrame) -> pd.DataFrame:
    """Label high-pressure days from exogenous summer and statutory-holiday information."""
    if "date" not in calendar.columns:
        raise ValueError("calendar missing column: date")
    result = calendar.copy()
    result["date"] = pd.to_datetime(result["date"], errors="coerce")
    holiday = _calendar_flag(result, "是否法定放假")
    result["is_summer"] = result["date"].dt.month.isin([7, 8]).astype(int)
    result["is_holiday"] = holiday.clip(lower=0, upper=1)
    result["is_weekend"] = _calendar_flag(result, "是否周末")
    result["regime"] = np.where(
        (result["is_summer"] == 1) | (result["is_holiday"] == 1),
        "high_pressure",
        "normal",
    )
```

```
    return result


def fit_pressure_baseline(
    panel: pd.DataFrame,
    calendar: pd.DataFrame,
    weather: pd.DataFrame,
    search_features: pd.DataFrame,
    ridge_alpha: float = 10.0,
    prediction_frame: pd.DataFrame | None = None,
) -> pd.DataFrame:
    """Fit a ridge baseline to observed relative indices and predict sampled region-days."""
    required = {"date", "region_code", "is_observed", "visitor_index"}
    if missing := required.difference(panel.columns):
        raise ValueError(f"panel missing columns: {sorted(missing)}")
    if ridge_alpha < 0:
        raise ValueError("ridge_alpha must be non-negative")

    raw = panel.copy()
    raw["date"] = pd.to_datetime(raw["date"], errors="coerce")
    raw["visitor_index"] = pd.to_numeric(raw["visitor_index"], errors="coerce")
    observed = raw.loc[raw["is_observed"] & raw["visitor_index"].notna()]
    targets = (
        observed.groupby(["date", "region_code"], as_index=False)["visitor_index"]
        .median()
        .rename(columns={"visitor_index": "observed_visitor_index"})
    )
    training_pairs = raw.loc[:, ["date", "region_code"]].drop_duplicates()
    if prediction_frame is None:
        output_pairs = training_pairs.copy()
    else:
        prediction_required = {"date", "region_code"}
        if missing := prediction_required.difference(prediction_frame.columns):
            raise ValueError(f"prediction_frame missing columns: {sorted(missing)}")
        output_pairs = prediction_frame.loc[:, ["date", "region_code"]].copy()
        output_pairs["date"] = pd.to_datetime(output_pairs["date"], errors="coerce")
        output_pairs = output_pairs.dropna(subset=["date", "region_code"]).drop_duplicates()
    result = pd.concat(
        [training_pairs.assign(_fit_role="train"), output_pairs.assign(_fit_role="output")],
        ignore_index=True,
    ).merge(targets, on=["date", "region_code"], how="left")
    calendar_features = assign_calendar_regime(calendar)
    result = result.merge(
        calendar_features.loc[:, ["date", "is_summer", "is_holiday", "is_weekend", "regime"]],
        on="date",
        how="left",
    )

    weather = weather.copy()
    weather["date"] = pd.to_datetime(weather["date"], errors="coerce")
    weather_columns = [name for name in ["日均气温_℃", "日降水量_mm"] if name in weather.columns]
    if weather_columns:
        result = result.merge(weather.loc[:, ["date", *weather_columns]], on="date", how="left")
    search = search_features.copy()
    search["date"] = pd.to_datetime(search["date"], errors="coerce")
    search_columns = [name for name in search.columns if name.startswith("theme_") and name.endswith("_lag_1")]
    if search_columns:
        result = result.merge(search.loc[:, ["date", *search_columns]], on="date", how="left")

    numeric_columns = ["is_summer", "is_holiday", "is_weekend", *weather_columns, *search_columns]
    for column in numeric_columns:
        result[column] = pd.to_numeric(result[column], errors="coerce")
        result[column] = result[column].fillna(result[column].median()).fillna(0.0)
    region_dummies = pd.get_dummies(result["region_code"], prefix="region", dtype=float)
```

```
    feature_frame = pd.concat(
        [pd.Series(1.0, index=result.index, name="intercept"), result.loc[:, numeric_columns], region_dummies],
        axis=1,
    )
    target_mask = result["_fit_role"].eq("train") & result["observed_visitor_index"].notna()
    if not target_mask.any():
        raise ValueError("no observed visitor-index rows available for baseline fitting")
    x = feature_frame.to_numpy(dtype=float)
    train_x = x[target_mask.to_numpy()]
    train_y = np.log1p(result.loc[target_mask, "observed_visitor_index"].clip(lower=0).to_numpy(dtype=float))
    penalty = np.eye(train_x.shape[1]) * ridge_alpha
    penalty[0, 0] = 0.0
    coefficients = np.linalg.pinv(train_x.T @ train_x + penalty) @ train_x.T @ train_y
    prediction_log = x @ coefficients
    result["pressure_index"] = np.maximum(np.expm1(prediction_log), 0.0)
    residuals = train_y - train_x @ coefficients
    result["uncertainty_proxy"] = float(np.std(residuals, ddof=0))
    result["estimate_label"] = "relative_pressure_index"
    result = result.loc[result["_fit_role"].eq("output")].drop(columns="_fit_role")
    return result.sort_values(["date", "region_code"]).reset_index(drop=True)


def rolling_time_validation(
    panel: pd.DataFrame,
    calendar: pd.DataFrame,
    weather: pd.DataFrame,
    search_features: pd.DataFrame,
    cutoffs: list[str],
    ridge_alpha: float = 10.0,
) -> pd.DataFrame:
    """Evaluate forward predictions using only observations available before each cutoff."""
    raw = panel.copy()
    raw["date"] = pd.to_datetime(raw["date"], errors="coerce")
    raw["visitor_index"] = pd.to_numeric(raw["visitor_index"], errors="coerce")
    actual = (
        raw.loc[raw["is_observed"] & raw["visitor_index"].notna()]
        .groupby(["date", "region_code"], as_index=False)["visitor_index"]
        .median()
        .rename(columns={"visitor_index": "actual_visitor_index"})
    )
    scores: list[dict[str, object]] = []
    for cutoff_value in cutoffs:
        cutoff = pd.Timestamp(cutoff_value)
        train = raw.loc[raw["date"] <= cutoff]
        test = actual.loc[actual["date"] > cutoff]
        if train.empty or test.empty:
            continue
        prediction = fit_pressure_baseline(
            train,
            calendar,
            weather,
            search_features,
            ridge_alpha=ridge_alpha,
            prediction_frame=test.loc[:, ["date", "region_code"]],
        )
        comparison = prediction.merge(test, on=["date", "region_code"], how="inner")
        scores.append(
            {
                "train_end": cutoff,
                "test_rows": int(len(comparison)),
                "mae": float(
                    (comparison["pressure_index"] - comparison["actual_visitor_index"]).abs().mean()
                ),
            }
```

```
        )
    return pd.DataFrame(scores, columns=["train_end", "test_rows", "mae"])
```

2.3 问题二：客流预测

scripts/build_q2_forecasts.py **功能说明：**完成问题二的双尺度预测与外部锚点校验。

```
"""Run the Question 2 two-scale forecast pipeline."""

from pathlib import Path
import sys

import pandas as pd

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))

from src.analysis.q2_forecast_outputs import build_external_anchor_validation, build_question2_outputs,
  select_regional_search_lags


annual = pd.read_csv(ROOT / "data/clean/annual_city_tourism_1999_2024.csv", encoding="utf-8-sig")
annual = annual.loc[annual["\u6307\u6807"].eq("\u56fd\u5185\u65c5\u6e38\u4eba\u6b21"), ["year", "\u6570\u503c",
  "\u5355\u4f4d"]].rename(columns={"\u6570\u503c": "official_total"})
if not annual["\u5355\u4f4d"].eq("\u5343\u4eba\u6b21").all():
    raise ValueError("official annual unit must be thousand visitor-trips before conversion")
annual["official_total"] = annual["official_total"] * 1000.0
regional = pd.read_csv(ROOT / "data/model_input/daily_region_visitor_scale_censored_likelihood_2023_2025.csv",
  encoding="utf-8-sig")
observed = pd.read_csv(ROOT / "data/model_input/censored_attraction_observation_panel.csv", encoding="utf-8-sig")
weather = pd.read_csv(ROOT / "data/clean/daily_weather_2015_2025.csv", encoding="utf-8-sig")
weather = weather.loc[:, ["date", "\u65e5\u5747\u6c14\u6e29_\u2103",
  "\u65e5\u964d\u6c34\u91cf_mm"]].rename(columns={"\u65e5\u5747\u6c14\u6e29_\u2103": "temperature_c",
  "\u65e5\u964d\u6c34\u91cf_mm": "rain_mm"})
output = ROOT / "outputs/question2_analysis"
output.mkdir(parents=True, exist_ok=True)
search = pd.read_csv(ROOT / "data/model_input/daily_search_theme_features_2016_2026.csv", encoding="utf-8-sig")
model_regions = set(regional["region_code"].astype(str).unique())
lag_selection = select_regional_search_lags(observed.loc[observed["region_code"].astype(str).isin(model_regions)], search)
lag_selection.to_csv(output / "q2_regional_search_lag_selection.csv", index=False, encoding="utf-8-sig")
search_columns = ["date", "theme_destination_lag_1", "theme_destination_lag_2", "theme_destination_lag_3",
  "theme_destination_lag_7"]
search = search.loc[:, [column for column in search_columns if column in search.columns]]
base_covariates = weather.merge(search, on="date", how="outer")
regional_covariates = []
for row in lag_selection.itertuples(index=False):
    part = base_covariates.loc[:, ["date", "temperature_c", "rain_mm", row.selected_search_column]].copy()
    part = part.rename(columns={row.selected_search_column: "search_lag_1"})
    part["region_code"] = row.region_code
    regional_covariates.append(part)
covariates = pd.concat(regional_covariates, ignore_index=True)
print(build_question2_outputs(annual, regional, observed, output, daily_covariates=covariates))
anchors = pd.read_csv(ROOT / "data/clean/raw_cleaned/qinhuangdao_summer_regional_flow_anchors_2023_2025.csv",
  encoding="utf-8-sig")
city_daily = pd.read_csv(ROOT / "outputs/question1_analysis/q1_city_daily_annual_constrained.csv", encoding="utf-8-sig")
validation = build_external_anchor_validation(anchors, city_daily, regional)
validation.to_csv(output / "q2_external_anchor_validation.csv", index=False, encoding="utf-8-sig")
```

src/analysis/q2_forecast_outputs.py **功能说明：**生成问题二预测、滚动起点验证和

区域搜索滞后选择结果。

```
"""Auditable two-scale forecast outputs for Question 2."""

from __future__ import annotations

import json
from collections.abc import Sequence
from pathlib import Path

import matplotlib
import numpy as np
import pandas as pd
matplotlib.use("Agg", force=True)
import matplotlib.pyplot as plt

from src.models.question2_forecasting import (
    allocate_monthly_total,
    evaluate_annual_candidates,
    fit_daily_ridge_forecaster,
    forecast_annual_candidate,
    predict_daily_ridge_forecaster,
    scenario_adjust_weather,
    select_annual_candidate,
)


def _score(actual: pd.Series, prediction: pd.Series) -> dict[str, float]:
    residual = prediction.to_numpy(dtype=float) - actual.to_numpy(dtype=float)
    denominator = np.maximum((np.abs(actual.to_numpy(dtype=float)) + np.abs(prediction.to_numpy(dtype=float))) / 2.0, 1e-9)
    return {"mae": float(np.abs(residual).mean()), "rmse": float(np.sqrt(np.mean(residual**2))), "smape": float(100.0 *
  np.mean(np.abs(residual) / denominator))}


def q2_paper_figure_labels() -> dict[str, str]:
    """Return the Chinese, title-free labels used by paper figures."""
    return {
        "monthly_x": "月份",
        "monthly_y": "年度总量约束游客规模估计（人次）",
        "monthly_title": "",
        "summer_x": "日期",
        "summer_y": "锚点约束游客规模估计（人次）",
        "summer_title": "",
    }


def apply_relative_split_conformal_interval(
    prediction: pd.DataFrame,
    calibration_actual: pd.Series,
    calibration_point: pd.Series,
    coverage: float = 0.8,
) -> tuple[pd.DataFrame, float]:
    """Expand forecast intervals by a scale-free split-conformal residual radius.

    The calibration residual is expressed relative to the point prediction, so
    it can be transferred from an index-scale validation table to an
    anchor-scaled visitor forecast without mixing their units.
    """
    required = {"prediction_q10", "prediction_q50", "prediction_q90"}
    if missing := required - set(prediction.columns):
        raise ValueError(f"prediction missing columns: {sorted(missing)}")
    if not 0 < coverage < 1:
        raise ValueError("coverage must be in (0, 1)")
    actual = pd.to_numeric(calibration_actual, errors="coerce").to_numpy(dtype=float)
    point = pd.to_numeric(calibration_point, errors="coerce").to_numpy(dtype=float)
```

```
    usable = np.isfinite(actual) & np.isfinite(point) & (np.abs(point) > 1e-9)
    if not usable.any():
        raise ValueError("no usable calibration residuals")
    relative_errors = np.abs(actual[usable] - point[usable]) / np.abs(point[usable])
    radius = float(np.quantile(relative_errors, coverage, method="higher"))
    result = prediction.copy()
    lower = result["prediction_q50"] * max(0.0, 1.0 - radius)
    upper = result["prediction_q50"] * (1.0 + radius)
    result["prediction_q10"] = np.minimum(result["prediction_q10"], lower)
    result["prediction_q90"] = np.maximum(result["prediction_q90"], upper)
    return result, radius


def apply_regional_relative_split_conformal_interval(
    prediction: pd.DataFrame,
    calibration: pd.DataFrame,
    actual_column: str,
    point_column: str,
    coverage: float = 0.8,
    min_region_rows: int = 20,
) -> tuple[pd.DataFrame, pd.DataFrame]:
    """Calibrate relative conformal radii by region, with explicit global fallback.

    A shared radius obscures regional heterogeneity.  Each region with enough
    usable calibration observations obtains its own finite-sample radius; a
    smaller region falls back to the global radius and is labelled as such.
    """
    required_prediction = {"region_code", "prediction_q10", "prediction_q50", "prediction_q90"}
    required_calibration = {"region_code", actual_column, point_column}
    if required_prediction.difference(prediction.columns) or required_calibration.difference(calibration.columns):
        raise ValueError("regional conformal inputs are incomplete")
    if min_region_rows < 1:
        raise ValueError("min_region_rows must be positive")
    _, global_radius = apply_relative_split_conformal_interval(
        prediction,
        calibration_actual=calibration[actual_column],
        calibration_point=calibration[point_column],
        coverage=coverage,
    )
    result = prediction.copy()
    records: list[dict[str, object]] = []
    regions = sorted(result["region_code"].astype(str).unique())
    for region in regions:
        subset = calibration.loc[calibration["region_code"].astype(str).eq(region), [actual_column, point_column]]
        actual = pd.to_numeric(subset[actual_column], errors="coerce")
        point = pd.to_numeric(subset[point_column], errors="coerce")
        usable = actual.notna() & point.notna() & point.abs().gt(1e-9)
        usable_rows = int(usable.sum())
        if usable_rows >= min_region_rows:
            _, radius = apply_relative_split_conformal_interval(
                result.loc[result["region_code"].astype(str).eq(region)],
                calibration_actual=actual.loc[usable],
                calibration_point=point.loc[usable],
                coverage=coverage,
            )
            status = "region_specific"
        else:
            radius = global_radius
            status = "global_fallback_insufficient_region_calibration"
        mask = result["region_code"].astype(str).eq(region)
        lower = result.loc[mask, "prediction_q50"] * max(0.0, 1.0 - radius)
        upper = result.loc[mask, "prediction_q50"] * (1.0 + radius)
        result.loc[mask, "prediction_q10"] = np.minimum(result.loc[mask, "prediction_q10"], lower)
        result.loc[mask, "prediction_q90"] = np.maximum(result.loc[mask, "prediction_q90"], upper)
```

```
        records.append({"region_code": region, "relative_radius": float(radius), "calibration_rows": usable_rows, "calibration_status": status, "global_fallback_radius": float(global_radius)})
    return result, pd.DataFrame(records)


def _monthly_shape(regional: pd.DataFrame, end_year: int) -> np.ndarray:
    frame = regional.copy(); frame["date"] = pd.to_datetime(frame["date"], errors="raise")
    frame = frame.loc[frame["date"].dt.year <= end_year].copy()
    frame["month"] = frame["date"].dt.month
    weights = frame.groupby("month")["pressure_index"].sum().reindex(range(1, 13), fill_value=0.0).to_numpy(dtype=float)
    return weights if weights.sum() > 0 else np.ones(12, dtype=float)


def build_climatological_covariates(covariates: pd.DataFrame, forecast_year: int) -> pd.DataFrame:
    """Build future day-level weather/search scenarios from pre-forecast calendar-day medians only."""
    if "date" not in covariates.columns:
        raise ValueError("covariates missing date")
    source = covariates.copy(); source["date"] = pd.to_datetime(source["date"], errors="raise")
    source = source.loc[source["date"].dt.year < forecast_year].copy()
    feature_columns = [column for column in ["rain_mm", "temperature_c", "search_lag_1"] if column in source.columns]
    dates = pd.DataFrame({"date": pd.date_range(f"{forecast_year}-01-01", f"{forecast_year}-12-31", freq="D")})
    if not feature_columns:
        return dates
    source["month"] = source["date"].dt.month; source["day"] = source["date"].dt.day
    region_specific = "region_code" in source.columns
    if region_specific:
        regions = pd.DataFrame({"region_code": sorted(source["region_code"].astype(str).unique())})
        dates["_key"] = 1; regions["_key"] = 1
        dates = dates.merge(regions, on="_key", how="inner").drop(columns="_key")
    group_columns = (["region_code"] if region_specific else []) + ["month", "day"]
    profile = source.groupby(group_columns, as_index=False)[feature_columns].median()
    dates["month"] = dates["date"].dt.month; dates["day"] = dates["date"].dt.day
    result = dates.merge(profile, on=group_columns, how="left")
    monthly_group = (["region_code"] if region_specific else []) + ["month"]
    monthly = source.groupby(monthly_group, as_index=False)[feature_columns].median()
    result = result.merge(monthly, on=monthly_group, how="left", suffixes=("", "_monthly"))
    for column in feature_columns:
        result[column] = result[column].fillna(result[f"{column}_monthly"]).fillna(float(source[column].median()))
    result = result.drop(columns=[f"{column}_monthly" for column in feature_columns])
    return result.drop(columns=["month", "day"])


def select_regional_search_lags(
    observed: pd.DataFrame,
    search_features: pd.DataFrame,
    min_rows: int = 20,
) -> pd.DataFrame:
    """Choose a destination-search lag per region from observed pre-forecast dates.

    The selector is deliberately descriptive: it uses only the sparse, real
    attraction-index observations available before the 2026 forecast year and
    records its sample size and rank correlation instead of claiming a causal
    demand effect.
    """
    required_observed = {"date", "region_code", "visitor_index"}
    if required_observed.difference(observed.columns) or "date" not in search_features.columns:
        raise ValueError("observed and search features do not contain required columns")
    candidates = [
        column for column in [
            "theme_destination_lag_1", "theme_destination_lag_2",
            "theme_destination_lag_3", "theme_destination_lag_7",
        ] if column in search_features.columns
    ]
    if not candidates:
```

```
        raise ValueError("search features contain no supported destination-search lags")
    source = search_features.loc[:, ["date", *candidates]].copy()
    source["date"] = pd.to_datetime(source["date"], errors="raise")
    panel = observed.loc[:, ["date", "region_code", "visitor_index"]].copy()
    panel["date"] = pd.to_datetime(panel["date"], errors="raise")
    panel["visitor_index"] = pd.to_numeric(panel["visitor_index"], errors="coerce")
    panel = panel.dropna(subset=["visitor_index"]).groupby(["date", "region_code"], as_index=False)["visitor_index"].median()
    merged = panel.merge(source, on="date", how="left")
    lag_order = {column: int(column.rsplit("_", 1)[-1]) for column in candidates}
    rows: list[dict[str, object]] = []
    for region, group in merged.groupby("region_code", sort=True):
        scores = []
        for column in candidates:
            valid = group.loc[:, ["visitor_index", column]].dropna()
            has_variation = valid["visitor_index"].nunique() > 1 and valid[column].nunique() > 1
            correlation = valid["visitor_index"].corr(valid[column], method="spearman") if len(valid) >= min_rows and
  has_variation else np.nan
            scores.append((column, int(len(valid)), float(correlation) if pd.notna(correlation) else np.nan))
        eligible = [score for score in scores if score[1] >= min_rows and np.isfinite(score[2])]
        if eligible:
            selected, rows_used, correlation = sorted(eligible, key=lambda item: (-abs(item[2]), lag_order[item[0]]))[0]
            status = "selected_from_observed_training_dates"
        else:
            selected = min(candidates, key=lambda column: lag_order[column])
            rows_used = next(score[1] for score in scores if score[0] == selected)
            correlation = next(score[2] for score in scores if score[0] == selected)
            status = "fallback_shortest_lag_insufficient_observations"
        rows.append({"region_code": str(region), "selected_search_column": selected, "selected_lag_days": lag_order[selected],
  "observed_rows": rows_used, "spearman_correlation": correlation, "selection_status": status})
    return pd.DataFrame(rows)


def build_external_anchor_validation(anchors: pd.DataFrame, city_daily: pd.DataFrame, regional_daily: pd.DataFrame) ->
  pd.DataFrame:
    """Compare reported anchors only where their geographical and metric scopes truly match."""
    required = {"period_start", "period_end", "region_name", "visitor_count_10k_persons", "value_qualifier", "metric_scope",
  "source_url"}
    if required.difference(anchors.columns):
        raise ValueError("anchor table is incomplete")
    city = city_daily.copy(); city["date"] = pd.to_datetime(city["date"], errors="raise")
    regional = regional_daily.copy(); regional["date"] = pd.to_datetime(regional["date"], errors="raise")
    region_map = {"Beidaihe_District": "BDH", "Shanhaiguan_District": "SHG", "Beidaihe_New_District": "HGA", "Haigang_Aranya":
  "HGA"}
    rows: list[dict[str, object]] = []
    for _, anchor in anchors.iterrows():
        start, end = pd.Timestamp(anchor["period_start"]), pd.Timestamp(anchor["period_end"])
        reported = float(anchor["visitor_count_10k_persons"]) * 10000.0
        name, scope = str(anchor["region_name"]), str(anchor["metric_scope"])
        direct = name == "Qinhuangdao_City" and scope == "citywide_tourist_visitors" and str(anchor["value_qualifier"]) ==
  "exact"
        if name == "Qinhuangdao_City":
            values = city.loc[city["date"].between(start, end), "city_daily_visitor_estimate"]
        else:
            code = region_map.get(name)
            values = regional.loc[regional["date"].between(start, end) & regional["region_code"].eq(code),
  "visitor_estimate_baseline"] if code else pd.Series(dtype=float)
        estimate = float(values.sum()) if not values.empty else np.nan
        if direct and np.isfinite(estimate):
            status, error = "directly_comparable", estimate / reported - 1.0
        elif not np.isfinite(estimate):
            status, error = "model_series_not_available_for_period", np.nan
        else:
            status, error = "scope_not_directly_comparable", np.nan
        rows.append({"period_start": start, "period_end": end, "region_name": name, "metric_scope": scope,
```

```
    "reported_visitor_count": reported, "model_period_estimate": estimate, "comparison_status": status, "relative_error": error,
    "source_url": anchor["source_url"]})
    return pd.DataFrame(rows)


def _attach_covariates(frame: pd.DataFrame, covariates: pd.DataFrame | None) -> pd.DataFrame:
    if covariates is None:
        return frame.copy()
    result = frame.copy(); result["date"] = pd.to_datetime(result["date"], errors="raise")
    source = covariates.copy(); source["date"] = pd.to_datetime(source["date"], errors="raise")
    join_columns = ["date"]
    if "region_code" in source.columns:
        if "region_code" not in result.columns:
            raise ValueError("region-specific covariates require region_code in target frame")
        join_columns.append("region_code")
    columns = [*join_columns, *[column for column in ["rain_mm", "temperature_c", "search_lag_1"] if column in source.columns]]
    return result.merge(source.loc[:, columns].drop_duplicates(join_columns), on=join_columns, how="left")


def _daily_comparison(regional: pd.DataFrame, observed: pd.DataFrame, covariates: pd.DataFrame | None = None,
    min_holdout_rows: int = 50) -> tuple[pd.DataFrame, bool]:
    observed_frame = observed.loc[:, ["date", "region_code", "visitor_index"]].copy()
    observed_frame["date"] = pd.to_datetime(observed_frame["date"], errors="raise")
    observed_frame["visitor_index"] = pd.to_numeric(observed_frame["visitor_index"], errors="coerce")
    observed_frame = observed_frame.dropna(subset=["visitor_index"])
    regional_dates = pd.to_datetime(regional["date"], errors="raise")
    observed_frame = observed_frame.loc[
        observed_frame["region_code"].astype(str).isin(regional["region_code"].astype(str).unique())
        & observed_frame["date"].between(regional_dates.min(), regional_dates.max())
    ].copy()
    observed_panel = observed_frame.groupby(["date", "region_code"], as_index=False)["visitor_index"].median()
    if observed_panel.empty:
        return pd.DataFrame(columns=["candidate", "test_rows", "mae", "rmse", "smape"]), True
    year_counts = observed_panel.groupby(observed_panel["date"].dt.year).size()
    eligible_years = year_counts.loc[year_counts.ge(min_holdout_rows)]
    if eligible_years.empty:
        return pd.DataFrame(columns=["candidate", "test_rows", "mae", "rmse", "smape", "picp_80", "mean_interval_width",
    "holdout_year"]), True
    holdout_year = int(eligible_years.index.max())
    first_test_date = pd.Timestamp(f"{holdout_year}-01-01")
    observed_frame = observed_panel.loc[observed_panel["date"].dt.year.eq(holdout_year)].copy()
    train = regional.loc[pd.to_datetime(regional["date"], errors="raise") < first_test_date].copy()
    train = _attach_covariates(train, covariates)
    if len(train) < 3:
        return pd.DataFrame(columns=["candidate", "test_rows", "mae", "rmse", "smape"]), True
    features = _attach_covariates(observed_frame.loc[:, ["date", "region_code"]], covariates)
    calibration = regional.copy(); calibration["date"] = pd.to_datetime(calibration["date"], errors="raise")
    calibration = calibration.loc[calibration["date"] < first_test_date].merge(
        observed_panel, on=["date", "region_code"], how="inner"
    )
    calibration["pressure_index"] = pd.to_numeric(calibration["pressure_index"], errors="coerce")
    calibration["visitor_index"] = pd.to_numeric(calibration["visitor_index"], errors="coerce")
    calibration = calibration.loc[calibration["pressure_index"].gt(0) & calibration["visitor_index"].notna()]
    scales = calibration.groupby("region_code").apply(lambda x: float(np.median(x["visitor_index"] / x["pressure_index"])),
    include_groups=False).to_dict()
    rows = []
    for name, dynamic in [("seasonal_calendar_ridge", False), ("dynamic_ridge", True)]:
        model = fit_daily_ridge_forecaster(train, "pressure_index", dynamic=dynamic)
        predicted = predict_daily_ridge_forecaster(model, features)
        merged = observed_frame.merge(predicted, on=["date", "region_code"], how="inner")
        scale = merged["region_code"].map(scales).fillna(1.0)
        for q in ["q10", "q50", "q90"]:
            merged[f"prediction_{q}"] *= scale
        if merged.empty:
```

```
            continue
        calibration_features = _attach_covariates(calibration.loc[:, ["date", "region_code"]], covariates)
        calibration_prediction = calibration.loc[:, ["date", "region_code", "visitor_index"]].merge(
            predict_daily_ridge_forecaster(model, calibration_features), on=["date", "region_code"], how="inner"
        )
        calibration_scale = calibration_prediction["region_code"].map(scales).fillna(1.0)
        calibration_prediction["prediction_q50"] *= calibration_scale
        merged, regional_radii = apply_regional_relative_split_conformal_interval(
            merged,
            calibration_prediction,
            actual_column="visitor_index",
            point_column="prediction_q50",
        )
        radius_map = dict(zip(regional_radii["region_code"], regional_radii["relative_radius"]))
        row = {"candidate": name, "test_rows": int(len(merged)), "holdout_year": holdout_year,
  **_score(merged["visitor_index"], merged["prediction_q50"]), "picp_80": float(((merged["visitor_index"] >=
  merged["prediction_q10"]) & (merged["visitor_index"] <= merged["prediction_q90"])).mean()), "mean_interval_width":
  float((merged["prediction_q90"] - merged["prediction_q10"]).mean()), "interval_calibration":
  f"regional_relative_split_conformal_pre_{holdout_year}", "conformal_relative_radius":
  float(np.median(regional_radii["relative_radius"])), "conformal_relative_radius_by_region": json.dumps(radius_map,
  ensure_ascii=False, sort_keys=True), "conformal_calibration_by_region": regional_radii.to_json(orient="records",
  force_ascii=False), "calibration_rows": int(len(calibration_prediction))}
        rows.append(row)
    weekday_median = train.copy(); weekday_median["weekday"] = pd.to_datetime(weekday_median["date"]).dt.dayofweek
    test_naive = observed_frame.copy(); test_naive["weekday"] = test_naive["date"].dt.dayofweek
    historical = calibration.copy(); historical["weekday"] = historical["date"].dt.dayofweek
    medians = historical.groupby(["region_code", "weekday"])["visitor_index"].median()
    test_naive["prediction"] = [medians.get((row.region_code, row.weekday), historical["visitor_index"].median()) for row in
  test_naive.itertuples()]
    if not test_naive.empty:
        rows.append({"candidate": "historical_weekday_median", "test_rows": int(len(test_naive)), "holdout_year":
  holdout_year, **_score(test_naive["visitor_index"], test_naive["prediction"]), "picp_80": np.nan, "mean_interval_width":
  np.nan, "interval_calibration": "not_applicable", "conformal_relative_radius": np.nan,
  "conformal_relative_radius_by_region": "", "conformal_calibration_by_region": "", "calibration_rows": int(len(calibration))})
    comparison = pd.DataFrame(rows)
    return comparison, comparison.empty


def daily_rolling_origin_validation(
    regional: pd.DataFrame,
    observed: pd.DataFrame,
    covariates: pd.DataFrame | None = None,
    origins: Sequence[str | pd.Timestamp] = (),
    horizon_days: int = 92,
    min_test_rows: int = 20,
) -> pd.DataFrame:
    """Evaluate daily candidates on forward, fixed-horizon observation windows.

    Each origin is strictly causal: model fitting, anchor scaling, and conformal
    calibration use only dates before the origin.  Weather and lagged-search
    covariates in a held-out window are retained for conditional validation,
    which is recorded in the output rather than presented as an ex-ante test.
    """
    if horizon_days < 1 or min_test_rows < 1:
        raise ValueError("horizon_days and min_test_rows must be positive")
    required_regional = {"date", "region_code", "pressure_index"}
    required_observed = {"date", "region_code", "visitor_index"}
    if required_regional.difference(regional.columns) or required_observed.difference(observed.columns):
        raise ValueError("regional and observed tables do not contain required columns")
    columns = [
        "candidate", "origin_date", "window_start", "window_end", "training_end", "calibration_end",
        "evaluation_scope", "region_code", "test_rows", "mae", "rmse", "smape", "wape", "picp_80",
        "mean_interval_width", "interval_calibration", "conformal_relative_radius",
        "conformal_relative_radius_by_region", "conformal_calibration_by_region", "calibration_rows",
```

```
        "validation_covariate_mode",
    ]
    regional_frame = regional.copy()
    regional_frame["date"] = pd.to_datetime(regional_frame["date"], errors="raise")
    observed_frame = observed.loc[:, ["date", "region_code", "visitor_index"]].copy()
    observed_frame["date"] = pd.to_datetime(observed_frame["date"], errors="raise")
    observed_frame["visitor_index"] = pd.to_numeric(observed_frame["visitor_index"], errors="coerce")
    observed_panel = observed_frame.dropna(subset=["visitor_index"])
    observed_panel = observed_panel.loc[
        observed_panel["region_code"].astype(str).isin(regional_frame["region_code"].astype(str).unique())
        & observed_panel["date"].between(regional_frame["date"].min(), regional_frame["date"].max())
    ].groupby(["date", "region_code"], as_index=False)["visitor_index"].median()
    records: list[dict[str, object]] = []
    for raw_origin in origins:
        origin = pd.Timestamp(raw_origin).normalize()
        window_end = origin + pd.Timedelta(days=horizon_days - 1)
        train = regional_frame.loc[regional_frame["date"] < origin].copy()
        test = observed_panel.loc[observed_panel["date"].between(origin, window_end)].copy()
        calibration = regional_frame.loc[regional_frame["date"] < origin].merge(
            observed_panel, on=["date", "region_code"], how="inner"
        )
        calibration["pressure_index"] = pd.to_numeric(calibration["pressure_index"], errors="coerce")
        calibration["visitor_index"] = pd.to_numeric(calibration["visitor_index"], errors="coerce")
        calibration = calibration.loc[calibration["pressure_index"].gt(0) & calibration["visitor_index"].notna()].copy()
        if len(train) < 3 or len(test) < min_test_rows or calibration.empty:
            continue
        scales = calibration.groupby("region_code").apply(
            lambda group: float(np.median(group["visitor_index"] / group["pressure_index"])),
            include_groups=False,
        ).to_dict()
        train_features = _attach_covariates(train, covariates)
        test_features = _attach_covariates(test.loc[:, ["date", "region_code"]], covariates)
        calibration_features = _attach_covariates(calibration.loc[:, ["date", "region_code"]], covariates)
        for candidate, dynamic in [("seasonal_calendar_ridge", False), ("dynamic_ridge", True)]:
            model = fit_daily_ridge_forecaster(train_features, "pressure_index", dynamic=dynamic)
            prediction = predict_daily_ridge_forecaster(model, test_features)
            merged = test.merge(prediction, on=["date", "region_code"], how="inner")
            if merged.empty:
                continue
            scale = merged["region_code"].map(scales).fillna(1.0)
            for quantile in ["q10", "q50", "q90"]:
                merged[f"prediction_{quantile}"] *= scale
            calibration_prediction = calibration.loc[:, ["date", "region_code", "visitor_index"]].merge(
                predict_daily_ridge_forecaster(model, calibration_features), on=["date", "region_code"], how="inner"
            )
            calibration_prediction["prediction_q50"] *= calibration_prediction["region_code"].map(scales).fillna(1.0)
            merged, regional_radii = apply_regional_relative_split_conformal_interval(
                merged,
                calibration_prediction,
                actual_column="visitor_index",
                point_column="prediction_q50",
            )
            radius_map = dict(zip(regional_radii["region_code"], regional_radii["relative_radius"]))
            scopes: list[tuple[str, str, pd.DataFrame]] = [("all_dates_all_regions", "ALL", merged)]
            scopes.extend((f"region__{region}", str(region), group) for region, group in merged.groupby("region_code",
sort=True))
            summer = merged.loc[merged["date"].dt.month.isin([7, 8])]
            if not summer.empty:
                scopes.append(("summer_all_regions", "ALL", summer))
                scopes.extend((f"summer__{region}", str(region), group) for region, group in summer.groupby("region_code",
sort=True))
            for scope, region_code, group in scopes:
                if len(group) < min_test_rows:
                    continue
```

```
                scores = _score(group["visitor_index"], group["prediction_q50"])
                scores["wape"] = float(100.0 * np.abs(group["visitor_index"] - group["prediction_q50"]).sum() / max(np.abs(group["visitor_index"]).sum(), 1e-9))
                records.append({
                    "candidate": candidate, "origin_date": origin, "window_start": origin, "window_end": window_end,
                    "training_end": train["date"].max(), "calibration_end": calibration["date"].max(),
                    "evaluation_scope": scope, "region_code": region_code, "test_rows": int(len(group)), **scores,
                    "picp_80": float(((group["visitor_index"] >= group["prediction_q10"]) & (group["visitor_index"] <= group["prediction_q90"])).mean()),
                    "mean_interval_width": float((group["prediction_q90"] - group["prediction_q10"]).mean()),
                    "interval_calibration": f"regional_relative_split_conformal_pre_{origin.date()}",
                    "conformal_relative_radius": float(np.median(regional_radii["relative_radius"])),
                    "conformal_relative_radius_by_region": json.dumps(radius_map, ensure_ascii=False, sort_keys=True),
                    "conformal_calibration_by_region": regional_radii.to_json(orient="records", force_ascii=False),
                    "calibration_rows": int(len(calibration_prediction)),
                    "validation_covariate_mode": "conditional_on_realized_weather_and_lagged_search" if covariates is not None else "calendar_only",
                })
    return pd.DataFrame(records, columns=columns)


def build_q2_diagnostic_figures(monthly: pd.DataFrame, daily: pd.DataFrame, output_directory: str | Path) -> None:
    """Render two compact paper figures from forecast tables, never as observed counts."""
    output = Path(output_directory); output.mkdir(parents=True, exist_ok=True)
    labels = q2_paper_figure_labels()
    plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "DejaVu Sans"]
    plt.rcParams["axes.unicode_minus"] = False
    figure, axis = plt.subplots(figsize=(7.2, 3.6))
    axis.bar(monthly["month"], monthly["visitor_estimate"], color="#4C78A8")
    axis.set(xlabel=labels["monthly_x"], ylabel=labels["monthly_y"], title=labels["monthly_title"])
    if {"annual_forecast_lower", "annual_forecast_upper"}.issubset(monthly.columns):
        interval = f"年度 90% 预测区间: {monthly['annual_forecast_lower'].iloc[0]:,.0f}–{monthly['annual_forecast_upper'].iloc[0]:,.0f}"
        axis.text(0.02, 0.95, interval, transform=axis.transAxes, va="top")
    figure.tight_layout(); figure.savefig(output / "q2_city_monthly_forecast.png", dpi=220); plt.close(figure)
    frame = daily.copy(); frame["date"] = pd.to_datetime(frame["date"], errors="raise")
    frame = frame.loc[frame["date"].dt.month.isin([7, 8])]
    figure, axis = plt.subplots(figsize=(7.2, 3.6))
    region_labels = {"BDH": "北戴河", "HGA": "海港--阿那亚", "SHG": "山海关"}
    for region, group in frame.groupby("region_code"):
        group = group.sort_values("date")
        axis.plot(group["date"], group["visitor_estimate_q50"], label=region_labels.get(str(region), str(region)))
        axis.fill_between(group["date"], group["visitor_estimate_q10"], group["visitor_estimate_q90"], alpha=0.16)
    axis.set(xlabel=labels["summer_x"], ylabel=labels["summer_y"], title=labels["summer_title"])
    axis.legend(ncol=3, fontsize=8); figure.autofmt_xdate(); figure.tight_layout()
    figure.savefig(output / "q2_summer_regional_daily_forecast.png", dpi=220); plt.close(figure)


def build_question2_outputs(
    annual: pd.DataFrame,
    regional: pd.DataFrame,
    observed: pd.DataFrame,
    output_directory: str | Path,
    forecast_year: int = 2026,
    annual_value_column: str = "official_total",
    daily_covariates: pd.DataFrame | None = None,
) -> dict[str, object]:
    """Create Question 2 tables with explicit forecast and validation provenance."""
    required_annual = {"year", annual_value_column}
    required_regional = {"date", "region_code", "pressure_index", "baseline_scale"}
    if required_annual.difference(annual.columns) or required_regional.difference(regional.columns):
        raise ValueError("input tables do not contain the required Question 2 columns")
    output = Path(output_directory); output.mkdir(parents=True, exist_ok=True)
    annual_comparison = evaluate_annual_candidates(annual, annual_value_column)
```

```
    annual_comparison.to_csv(output / "q2_annual_model_comparison.csv", index=False, encoding="utf-8-sig")
    annual_selected_model = select_annual_candidate(annual_comparison)
    annual_forecast = forecast_annual_candidate(annual, annual_value_column, forecast_year, annual_selected_model)
    monthly = allocate_monthly_total(annual_forecast["point"], _monthly_shape(regional, forecast_year - 1), forecast_year)
    monthly["annual_forecast_lower"] = annual_forecast["lower"]
    monthly["annual_forecast_upper"] = annual_forecast["upper"]
    monthly.to_csv(output / f"q2_city_monthly_forecast_{forecast_year}.csv", index=False, encoding="utf-8-sig")

    comparison, no_independent_observation = _daily_comparison(regional, observed, daily_covariates)
    comparison.to_csv(output / "q2_daily_model_comparison.csv", index=False, encoding="utf-8-sig")
    rolling_validation = daily_rolling_origin_validation(
        regional,
        observed,
        daily_covariates,
        origins=("2024-01-01", "2024-04-01", "2024-07-01"),
        horizon_days=92,
        min_test_rows=20,
    )
    rolling_validation.to_csv(output / "q2_daily_rolling_origin_validation.csv", index=False, encoding="utf-8-sig")
    model_comparison = comparison.loc[comparison["candidate"].isin(["seasonal_calendar_ridge", "dynamic_ridge"])]
    selected_validation = model_comparison.sort_values("mae").iloc[0] if not model_comparison.empty else None
    selected_dynamic = bool(selected_validation["candidate"] == "dynamic_ridge") if selected_validation is not None else True
    conformal_radius = float(selected_validation["conformal_relative_radius"]) if selected_validation is not None and
pd.notna(selected_validation.get("conformal_relative_radius")) else np.nan
    regional_radius_map = json.loads(str(selected_validation["conformal_relative_radius_by_region"])) if selected_validation
is not None and pd.notna(selected_validation.get("conformal_relative_radius_by_region")) and
str(selected_validation["conformal_relative_radius_by_region"]).strip() else {}
    regional_calibration_records = json.loads(str(selected_validation["conformal_calibration_by_region"])) if
selected_validation is not None and pd.notna(selected_validation.get("conformal_calibration_by_region")) and
str(selected_validation["conformal_calibration_by_region"]).strip() else []
    interval_calibration = str(selected_validation["interval_calibration"]) if selected_validation is not None and
pd.notna(selected_validation.get("interval_calibration")) else "model_quantile_only_no_calibration_observations"
    train = regional.loc[pd.to_datetime(regional["date"], errors="raise").dt.year < forecast_year].copy()
    train = _attach_covariates(train, daily_covariates)
    daily_model = fit_daily_ridge_forecaster(train, "pressure_index", dynamic=selected_dynamic)
    regions = sorted(train["region_code"].astype(str).unique())
    future_dates = pd.date_range(f"{forecast_year}-01-01", f"{forecast_year}-12-31", freq="D")
    future = pd.DataFrame({"date": np.repeat(future_dates, len(regions)), "region_code": regions * len(future_dates)})
    future_covariates = build_climatological_covariates(daily_covariates, forecast_year) if daily_covariates is not None else
None
    future = _attach_covariates(future, future_covariates)
    daily = predict_daily_ridge_forecaster(daily_model, future)
    if regional_radius_map:
        radii = daily["region_code"].map(regional_radius_map).fillna(conformal_radius)
        lower = daily["prediction_q50"] * np.maximum(0.0, 1.0 - radii)
        upper = daily["prediction_q50"] * (1.0 + radii)
        daily["prediction_q10"] = np.minimum(daily["prediction_q10"], lower)
        daily["prediction_q90"] = np.maximum(daily["prediction_q90"], upper)
    elif np.isfinite(conformal_radius):
        lower = daily["prediction_q50"] * max(0.0, 1.0 - conformal_radius)
        upper = daily["prediction_q50"] * (1.0 + conformal_radius)
        daily["prediction_q10"] = np.minimum(daily["prediction_q10"], lower)
        daily["prediction_q90"] = np.maximum(daily["prediction_q90"], upper)
    daily["interval_calibration"] = interval_calibration
    daily["conformal_relative_radius"] = daily["region_code"].map(regional_radius_map).fillna(conformal_radius)
    calibration_table = pd.DataFrame(regional_calibration_records)
    if not calibration_table.empty:
        calibration_table["calibration_method"] = interval_calibration
    calibration_table.to_csv(output / f"q2_daily_conformal_calibration_{forecast_year}.csv", index=False, encoding="utf-8-sig")
    scales = train.assign(date=pd.to_datetime(train["date"], errors="raise")).sort_values("date").groupby("region_code",
as_index=False)["baseline_scale"].last()
    daily = daily.merge(scales, on="region_code", how="left")
    for quantile in ["q10", "q50", "q90"]:
        daily[f"visitor_estimate_{quantile}"] = daily[f"prediction_{quantile}"] * daily["baseline_scale"]
```

```
    daily["forecast_provenance"] = "pressure_forecast_scaled_by_latest_available_region_anchor"
    daily.to_csv(output / f"q2_region_daily_forecast_{forecast_year}.csv", index=False, encoding="utf-8-sig")

    summer = daily.loc[pd.to_datetime(daily["date"]).dt.month.isin([7, 8])].groupby("region_code", as_index=False).agg(
        summer_peak_q10=("visitor_estimate_q10", "max"), summer_peak_q50=("visitor_estimate_q50", "max"),
summer_peak_q90=("visitor_estimate_q90", "max"),
    )
    summer.to_csv(output / f"q2_summer_peak_range_{forecast_year}.csv", index=False, encoding="utf-8-sig")
    weather_model = daily_model
    weather_response_model = "selected_daily_model"
    if not {"rain_mm", "temperature_c"}.intersection(daily_model.feature_names) and daily_covariates is not None:
        weather_model = fit_daily_ridge_forecaster(train, "pressure_index", dynamic=True)
        weather_response_model = "dynamic_ridge_sensitivity_only"
    weather_base = predict_daily_ridge_forecaster(weather_model, future).merge(scales, on="region_code", how="left")
    rainy = scenario_adjust_weather(future, rain_add_mm=20.0)
    rain_daily = predict_daily_ridge_forecaster(weather_model, rainy).merge(scales, on="region_code", how="left")
    scenario = daily.loc[:, ["date", "region_code", "visitor_estimate_q50"]].rename(columns={"visitor_estimate_q50":
"baseline_estimate"})
    scenario["weather_response_baseline_estimate"] = weather_base["prediction_q50"].to_numpy() *
weather_base["baseline_scale"].to_numpy()
    scenario["rain_shock_estimate"] = rain_daily["prediction_q50"].to_numpy() * rain_daily["baseline_scale"].to_numpy()
    heat = scenario_adjust_weather(future, temperature_shift_c=5.0)
    heat_daily = predict_daily_ridge_forecaster(weather_model, heat).merge(scales, on="region_code", how="left")
    scenario["heat_shock_estimate"] = heat_daily["prediction_q50"].to_numpy() * heat_daily["baseline_scale"].to_numpy()
    scenario["scenario_label"] = "additive_20mm_rainfall_and_plus5c_weather_shocks"
    scenario.to_csv(output / f"q2_weather_shock_{forecast_year}.csv", index=False, encoding="utf-8-sig")
    rain_shock_effective_share = float(
        1.0 - np.isclose(
            scenario["weather_response_baseline_estimate"],
            scenario["rain_shock_estimate"],
        ).mean()
    )
    build_q2_diagnostic_figures(monthly, daily, output)
    report = {
        "forecast_year": forecast_year,
        "annual_forecast_point": annual_forecast["point"],
        "annual_forecast_interval": [annual_forecast["lower"], annual_forecast["upper"]],
        "annual_selected_model": annual_selected_model,
        "daily_selected_model": "dynamic_ridge" if selected_dynamic else "seasonal_calendar_ridge",
        "daily_interval_calibration": interval_calibration,
        "daily_conformal_relative_radius": None if not np.isfinite(conformal_radius) else conformal_radius,
        "daily_conformal_relative_radius_by_region": regional_radius_map,
        "daily_validation_scope": "raw_attraction_observation_dates_only" if not no_independent_observation else
"no_independent_observation_available",
        "daily_validation_covariate_mode": "conditional_on_realized_weather_and_lagged_search" if daily_covariates is not None
else "calendar_only",
        "daily_rolling_origin_windows": ["2024-01-01", "2024-04-01", "2024-07-01"],
        "daily_rolling_validation_rows": int(len(rolling_validation)),
        "weather_response_model": weather_response_model,
        "weather_shock_effective": bool(rain_shock_effective_share > 0.0),
        "rain_shock_effective_share": rain_shock_effective_share,
        "regional_daily_unit": "anchor_constrained_visitor_scale_estimate_not_observed_count",
    }
    (output / "q2_quality_report.json").write_text(json.dumps(report, ensure_ascii=False, indent=2), encoding="utf-8")
    return report
```

src/models/question2_forecasting.py **功能说明：**定义问题二日级预测所用的特征、模型与评价计算。

```
"""Forecasting primitives for Question 2 with explicit scale and provenance boundaries."""

from __future__ import annotations

```

```
from dataclasses import dataclass
from typing import Iterable

import numpy as np
import pandas as pd


@dataclass(frozen=True)
class AnnualLogTrend:
    """Weighted log-linear annual visitor model fitted to official totals only."""

    origin_year: int
    intercept: float
    slope: float
    residual_std: float
    x_mean: float
    weighted_sxx: float
    effective_n: float


@dataclass(frozen=True)
class DailyRidgeForecaster:
    """Regularized daily forecast model with serializable feature scaling."""

    feature_names: tuple[str, ...]
    feature_means: tuple[float, ...]
    feature_scales: tuple[float, ...]
    coefficients: tuple[float, ...]
    residual_std: float
    region_levels: tuple[str, ...]
    dynamic: bool
    prediction_floor: float


def annual_rolling_origin_splits(years: Iterable[int], min_train_years: int = 8) -> list[tuple[int, int, int]]:
    """Return chronological (first_train_year, last_train_year, target_year) splits."""
    unique_years = sorted({int(year) for year in years})
    if min_train_years < 2:
        raise ValueError("min_train_years must be at least 2")
    return [
        (unique_years[0], unique_years[target_index - 1], unique_years[target_index])
        for target_index in range(min_train_years, len(unique_years))
    ]


def fit_annual_log_trend(
    annual: pd.DataFrame,
    value_column: str,
    recency_half_life_years: float = 8.0,
) -> AnnualLogTrend:
    """Fit a recency-weighted trend to positive official annual totals."""
    if {"year", value_column}.difference(annual.columns):
        raise ValueError("annual frame must contain year and the requested value column")
    if recency_half_life_years <= 0:
        raise ValueError("recency_half_life_years must be positive")
    frame = annual.loc[:, ["year", value_column]].copy()
    frame["year"] = pd.to_numeric(frame["year"], errors="raise").astype(int)
    frame["value"] = pd.to_numeric(frame[value_column], errors="raise")
    frame = frame.loc[frame["value"] > 0].drop_duplicates("year", keep="last").sort_values("year")
    if len(frame) < 3:
        raise ValueError("at least three positive annual totals are required")
    origin_year = int(frame["year"].max())
    x = (frame["year"].to_numpy(dtype=float) - origin_year)
    y = np.log(frame["value"].to_numpy(dtype=float))
```

```
    weights = np.exp(np.log(0.5) * (origin_year - frame["year"].to_numpy(dtype=float)) / recency_half_life_years)
    x_mean = float(np.average(x, weights=weights))
    y_mean = float(np.average(y, weights=weights))
    centered_x = x - x_mean
    weighted_sxx = float(np.sum(weights * centered_x**2))
    if weighted_sxx <= 0:
        raise ValueError("annual years must not all be identical")
    slope = float(np.sum(weights * centered_x * (y - y_mean)) / weighted_sxx)
    intercept = float(y_mean - slope * x_mean)
    residual_std = float(np.sqrt(np.average((y - (intercept + slope * x)) ** 2, weights=weights)))
    effective_n = float(weights.sum() ** 2 / np.square(weights).sum())
    return AnnualLogTrend(origin_year, intercept, slope, residual_std, x_mean, weighted_sxx, effective_n)


def forecast_annual_total(model: AnnualLogTrend, target_year: int, interval_z: float = 1.645) -> dict[str, float]:
    """Forecast one official-scale annual total with an 90% log-scale interval by default."""
    if interval_z < 0:
        raise ValueError("interval_z must be non-negative")
    x_target = float(int(target_year) - model.origin_year)
    log_point = model.intercept + model.slope * x_target
    leverage = 1.0 + 1.0 / model.effective_n + (x_target - model.x_mean) ** 2 / model.weighted_sxx
    log_half_width = interval_z * model.residual_std * np.sqrt(leverage)
    return {
        "year": int(target_year),
        "point": float(np.exp(log_point)),
        "lower": float(np.exp(log_point - log_half_width)),
        "upper": float(np.exp(log_point + log_half_width)),
    }


def _annual_candidate_point(train: pd.DataFrame, value_column: str, target_year: int, candidate: str) -> float:
    ordered = train.loc[:, ["year", value_column]].copy().sort_values("year")
    values = pd.to_numeric(ordered[value_column], errors="raise").to_numpy(dtype=float)
    if candidate == "weighted_log_trend":
        return forecast_annual_total(fit_annual_log_trend(ordered, value_column), target_year)["point"]
    if candidate == "last_year_naive":
        return float(values[-1])
    if candidate == "recent_median_growth":
        growth = values[1:] / values[:-1]
        return float(values[-1] * np.median(growth[-min(3, len(growth)):]))
    raise ValueError(f"unknown annual candidate: {candidate}")


def evaluate_annual_candidates(annual: pd.DataFrame, value_column: str, min_train_years: int = 8) -> pd.DataFrame:
    """Score interpretable annual candidates by strictly forward rolling forecasts."""
    frame = annual.loc[:, ["year", value_column]].copy()
    frame["year"] = pd.to_numeric(frame["year"], errors="raise").astype(int)
    frame[value_column] = pd.to_numeric(frame[value_column], errors="raise")
    frame = frame.loc[frame[value_column] > 0].drop_duplicates("year", keep="last").sort_values("year")
    effective_min_train_years = min(min_train_years, len(frame) - 1)
    if effective_min_train_years < 2:
        raise ValueError("at least three positive annual totals are required for rolling validation")
    candidates = ("weighted_log_trend", "last_year_naive", "recent_median_growth")
    records: list[dict[str, object]] = []
    for candidate in candidates:
        actuals: list[float] = []; predictions: list[float] = []
        for _, train_end, target_year in annual_rolling_origin_splits(frame["year"], effective_min_train_years):
            train = frame.loc[frame["year"] <= train_end]
            actual = float(frame.loc[frame["year"].eq(target_year), value_column].iloc[0])
            predictions.append(_annual_candidate_point(train, value_column, target_year, candidate)); actuals.append(actual)
        residual = np.asarray(predictions) - np.asarray(actuals)
        scale = np.maximum((np.abs(np.asarray(predictions)) + np.abs(np.asarray(actuals))) / 2.0, 1e-9)
        records.append({"candidate": candidate, "test_rows": len(actuals), "mae": float(np.abs(residual).mean()), "rmse":
  float(np.sqrt(np.mean(residual**2))), "smape": float(100.0 * np.mean(np.abs(residual) / scale))})
```

```
    return pd.DataFrame(records)


def select_annual_candidate(scores: pd.DataFrame) -> str:
    """Choose lowest sMAPE; use model simplicity then MAE/RMSE as deterministic tie-breakers."""
    required = {"candidate", "mae", "rmse", "smape"}
    if required.difference(scores.columns) or scores.empty:
        raise ValueError("annual score table is empty or incomplete")
    complexity = {"last_year_naive": 0, "recent_median_growth": 1, "weighted_log_trend": 2}
    ordered = scores.copy(); ordered["complexity"] = ordered["candidate"].map(complexity).fillna(99)
    return str(ordered.sort_values(["smape", "complexity", "mae", "rmse"]).iloc[0]["candidate"])


def forecast_annual_candidate(
    annual: pd.DataFrame, value_column: str, target_year: int, candidate: str, min_train_years: int = 8, interval_z: float =
  1.645,
) -> dict[str, float | int | str]:
    """Forecast a selected annual candidate with interval width from its own rolling log residuals."""
    frame = annual.loc[:, ["year", value_column]].copy().sort_values("year")
    frame["year"] = pd.to_numeric(frame["year"], errors="raise").astype(int)
    frame[value_column] = pd.to_numeric(frame[value_column], errors="raise")
    frame = frame.loc[frame[value_column] > 0].drop_duplicates("year", keep="last")
    point = _annual_candidate_point(frame, value_column, target_year, candidate)
    log_residuals: list[float] = []
    for _, train_end, test_year in annual_rolling_origin_splits(frame["year"], min_train_years):
        train = frame.loc[frame["year"] <= train_end]
        actual = float(frame.loc[frame["year"].eq(test_year), value_column].iloc[0])
        predicted = _annual_candidate_point(train, value_column, test_year, candidate)
        log_residuals.append(float(np.log(actual) - np.log(max(predicted, 1e-9))))
    residual_std = float(np.std(log_residuals, ddof=0)) if log_residuals else 0.0
    half_width = interval_z * residual_std
    return {"year": int(target_year), "candidate": candidate, "point": float(point), "lower": float(point *
  np.exp(-half_width)), "upper": float(point * np.exp(half_width)), "rolling_log_residual_std": residual_std}


def allocate_monthly_total(annual_total: float, monthly_shape: Iterable[float], year: int) -> pd.DataFrame:
    """Allocate an annual estimate by non-negative monthly weights while preserving its total."""
    total = float(annual_total)
    weights = np.asarray(list(monthly_shape), dtype=float)
    if total < 0:
        raise ValueError("annual_total must be non-negative")
    if len(weights) != 12:
        raise ValueError("monthly_shape must contain exactly 12 weights")
    if not np.isfinite(weights).all() or (weights < 0).any() or weights.sum() <= 0:
        raise ValueError("monthly_shape must be finite, non-negative, and contain positive mass")
    allocation = total * weights / weights.sum()
    allocation[-1] = total - allocation[:-1].sum()
    return pd.DataFrame({
        "year": int(year),
        "month": np.arange(1, 13, dtype=int),
        "monthly_shape_weight": weights,
        "visitor_estimate": allocation,
        "estimate_label": "official_annual_total_constrained_monthly_visitor_estimate",
    })


def build_daily_calendar_features(frame: pd.DataFrame) -> pd.DataFrame:
    """Create calendar covariates that remain known for any future forecast date."""
    if "date" not in frame.columns:
        raise ValueError("daily frame missing date")
    result = frame.copy()
    date = pd.to_datetime(result["date"], errors="raise")
    day_of_year = date.dt.dayofyear.to_numpy(dtype=float)
    result["is_summer"] = date.dt.month.isin([7, 8]).astype(float)
```

```
    result["is_weekend"] = (date.dt.dayofweek >= 5).astype(float)
    for weekday in range(1, 7):
        result[f"weekday_{weekday}"] = (date.dt.dayofweek == weekday).astype(float)
    result["annual_sin"] = np.sin(2.0 * np.pi * day_of_year / 365.25)
    result["annual_cos"] = np.cos(2.0 * np.pi * day_of_year / 365.25)
    return result


def _daily_design(frame: pd.DataFrame, region_levels: tuple[str, ...], dynamic: bool) -> tuple[np.ndarray, tuple[str, ...]]:
    calendar = build_daily_calendar_features(frame)
    pieces = [np.ones((len(calendar), 1), dtype=float)]
    names = ["intercept"]
    for name in ["is_summer", "annual_sin", "annual_cos", *[f"weekday_{weekday}" for weekday in range(1, 7)]]:
        pieces.append(calendar[[name]].to_numpy(dtype=float))
        names.append(name)
    if dynamic:
        for name in ["rain_mm", "temperature_c", "search_lag_1"]:
            if name in calendar.columns:
                values = pd.to_numeric(calendar[name], errors="coerce").to_numpy(dtype=float)
                finite_median = float(np.nanmedian(values)) if np.isfinite(values).any() else 0.0
                pieces.append(np.nan_to_num(values, nan=finite_median).reshape(-1, 1))
                names.append(name)
    if "region_code" in calendar.columns:
        regions = calendar["region_code"].astype(str)
        for region in region_levels[1:]:
            pieces.append(regions.eq(region).to_numpy(dtype=float).reshape(-1, 1))
            names.append(f"region__{region}")
    return np.hstack(pieces), tuple(names)


def fit_daily_ridge_forecaster(
    frame: pd.DataFrame, target_column: str, dynamic: bool, ridge_alpha: float = 10.0,
) -> DailyRidgeForecaster:
    """Fit a log-scale daily model with forecast-available dynamic covariates only."""
    if target_column not in frame.columns:
        raise ValueError("daily frame missing target column")
    if ridge_alpha < 0:
        raise ValueError("ridge_alpha must be non-negative")
    target = pd.to_numeric(frame[target_column], errors="coerce")
    valid = target.notna() & target.ge(0)
    if valid.sum() < 3:
        raise ValueError("at least three non-negative daily targets are required")
    regions = tuple(sorted(frame.get("region_code", pd.Series(["CITY"] * len(frame))).astype(str).unique()))
    design, names = _daily_design(frame, regions, dynamic)
    x = design[valid.to_numpy()]
    means = x.mean(axis=0); scales = x.std(axis=0)
    means[0] = 0.0; scales[0] = 1.0
    scales[scales == 0] = 1.0
    standardized = (x - means) / scales
    standardized[:, 0] = 1.0
    y = np.log1p(target.loc[valid].to_numpy(dtype=float))
    penalty = np.eye(standardized.shape[1]) * ridge_alpha
    penalty[0, 0] = 0.0
    coefficients = np.linalg.pinv(standardized.T @ standardized + penalty) @ standardized.T @ y
    residual_std = float(np.std(y - standardized @ coefficients, ddof=0))
    positive_target = target.loc[valid & target.gt(0)].to_numpy(dtype=float)
    floor = float(np.quantile(positive_target, 0.05)) if len(positive_target) else 0.0
    return DailyRidgeForecaster(names, tuple(means), tuple(scales), tuple(coefficients), residual_std, regions, dynamic, floor)


def predict_daily_ridge_forecaster(model: DailyRidgeForecaster, frame: pd.DataFrame, interval_z: float = 1.645) ->
  pd.DataFrame:
    """Return direct multi-step daily forecasts using only supplied scenario covariates."""
    if interval_z < 0:
```

```
        raise ValueError("interval_z must be non-negative")
    design, names = _daily_design(frame, model.region_levels, model.dynamic)
    if names != model.feature_names:
        raise ValueError("prediction feature columns differ from fitted feature columns")
    means = np.asarray(model.feature_means); scales = np.asarray(model.feature_scales)
    standardized = (design - means) / scales
    standardized[:, 0] = 1.0
    log_point = standardized @ np.asarray(model.coefficients)
    half_width = interval_z * model.residual_std
    result = frame.loc[:, [name for name in ["date", "region_code"] if name in frame.columns]].copy()
    result["prediction_q10"] = np.maximum(np.expm1(log_point - half_width), 0.0)
    result["prediction_q50"] = np.maximum(np.expm1(log_point), model.prediction_floor)
    result["prediction_q90"] = np.maximum(np.expm1(log_point + half_width), result["prediction_q50"])
    result["estimate_label"] = "anchor_constrained_daily_visitor_forecast"
    result["forecast_method"] = "dynamic_ridge" if model.dynamic else "seasonal_calendar_ridge"
    return result


def scenario_adjust_weather(frame: pd.DataFrame, rain_multiplier: float = 1.0, rain_add_mm: float = 0.0, temperature_shift_c:
  float = 0.0) -> pd.DataFrame:
    """Create a weather-only scenario without changing date, search, or regional inputs."""
    if rain_multiplier < 0 or rain_add_mm < 0:
        raise ValueError("rain_multiplier and rain_add_mm must be non-negative")
    result = frame.copy()
    if "rain_mm" in result.columns:
        result["rain_mm"] = pd.to_numeric(result["rain_mm"], errors="coerce") * rain_multiplier + rain_add_mm
    if "temperature_c" in result.columns:
        result["temperature_c"] = pd.to_numeric(result["temperature_c"], errors="coerce") + temperature_shift_c
    return result
```

2.4 问题三：资源配置

scripts/build_q3_absolute_resource_plan.py **功能说明：**在游客规模情景下求解停车、接驳和排班配置。

```
"""Build absolute, scenario-based Question 3 resource planning outputs."""

from pathlib import Path
import sys

import pandas as pd

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))

from src.analysis.q3_absolute_resource_outputs import build_all_scenario_plans
from src.analysis.vot_outputs import attach_vot_costs
from src.models.question3_absolute_optimizer import optimize_absolute_resources


def season_from_month(month: int) -> str:
    if month in (7, 8):
        return "peak_summer"
    if month in (4, 5, 6, 9, 10):
        return "shoulder"
    return "low_season"


forecast = pd.read_csv(ROOT / "outputs/question2_analysis/q2_region_daily_forecast_2026.csv", encoding="utf-8-sig")
scenarios = pd.read_csv(ROOT / "data/model_input/q3_absolute_planning_scenarios.csv", encoding="utf-8-sig")
vot_scenarios = pd.read_csv(ROOT / "data/model_input/q3_vot_scenarios.csv", encoding="utf-8-sig")
```

```
daily = build_all_scenario_plans(forecast, scenarios)
daily = daily.merge(scenarios.loc[:, ["region_code", "demand_scenario", "parameter_basis", "parameter_note", "average_party_size", "transfer_share"]], on=["region_code", "demand_scenario"], how="left", validate="many_to_one")
daily["season"] = pd.to_datetime(daily["date"]).dt.month.map(season_from_month)
output = ROOT / "outputs/question3_analysis"
output.mkdir(parents=True, exist_ok=True)
daily.to_csv(output / "q3_absolute_daily_resource_plan_2026.csv", index=False, encoding="utf-8-sig")

def optimize_frame(frame: pd.DataFrame, penalty: float) -> pd.DataFrame:
    selected = frame.apply(
        lambda row: optimize_absolute_resources(
            int(row["peak_spaces_required"]),
            int(row["permanent_capacity_available"]),
            int(row["temporary_capacity_available"]),
            int(row["recommended_shuttle_vehicles"]),
            int(row["recommended_total_staff_shifts"]),
            penalty,
        )[0],
        axis=1,
    ).apply(pd.Series)
    return pd.concat([frame.reset_index(drop=True), selected], axis=1)

optimized_daily = optimize_frame(daily, 4.0)
optimized_daily["risk_penalty"] = 4.0
optimized_daily["optimization_status"] = "relative_cost_and_standardized_experience_loss__not_currency_or_survey_score"
optimized_daily = attach_vot_costs(optimized_daily, vot_scenarios)
optimized_daily.to_csv(output / "q3_absolute_daily_optimized_plan_2026.csv", index=False, encoding="utf-8-sig")

pareto_rows = []
for penalty in [1.0, 2.0, 4.0, 6.0, 8.0]:
    candidate = optimize_frame(daily, penalty)
    pareto_rows.append({"risk_penalty": penalty, "mean_relative_operating_cost": candidate["relative_operating_cost"].mean(), "mean_standardized_experience_loss": candidate["standardized_experience_loss"].mean()})
pareto = pd.DataFrame(pareto_rows).sort_values("mean_relative_operating_cost")
pareto["is_nondominated"] = ~pareto.apply(lambda row: ((pareto["mean_relative_operating_cost"] <= row["mean_relative_operating_cost"]) & (pareto["mean_standardized_experience_loss"] <= row["mean_standardized_experience_loss"]) & ((pareto["mean_relative_operating_cost"] < row["mean_relative_operating_cost"]) | (pareto["mean_standardized_experience_loss"] < row["mean_standardized_experience_loss"]))).any(), axis=1)
pareto.to_csv(output / "q3_absolute_cost_experience_pareto_2026.csv", index=False, encoding="utf-8-sig")

seasonal = daily.groupby(["region_code", "demand_scenario", "forecast_quantile", "season", "parking_capacity_basis"], as_index=False).agg(
    representative_daily_visitors=("visitor_estimate_q50", "median"),
    peak_daily_visitors=("visitor_estimate_q50", "max"),
    max_peak_spaces_required=("peak_spaces_required", "max"),
    max_temporary_spaces_open=("temporary_spaces_open", "max"),
    max_unserved_spaces=("unserved_spaces", "max"),
    max_shuttle_vehicles=("recommended_shuttle_vehicles", "max"),
    max_total_staff_shifts=("recommended_total_staff_shifts", "max"),
    parking_overflow_days=("capacity_status", lambda values: (values == "parking_overflow_unserved").sum()),
)
seasonal["result_status"] = "scenario_planning_value_not_operating_ledger"
seasonal.to_csv(output / "q3_absolute_seasonal_resource_plan_2026.csv", index=False, encoding="utf-8-sig")

bdh_summer = daily.loc[(daily["region_code"] == "BDH") & (pd.to_datetime(daily["date"]).dt.month.isin([7, 8]))].copy()
visitor_trigger = bdh_summer.groupby("demand_scenario")["visitor_estimate_q50"].transform(lambda series: series.quantile(0.75))
bdh_summer["temporary_plan_trigger"] = bdh_summer["visitor_estimate_q50"] >= visitor_trigger
bdh_summer.loc[bdh_summer["capacity_status"] != "permanent_capacity_sufficient", "temporary_plan_trigger"] = True
bdh_summer.loc[bdh_summer["temporary_plan_trigger"], "recommended_action"] = "add_peak_shift_staff__dispatch_planned_shuttle_fleet__issue_parking_guidance"
bdh_summer.loc[~bdh_summer["temporary_plan_trigger"], "recommended_action"] = "maintain_seasonal_baseline"
bdh_summer.to_csv(output / "q3_beidaihe_absolute_summer_temporary_plan_2026.csv", index=False, encoding="utf-8-sig")

sensitivity = daily.groupby(["region_code", "demand_scenario", "forecast_quantile"], as_index=False).agg(
```

```
    annual_max_peak_spaces_required=("peak_spaces_required", "max"),
    annual_max_shuttle_vehicles=("recommended_shuttle_vehicles", "max"),
    annual_max_staff_shifts=("recommended_total_staff_shifts", "max"),
    annual_parking_overflow_days=("capacity_status", lambda values: (values == "parking_overflow_unserved").sum()),
)
sensitivity["interpretation"] = "low_central_stress_parameter_and_forecast_sensitivity"
sensitivity.to_csv(output / "q3_absolute_resource_sensitivity_2026.csv", index=False, encoding="utf-8-sig")

# Capacity-relaxation diagnostic: transparent counterfactual, not an asserted facility inventory.
bdh_stress = daily.loc[(daily["region_code"] == "BDH") & (daily["demand_scenario"] == "stress")].copy()
peak = bdh_stress.loc[bdh_stress["unserved_spaces"].idxmax()]
slack_rows = []
for added_capacity in (0, 500, 1000):
    chosen, _ = optimize_absolute_resources(int(peak["peak_spaces_required"]), int(peak["permanent_capacity_available"]),
  added_capacity, int(peak["recommended_shuttle_vehicles"]), int(peak["recommended_total_staff_shifts"]), 4.0)
    physical_shortage = max(int(peak["peak_spaces_required"]) - int(peak["permanent_capacity_available"]) - added_capacity, 0)
    slack_rows.append({"region_code": "BDH", "demand_scenario": "stress", "reference_date": peak["date"],
  "added_temporary_capacity_scenario": added_capacity, "peak_spaces_required": int(peak["peak_spaces_required"]),
  "remaining_unserved_spaces_after_full_activation": physical_shortage, "shortage_reduction_vs_no_added_capacity":
  int(peak["unserved_spaces"]) - physical_shortage, "unconstrained_optimizer_objective": chosen["objective"],
  "interpretation": "physical_capacity_relaxation_counterfactual_not_verified_supply_or_selected_plan"})
pd.DataFrame(slack_rows).to_csv(output / "q3_bdh_temporary_capacity_relaxation.csv", index=False, encoding="utf-8-sig")
```

src/analysis/q3_absolute_resource_outputs.py **功能说明：**把绝对游客规模转换为逐日停车、接驳和人员需求。

```
"""Create transparent absolute resource planning values for Question 3."""

from __future__ import annotations

import math

import pandas as pd

from src.models.question3_absolute_capacity import minimum_staff, parking_plan, peak_parking_demand, required_shuttle_vehicles
from src.models.question3_staff_scheduling import optimize_three_shifts


BLOCK_SHARES = (0.25, 0.50, 0.25)
BLOCK_HOURS = 4.0


def _staff_schedule(daily_visitors: float, block_shares: tuple[float, float, float], service_rate: float, utilisation: float,
  minimum_people: int, multiplier: float = 1.0) -> dict[str, int]:
    demand = [
        minimum_staff(daily_visitors * share * multiplier / BLOCK_HOURS, service_rate, utilisation, minimum_people)
        for share in block_shares
    ]
    return optimize_three_shifts(demand)


def _shuttle_schedule(daily_visitors: float, row: pd.Series) -> tuple[int, dict[str, int]]:
    fleets = [
        required_shuttle_vehicles(
            daily_visitors * share * float(row["transfer_share"]) / BLOCK_HOURS,
            int(row["seats_per_vehicle"]),
            float(row["load_factor"]),
            float(row["round_trip_minutes"]),
            float(row["maximum_headway_minutes"]),
        )
        for share in BLOCK_SHARES
    ]
    return max(fleets), optimize_three_shifts(fleets)
```

```python


def build_daily_absolute_plan(forecast: pd.DataFrame, scenarios: pd.DataFrame) -> pd.DataFrame:
    """Build daily planning recommendations from q50 visitors and scenario inputs.

    Inputs are deliberately explicit because car mode share, vehicle type, service
    productivity and unobserved capacity are planning assumptions rather than
    observed operational records.
    """
    required_forecast = {"date", "region_code", "visitor_estimate_q50"}
    required_scenario = {"region_code", "demand_scenario", "car_share", "average_party_size", "peak_arrival_share", "dwell_hours", "peak_window_hours", "transfer_share", "seats_per_vehicle", "load_factor", "round_trip_minutes", "maximum_headway_minutes", "entry_service_rate", "parking_service_rate", "utilisation_target", "permanent_spaces", "temporary_spaces"}
    if missing := required_forecast - set(forecast.columns):
        raise ValueError(f"forecast missing columns: {sorted(missing)}")
    if missing := required_scenario - set(scenarios.columns):
        raise ValueError(f"scenarios missing columns: {sorted(missing)}")
    merged = forecast.loc[:, ["date", "region_code", "visitor_estimate_q50"]].merge(scenarios, on="region_code", how="inner", validate="many_to_one")
    if len(merged) != len(forecast):
        raise ValueError("each forecast region requires exactly one scenario row")
    records: list[dict[str, object]] = []
    for _, row in merged.iterrows():
        visitors = float(row["visitor_estimate_q50"])
        peak_spaces = peak_parking_demand(visitors, float(row["car_share"]), float(row["average_party_size"]), float(row["peak_arrival_share"]), float(row["dwell_hours"]), float(row["peak_window_hours"]))
        park = parking_plan(peak_spaces, int(row["permanent_spaces"]), int(row["temporary_spaces"]))
        shuttle_vehicles, driver_schedule = _shuttle_schedule(visitors, row)
        entry_schedule = _staff_schedule(visitors, BLOCK_SHARES, float(row["entry_service_rate"]), float(row["utilisation_target"]), 1, multiplier=1 / float(row["average_party_size"]))
        parking_schedule = _staff_schedule(visitors, BLOCK_SHARES, float(row["parking_service_rate"]), float(row["utilisation_target"]), 1, multiplier=float(row["car_share"]) / float(row["average_party_size"]))
        records.append({
            "date": row["date"], "region_code": row["region_code"], "visitor_estimate_q50": visitors,
            "demand_scenario": row["demand_scenario"], "data_basis": "derived_planning_value",
            "parking_capacity_basis": row.get("parking_capacity_basis", "scenario_parameter"),
            "permanent_capacity_available": int(row["permanent_spaces"]),
            "temporary_capacity_available": int(row["temporary_spaces"]),
            **park,
            "parking_transfer_vehicle_demand": int(park["unserved_spaces"]),
            "parking_transfer_visitor_demand": int(math.ceil(park["unserved_spaces"] * float(row["average_party_size"]))),
            "parking_management_action": "outer_park_and_ride_or_timed_reservation" if park["unserved_spaces"] > 0 else "on_site_capacity_sufficient",
            "maximum_headway_minutes": int(row["maximum_headway_minutes"]),
            "operational_anchor_id": row.get("operational_anchor_id", "no_direct_operational_anchor"),
            "headway_evidence_scope": row.get("headway_evidence_scope", "scenario_parameter_without_direct_operational_anchor"),
            "headway_parameter_role": "service_level_planning_constraint__not_actual_fleet_or_trip_ledger",
            "recommended_shuttle_vehicles": shuttle_vehicles,
            "entry_early_shift": entry_schedule["early_shift"], "entry_middle_shift": entry_schedule["middle_shift"], "entry_late_shift": entry_schedule["late_shift"], "entry_staff_shifts": entry_schedule["staff_shifts"],
            "parking_early_shift": parking_schedule["early_shift"], "parking_middle_shift": parking_schedule["middle_shift"], "parking_late_shift": parking_schedule["late_shift"], "parking_staff_shifts": parking_schedule["staff_shifts"],
            "driver_early_shift": driver_schedule["early_shift"], "driver_middle_shift": driver_schedule["middle_shift"], "driver_late_shift": driver_schedule["late_shift"], "driver_staff_shifts": driver_schedule["staff_shifts"],
            "recommended_total_staff_shifts": entry_schedule["staff_shifts"] + parking_schedule["staff_shifts"] + driver_schedule["staff_shifts"],
            "output_status": "recommended_planning_configuration_not_operating_ledger",
        })
    return pd.DataFrame(records)


def build_all_scenario_plans(forecast: pd.DataFrame, scenarios: pd.DataFrame) -> pd.DataFrame:
```

```
    """Apply one scenario row to its matching regional forecast only."""
    if "forecast_quantile" not in scenarios.columns:
        raise ValueError("scenarios must specify forecast_quantile")
    plans: list[pd.DataFrame] = []
    for _, scenario in scenarios.iterrows():
        quantile = scenario["forecast_quantile"]
        if quantile not in forecast.columns:
            raise ValueError(f"forecast is missing quantile column: {quantile}")
        regional_forecast = forecast.loc[forecast["region_code"].eq(scenario["region_code"]), ["date", "region_code",
  quantile]].rename(columns={quantile: "visitor_estimate_q50"})
        if regional_forecast.empty:
            raise ValueError(f"forecast has no records for {scenario['region_code']}")
        plan = build_daily_absolute_plan(regional_forecast, pd.DataFrame([scenario]))
        plan["forecast_quantile"] = quantile
        plans.append(plan)
    return pd.concat(plans, ignore_index=True)
```

src/models/question3_absolute_capacity.py **功能说明：**给出停车泊位、接驳车辆和人员数量的承载力计算。

```
"""Transparent capacity primitives for Question 3 planning recommendations.

The functions return *recommended planning quantities*.  They must not be
interpreted as a reconstruction of a scenic area's actual operating ledger.
"""

from __future__ import annotations

import math


def _positive(value: float, name: str) -> None:
    if value <= 0:
        raise ValueError(f"{name} must be positive")


def peak_parking_demand(
    daily_visitors: float,
    car_share: float,
    average_party_size: float,
    peak_arrival_share: float,
    dwell_hours: float,
    peak_window_hours: float,
) -> int:
    """Estimate concurrent parking spaces in the peak window.

    ``daily_visitors`` is persons/day; car share is the fraction arriving by
    private car; party size is persons/vehicle.  The result is vehicles/spaces.
    """
    if daily_visitors < 0:
        raise ValueError("daily_visitors must be non-negative")
    if not 0 <= car_share <= 1:
        raise ValueError("car_share must be in [0, 1]")
    if not 0 <= peak_arrival_share <= 1:
        raise ValueError("peak_arrival_share must be in [0, 1]")
    _positive(average_party_size, "average_party_size")
    _positive(dwell_hours, "dwell_hours")
    _positive(peak_window_hours, "peak_window_hours")
    concurrent = daily_visitors * car_share / average_party_size * peak_arrival_share * dwell_hours / peak_window_hours
    return math.ceil(concurrent)


def parking_plan(peak_spaces: int, permanent_spaces: int, temporary_spaces: int = 0) -> dict[str, int | str]:
```

```
    """Allocate a peak parking requirement to permanent and temporary spaces."""
    for value, name in ((peak_spaces, "peak_spaces"), (permanent_spaces, "permanent_spaces"), (temporary_spaces,
  "temporary_spaces")):
        if value < 0:
            raise ValueError(f"{name} must be non-negative")
    permanent_open = min(peak_spaces, permanent_spaces)
    remaining = peak_spaces - permanent_open
    temporary_open = min(remaining, temporary_spaces)
    unserved = remaining - temporary_open
    if unserved:
        status = "parking_overflow_unserved"
    elif temporary_open:
        status = "temporary_capacity_required"
    else:
        status = "permanent_capacity_sufficient"
    return {
        "peak_spaces_required": peak_spaces,
        "permanent_spaces_open": permanent_open,
        "temporary_spaces_open": temporary_open,
        "unserved_spaces": unserved,
        "capacity_status": status,
    }


def required_shuttle_vehicles(
    passenger_demand_per_hour: float,
    seats_per_vehicle: int,
    load_factor: float,
    round_trip_minutes: float,
    maximum_headway_minutes: float,
) -> int:
    """Return the simultaneous fleet needed for demand and headway targets."""
    if passenger_demand_per_hour < 0:
        raise ValueError("passenger_demand_per_hour must be non-negative")
    _positive(seats_per_vehicle, "seats_per_vehicle")
    if not 0 < load_factor <= 1:
        raise ValueError("load_factor must be in (0, 1]")
    _positive(round_trip_minutes, "round_trip_minutes")
    _positive(maximum_headway_minutes, "maximum_headway_minutes")
    demand_fleet = math.ceil(passenger_demand_per_hour * round_trip_minutes / (60 * seats_per_vehicle * load_factor))
    headway_fleet = math.ceil(round_trip_minutes / maximum_headway_minutes)
    return max(demand_fleet, headway_fleet)


def minimum_staff(
    arrival_rate_per_hour: float,
    service_rate_per_staff_hour: float,
    utilisation_target: float,
    minimum_people: int = 1,
) -> int:
    """Return staff needed to keep workload at or below the utilisation target."""
    if arrival_rate_per_hour < 0:
        raise ValueError("arrival_rate_per_hour must be non-negative")
    _positive(service_rate_per_staff_hour, "service_rate_per_staff_hour")
    if not 0 < utilisation_target <= 1:
        raise ValueError("utilisation_target must be in (0, 1]")
    if minimum_people < 0:
        raise ValueError("minimum_people must be non-negative")
    required = math.ceil(arrival_rate_per_hour / (service_rate_per_staff_hour * utilisation_target))
    return max(required, minimum_people)
```

src/models/question3_absolute_optimizer.py **功能说明：**搜索满足约束的绝对资源配置，并权衡成本和服务损失。

```python
"""Cost--experience optimisation over absolute Q3 planning quantities."""

from __future__ import annotations

from itertools import product
import math

import pandas as pd


def optimize_absolute_resources(
    parking_required: int,
    permanent_spaces: int,
    temporary_capacity: int,
    shuttle_required: int,
    staff_required: int,
    risk_penalty: float = 4.0,
) -> tuple[dict[str, float | int], pd.DataFrame]:
    """Enumerate six coverage levels for temporary parking, shuttles and staff.

    Costs are relative scenario units: temporary-space activation, vehicle
    deployment, and staff-shift deployment each equal one at full coverage.
    Experience loss is the weighted fraction of unmet parking, shuttle and
    staffing need.  It is not a measured satisfaction score.
    """
    values = (parking_required, permanent_spaces, temporary_capacity, shuttle_required, staff_required)
    if any(int(value) != value or value < 0 for value in values) or risk_penalty < 0:
        raise ValueError("resource requirements and capacities must be non-negative integers; risk_penalty non-negative")
    levels = (0.0, 0.2, 0.4, 0.6, 0.8, 1.0)
    records = []
    for parking_level, shuttle_level, staff_level in product(levels, repeat=3):
        temporary = math.ceil(temporary_capacity * parking_level)
        shuttles = math.ceil(shuttle_required * shuttle_level)
        staff = math.ceil(staff_required * staff_level)
        parking_unserved = max(parking_required - permanent_spaces - temporary, 0)
        shuttle_shortfall = max(shuttle_required - shuttles, 0)
        staff_shortfall = max(staff_required - staff, 0)
        parking_loss = parking_unserved / max(parking_required, 1)
        shuttle_loss = shuttle_shortfall / max(shuttle_required, 1)
        staff_loss = staff_shortfall / max(staff_required, 1)
        experience_loss = 0.9 * parking_loss + 1.2 * shuttle_loss + staff_loss
        relative_cost = (
            0.6 * temporary / max(temporary_capacity, 1)
            + 1.2 * shuttles / max(shuttle_required, 1)
            + staff / max(staff_required, 1)
        )
        records.append({
            "selected_temporary_spaces": temporary,
            "selected_shuttle_vehicles": shuttles,
            "selected_staff_shifts": staff,
            "parking_unserved_spaces": parking_unserved,
            "shuttle_shortfall": shuttle_shortfall,
            "staff_shortfall": staff_shortfall,
            "standardized_experience_loss": experience_loss,
            "relative_operating_cost": relative_cost,
            "objective": relative_cost + risk_penalty * experience_loss,
        })
    diagnostics = pd.DataFrame(records).sort_values(["objective", "relative_operating_cost",
  "standardized_experience_loss"]).reset_index(drop=True)
    return diagnostics.iloc[0].to_dict(), diagnostics
```

src/models/question3_staff_scheduling.py **功能说明：**求解三个重叠班次下的最小

可行排班。

```
"""Small integer scheduler for three overlapping operational shifts."""

from __future__ import annotations


def optimize_three_shifts(required_people_by_block: list[int]) -> dict[str, int]:
    """Minimise staff-shifts while covering early/mid/late demand.

    The early shift covers blocks 1–2, the middle shift blocks 1–3 and the
    late shift blocks 2–3.  These are staff-shift assignments, not a record of
    currently employed people.
    """
    if len(required_people_by_block) != 3:
        raise ValueError("required_people_by_block must contain early, middle and late demand")
    if any(int(value) != value or value < 0 for value in required_people_by_block):
        raise ValueError("required_people_by_block must be non-negative integers")
    early_required, middle_required, late_required = required_people_by_block
    upper = max(required_people_by_block)
    candidates: list[dict[str, int]] = []
    for middle_shift in range(upper + 1):
        early_shift = max(early_required - middle_shift, 0)
        late_shift = max(late_required - middle_shift, 0)
        total = early_shift + middle_shift + late_shift
        if total < middle_required:
            late_shift += middle_required - total
            total = middle_required
        coverage = [early_shift + middle_shift, total, middle_shift + late_shift]
        candidates.append({
            "early_shift": early_shift,
            "middle_shift": middle_shift,
            "late_shift": late_shift,
            "staff_shifts": total,
            "overstaffing_person_blocks": sum(provided - required for provided, required in zip(coverage,
  required_people_by_block)),
        })
    return min(candidates, key=lambda row: (row["staff_shifts"], row["overstaffing_person_blocks"], row["middle_shift"],
  row["early_shift"]))
```

src/models/value_of_time.py **功能说明：**定义游客时间价值和时间损失的透明计算公式。

```
"""Transparent visitor time-loss valuation for Q3/Q4 planning scenarios."""

from __future__ import annotations


def _non_negative(value: float, name: str) -> float:
    numeric = float(value)
    if numeric < 0:
        raise ValueError(f"{name} must be non-negative")
    return numeric


def _share(value: float, name: str) -> float:
    numeric = float(value)
    if not 0 <= numeric <= 1:
        raise ValueError(f"{name} must be in [0, 1]")
    return numeric


def estimate_traveler_time_loss(
```

```
    *,
    daily_visitors: float,
    parking_unserved_spaces: float,
    average_party_size: float,
    parking_delay_hours: float,
    transfer_share: float,
    shuttle_shortfall_fraction: float,
    shuttle_delay_hours: float,
    staff_shortfall_fraction: float,
    staff_delay_hours: float,
    vot_cny_per_hour: float,
) -> dict[str, float]:
    """Estimate visitor time loss and its VOT-valued currency equivalent.

    The result measures scenario traveller inconvenience only.  It must not be
    interpreted as an operator cash ledger, compensation amount, or survey-based
    willingness-to-pay estimate.
    """
    visitors = _non_negative(daily_visitors, "daily_visitors")
    unserved_spaces = _non_negative(parking_unserved_spaces, "parking_unserved_spaces")
    party_size = float(average_party_size)
    if party_size <= 0:
        raise ValueError("average_party_size must be positive")
    parking_delay = _non_negative(parking_delay_hours, "parking_delay_hours")
    transfer = _share(transfer_share, "transfer_share")
    shuttle_shortfall = _share(shuttle_shortfall_fraction, "shuttle_shortfall_fraction")
    shuttle_delay = _non_negative(shuttle_delay_hours, "shuttle_delay_hours")
    staff_shortfall = _share(staff_shortfall_fraction, "staff_shortfall_fraction")
    staff_delay = _non_negative(staff_delay_hours, "staff_delay_hours")
    vot = _non_negative(vot_cny_per_hour, "vot_cny_per_hour")

    parking_hours = unserved_spaces * party_size * parking_delay
    shuttle_hours = visitors * transfer * shuttle_shortfall * shuttle_delay
    staff_hours = visitors * staff_shortfall * staff_delay
    total_hours = parking_hours + shuttle_hours + staff_hours
    return {
        "parking_time_loss_hours": parking_hours,
        "shuttle_time_loss_hours": shuttle_hours,
        "staff_time_loss_hours": staff_hours,
        "traveler_time_loss_hours": total_hours,
        "traveler_time_loss_cny": total_hours * vot,
    }
```

2.5 问题四：情景重优化

scripts/build_q4_scenario_analysis.py **功能说明：**模拟雨天和事件冲击并比较重优化方案。

```
"""Build Question 4 event/rain counterfactual and robustness outputs."""

from pathlib import Path
import sys
import pandas as pd
import matplotlib.pyplot as plt

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))
from src.analysis.q4_scenario_outputs import compare_baseline_with_reoptimisation, one_at_a_time_sensitivity
from src.models.question4_scenarios import apply_continuous_rain, apply_event_pulse

out = ROOT / "outputs/question4_analysis"; out.mkdir(parents=True, exist_ok=True)
```

```
plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "DejaVu Sans"]
plt.rcParams["axes.unicode_minus"] = False
base = pd.read_csv(ROOT / "outputs/question3_analysis/q3_absolute_daily_resource_plan_2026.csv", encoding="utf-8-sig")
params = pd.read_csv(ROOT / "data/model_input/q3_absolute_planning_scenarios.csv", encoding="utf-8-sig")
vot = pd.read_csv(ROOT / "data/model_input/q3_vot_scenarios.csv", encoding="utf-8-sig")

def scenario_dict(region: str) -> dict:
    return params[(params.region_code == region) & (params.demand_scenario == "central")].iloc[0].to_dict()


def central_vot() -> dict:
    return vot[vot.vot_scenario.eq("central")].iloc[0].to_dict()

bdh = base[(base.region_code == "BDH") & (base.demand_scenario == "central")].copy()
event = apply_event_pulse(bdh, "visitor_estimate_q50", "2026-08-08", 0.40)
event_result = compare_baseline_with_reoptimisation(bdh, event, scenario_dict("BDH"), vot_parameters=central_vot())
event_result["scenario_name"] = "beidaihe_summer_cultural_event"
event_result.to_csv(out / "q4_event_counterfactual_2026.csv", index=False, encoding="utf-8-sig")

shg = base[(base.region_code == "SHG") & (base.demand_scenario == "central")].copy()
rain = apply_continuous_rain(shg, "visitor_estimate_q50", "2026-07-15", 5, 0.12)
rain_result = compare_baseline_with_reoptimisation(shg, rain, scenario_dict("SHG"), vot_parameters=central_vot())
rain_result["scenario_name"] = "shanhaiguan_five_day_continuous_rain"
rain_result.to_csv(out / "q4_rain_counterfactual_2026.csv", index=False, encoding="utf-8-sig")

ranking = one_at_a_time_sensitivity(bdh, scenario_dict("BDH"), "2026-08-08", {"event_intensity": (0.20, 0.60), "car_share":
  (0.40, 0.70), "dwell_hours": (2.0, 4.0), "transfer_share": (0.05, 0.15)})
rain_gaps = []
for attenuation in (0.06, 0.18):
    probe = apply_continuous_rain(shg, "visitor_estimate_q50", "2026-07-15", 5, attenuation)
    comparison = compare_baseline_with_reoptimisation(shg, probe, scenario_dict("SHG"))
    rain_gaps.append(float(comparison[comparison.date.between("2026-07-15", "2026-07-19")].baseline_service_gap.max()))
ranking = pd.concat([ranking, pd.DataFrame([{"parameter": "rain_daily_attenuation", "low_value": 0.06, "high_value": 0.18,
  "low_max_service_gap": rain_gaps[0], "high_max_service_gap": rain_gaps[1], "sensitivity_effect": abs(rain_gaps[1] -
  rain_gaps[0])}])], ignore_index=True)
ranking["ranking"] = ranking.sensitivity_effect.rank(method="dense", ascending=False).astype(int)
ranking = ranking.sort_values(["ranking", "parameter"])
lambda_rows = []
for penalty in (0.25, 0.5, 0.75, 1.0, 1.25, 1.5, 2.0):
    probe = compare_baseline_with_reoptimisation(bdh, event, scenario_dict("BDH"), risk_penalty=penalty)
    window = probe[probe.date.between("2026-08-06", "2026-08-10")]
    lambda_rows.append({"risk_penalty": penalty, "mean_relative_cost": window.reoptimized_relative_cost.mean(),
  "mean_experience_loss": window.reoptimized_experience_loss.mean(), "max_temporary_spaces":
  window.reoptimized_temporary_spaces.max(), "max_shuttle_vehicles": window.reoptimized_shuttle_vehicles.max(),
  "max_staff_shifts": window.reoptimized_staff_shifts.max()})
lambda_table = pd.DataFrame(lambda_rows)
lambda_table.to_csv(out / "q4_lambda_tradeoff_2026.csv", index=False, encoding="utf-8-sig")
ranking.to_csv(out / "q4_robustness_sensitivity_ranking.csv", index=False, encoding="utf-8-sig")

summary = pd.concat([event_result.assign(scenario="event"), rain_result.assign(scenario="rain")]).groupby("scenario",
  as_index=False).agg(max_baseline_gap=("baseline_service_gap", "max"), mean_reoptimized_cost=("reoptimized_relative_cost",
  "mean"), mean_reoptimized_loss=("reoptimized_experience_loss", "mean"),
  max_reoptimized_shuttles=("reoptimized_shuttle_vehicles", "max"), max_reoptimized_staff=("reoptimized_staff_shifts", "max"))
summary.to_csv(out / "q4_baseline_reoptimization_summary_2026.csv", index=False, encoding="utf-8-sig")

def window_summary(frame, start, end, name):
    part = frame[frame.date.between(start, end)]
    return {"scenario": name, "window_start": start, "window_end": end, "days": len(part), "max_baseline_gap":
  part.baseline_service_gap.max(), "max_baseline_shuttles": part.baseline_shuttle_vehicles.max(), "max_reoptimized_shuttles":
  part.reoptimized_shuttle_vehicles.max(), "max_baseline_staff": part.baseline_staff_shifts.max(), "max_reoptimized_staff":
  part.reoptimized_staff_shifts.max(), "max_reoptimized_temporary_spaces": part.reoptimized_temporary_spaces.max(),
  "mean_reoptimized_relative_cost": part.reoptimized_relative_cost.mean(), "mean_reoptimized_experience_loss":
  part.reoptimized_experience_loss.mean()}

```

```
pd.DataFrame([window_summary(event_result, "2026-08-06", "2026-08-10", "beidaihe_event_window"), window_summary(rain_result, "2026-07-15", "2026-07-19", "shanhaiguan_rain_window")]).to_csv(out / "q4_scenario_window_adjustment_summary.csv", index=False, encoding="utf-8-sig")

parameter_labels = {"transfer_share": "接驳分担率", "event_intensity": "活动强度", "car_share": "私家车比例", "dwell_hours": "停留时长", "rain_daily_attenuation": "单日降雨衰减"}
plt.figure(figsize=(8, 4)); plt.bar(ranking.parameter.map(parameter_labels), ranking.sensitivity_effect); plt.xticks(rotation=25, ha="right"); plt.ylabel("对最大服务缺口的影响"); plt.tight_layout(); plt.savefig(out / "q4_sensitivity_ranking.png", dpi=180); plt.close()
plt.figure(figsize=(8, 4)); plt.plot(event_result.date, event_result.baseline_service_gap, label="活动冲击基线缺口"); plt.plot(rain_result.date, rain_result.baseline_service_gap, label="降雨情景基线缺口"); plt.legend(); plt.xticks(rotation=30, ha="right"); plt.ylabel("服务缺口"); plt.tight_layout(); plt.savefig(out / "q4_scenario_service_gap.png", dpi=180); plt.close()
```

src/analysis/q4_scenario_outputs.py **功能说明：**将问题四冲击情景接入问题三资源模型并比较方案。

```
"""Bridge Q4 counterfactual shocks to Q3 resource planning models."""

from __future__ import annotations

import pandas as pd

from src.analysis.q3_absolute_resource_outputs import build_daily_absolute_plan
from src.analysis.vot_outputs import attach_vot_costs
from src.models.question3_absolute_optimizer import optimize_absolute_resources
from src.models.question4_scenarios import apply_event_pulse, service_gap_index


def compare_baseline_with_reoptimisation(baseline: pd.DataFrame, shocked: pd.DataFrame, scenario: dict[str, float], risk_penalty: float = 4.0, vot_parameters: dict[str, float] | None = None) -> pd.DataFrame:
    """Compare fixed baseline resources against a shocked, re-optimised plan."""
    merged = baseline.merge(shocked.loc[:, ["date", "region_code", "scenario_visitors"]], on=["date", "region_code"], validate="one_to_one")
    scenario_row = {**scenario, "region_code": merged.loc[0, "region_code"], "demand_scenario": "q4_counterfactual", "permanent_spaces": int(merged.loc[0, "permanent_capacity_available"]), "temporary_spaces": int(merged.loc[0, "temporary_capacity_available"])}
    demand = build_daily_absolute_plan(merged.loc[:, ["date", "region_code", "scenario_visitors"]].rename(columns={"scenario_visitors": "visitor_estimate_q50"}), pd.DataFrame([scenario_row]))
    rows = []
    for index, base in merged.iterrows():
        required = demand.iloc[index]
        gap = service_gap_index(float(required["peak_spaces_required"]), float(base["permanent_capacity_available"] + base["temporary_capacity_available"]), float(required["recommended_shuttle_vehicles"]), float(base["recommended_shuttle_vehicles"]), float(required["recommended_total_staff_shifts"]), float(base["recommended_total_staff_shifts"]))
        chosen, _ = optimize_absolute_resources(int(required["peak_spaces_required"]), int(base["permanent_capacity_available"]), int(base["temporary_capacity_available"]), int(required["recommended_shuttle_vehicles"]), int(required["recommended_total_staff_shifts"]), risk_penalty)
        baseline_parking_unserved = max(int(required["peak_spaces_required"]) - int(base["permanent_capacity_available"]) - int(base["temporary_capacity_available"]), 0)
        baseline_shuttle_shortfall = max(int(required["recommended_shuttle_vehicles"]) - int(base["recommended_shuttle_vehicles"]), 0)
        baseline_staff_shortfall = max(int(required["recommended_total_staff_shifts"]) - int(base["recommended_total_staff_shifts"]), 0)
        rows.append({"date": base["date"], "region_code": base["region_code"], "baseline_visitors": base["visitor_estimate_q50"], "scenario_visitors": base["scenario_visitors"], "baseline_service_gap": gap, "baseline_shuttle_vehicles": base["recommended_shuttle_vehicles"], "reoptimized_shuttle_vehicles": chosen["selected_shuttle_vehicles"], "baseline_staff_shifts": base["recommended_total_staff_shifts"], "reoptimized_staff_shifts": chosen["selected_staff_shifts"], "reoptimized_temporary_spaces": chosen["selected_temporary_spaces"], "reoptimized_relative_cost": chosen["relative_operating_cost"], "reoptimized_experience_loss": chosen["standardized_experience_loss"], "parking_unserved_spaces": chosen["parking_unserved_spaces"], "shuttle_shortfall": chosen["shuttle_shortfall"], "staff_shortfall":
```

```
    chosen["staff_shortfall"], "baseline_parking_unserved_spaces": baseline_parking_unserved, "baseline_shuttle_shortfall":
    baseline_shuttle_shortfall, "baseline_staff_shortfall": baseline_staff_shortfall, "recommended_shuttle_vehicles":
    required["recommended_shuttle_vehicles"], "recommended_total_staff_shifts": required["recommended_total_staff_shifts"],
    "average_party_size": scenario["average_party_size"], "transfer_share": scenario["transfer_share"], "visitor_estimate_q50":
    base["scenario_visitors"], "scenario_status": "counterfactual_planning_simulation"})
    result = pd.DataFrame(rows)
    if vot_parameters is not None:
        parameters = dict(vot_parameters)
        parameters.setdefault("vot_scenario", "central")
        baseline_input = result.assign(
            parking_unserved_spaces=result["baseline_parking_unserved_spaces"],
            shuttle_shortfall=result["baseline_shuttle_shortfall"],
            staff_shortfall=result["baseline_staff_shortfall"],
        )
        baseline_vot = attach_vot_costs(baseline_input, pd.DataFrame([parameters]))
        result = attach_vot_costs(result, pd.DataFrame([parameters]))
        result["baseline_traveler_time_loss_hours"] = baseline_vot["traveler_time_loss_hours"]
        result["baseline_traveler_time_loss_cny"] = baseline_vot["traveler_time_loss_cny_central"]
        result = result.rename(columns={"traveler_time_loss_hours": "reoptimized_traveler_time_loss_hours",
    "traveler_time_loss_cny_central": "reoptimized_traveler_time_loss_cny"})
    return result


def one_at_a_time_sensitivity(baseline: pd.DataFrame, scenario: dict[str, float], event_date: str, ranges: dict[str,
    tuple[float, float]]) -> pd.DataFrame:
    """Recompute the event scenario at low/high values for each named parameter."""
    records = []
    for parameter, (low, high) in ranges.items():
        gaps = []
        for value in (low, high):
            adjusted = dict(scenario)
            intensity = 0.4
            if parameter == "event_intensity":
                intensity = value
            else:
                adjusted[parameter] = value
            shocked = apply_event_pulse(baseline, "visitor_estimate_q50", event_date, intensity)
            comparison = compare_baseline_with_reoptimisation(baseline, shocked, adjusted)
            gaps.append(float(comparison["baseline_service_gap"].max()))
        records.append({"parameter": parameter, "low_value": low, "high_value": high, "low_max_service_gap": gaps[0],
    "high_max_service_gap": gaps[1], "sensitivity_effect": abs(gaps[1] - gaps[0])})
    result = pd.DataFrame(records)
    result["ranking"] = result["sensitivity_effect"].rank(method="dense", ascending=False).astype(int)
    return result.sort_values(["ranking", "parameter"]).reset_index(drop=True)
```

src/models/question4_scenarios.py **功能说明：**生成连续降雨、文化活动等反事实客流冲击及服务缺口。

```
"""Counterfactual visitor shocks and baseline-plan adequacy measures for Q4."""

from __future__ import annotations

import pandas as pd


def _check_column(frame: pd.DataFrame, column: str) -> None:
    if "date" not in frame.columns or column not in frame.columns:
        raise ValueError("frame must contain date and visitor column")
    if (frame[column] < 0).any():
        raise ValueError("visitor values must be non-negative")


def apply_event_pulse(frame: pd.DataFrame, visitor_column: str, event_date: str, intensity: float) -> pd.DataFrame:
```

```
    """Apply a five-day 0.25/0.50/1.00/0.50/0.25 event impulse."""
    _check_column(frame, visitor_column)
    if intensity < 0:
        raise ValueError("intensity must be non-negative")
    result = frame.copy()
    dates = pd.to_datetime(result["date"])
    offset = (dates - pd.Timestamp(event_date)).dt.days
    pulse = offset.map({-2: 0.25, -1: 0.50, 0: 1.00, 1: 0.50, 2: 0.25}).fillna(0.0)
    result["scenario_visitors"] = result[visitor_column] * (1 + intensity * pulse)
    result["scenario_type"] = "cultural_tourism_event"
    return result


def apply_continuous_rain(frame: pd.DataFrame, visitor_column: str, start_date: str, duration_days: int, daily_attenuation: float) -> pd.DataFrame:
    """Apply cumulative visitor attenuation across consecutive rainy days."""
    _check_column(frame, visitor_column)
    if duration_days <= 0 or not 0 <= daily_attenuation <= 1:
        raise ValueError("duration_days must be positive and daily_attenuation in [0, 1]")
    result = frame.copy()
    offset = (pd.to_datetime(result["date"]) - pd.Timestamp(start_date)).dt.days
    rainy_day = offset.where(offset.between(0, duration_days - 1), -1)
    multiplier = 1 - daily_attenuation * (rainy_day + 1)
    multiplier = multiplier.where(rainy_day >= 0, 1.0).clip(lower=0)
    result["scenario_visitors"] = result[visitor_column] * multiplier
    result["scenario_type"] = "continuous_rain"
    return result


def service_gap_index(parking_required: float, parking_available: float, shuttle_required: float, shuttle_available: float, staff_required: float, staff_available: float) -> float:
    """Return the equally weighted mean proportional shortfall across services."""
    values = (parking_required, parking_available, shuttle_required, shuttle_available, staff_required, staff_available)
    if any(value < 0 for value in values):
        raise ValueError("service requirements and availability must be non-negative")
    gaps = [max(required - available, 0) / max(required, 1) for required, available in ((parking_required, parking_available), (shuttle_required, shuttle_available), (staff_required, staff_available))]
    return sum(gaps) / len(gaps)
```